

REPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR
UNIVERSITE DE BAMENDA



REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
MINISTRY OF HIGHER
EDUCATION
UNIVERSITY OF BAMENDA



**INSTITUT DES SCIENCES ECONOMIQUES ET
INFORMATIQUE DE GESTION**

AUT N° 05/2002/MINESUP du 03 Janvier
2005/2006/2011/2014/2015

MÉMOIRE DE MASTER 2

SPECIALITE : GENIE LOGICIEL ET MANAGEMENT DES DONNEES

**MISE EN ŒUVRE D'UN OUTIL D'AIDE A LA
MIGRATION POUR LES ENVIRONNEMENTS
CONTENEURISES ET LEURS
ADMINISTRATIONS**

Rédigé et soutenu par :

MOHAMADOU DAHIROU

Matricule : 241GLO047M1

Directeur du memoire :

Dr MOYOU LEONEL

Encadreur professionnel :

Ing MBO'O Jean Claude

Année academique : 2025/2026

SOMMAIRE

DEDICACES.....	ii
REMERCIEMENTS.....	iii
LISTE DES ABREVIATIONS	iv
LISTE DES FIGURES	v
LISTE DES TABLEAUX.....	vi
RESUME.....	vii
ABSTRACT	viii
CHAPITRE 1 : INTRODUCTION GENERALE.....	1
1.1 CONTEXTE ET JUSTIFICATION	1
1.2 PROBLEMATIQUE ACTUELLE	3
1.3 QUESTIONS DE RECHERCHE	7
1.4 OBJECTIFS DE L'ETUDE	8
1.5 LES PROPOSITIONS DE RECHERCHE.....	11
1.6 INTERET DE L'ETUDE.....	13
1.7 APPROCHE METHODOLOGIQUE.....	16
1.8 SCOPE ET LIMITES DE L'ETUDE	18
1.9 PLAN DU TRAVAIL.....	20
CHAPITRE 2 : REVUE DE LA LITTERATURE	24
2.1 ANALYSE DES CONCEPTS.....	24
2.2 CADRE THEORIQUE DE LA MIGRATION APPLICATIVE	31
2.3 REVUE DE LA LITTERATURE EMPIRIQUE : OUTILS DE MIGRATION ET D'ADMINISTRATION	32
CHAPITRE 3 : APPROCHE METHODOLOGIQUE	35
3.1 TYPE D'ETUDE.....	35
3.2 ETUDE CRITIQUE DE L'EXISTANT	37
3.3 MODELE DE L'ETUDE	40
3.4 CONCEPTION DE L'APPLICATION.....	42
CHAPITRE 4 : RESULTAT ET EXPERIMENTATION.....	61
4.1 EXPERIMENTATION DU MODELE.....	61
4.2 ÉTUDE DE CAS PRATIQUE ET CONTEXTUALISÉE	63
4.3 COUT DE LA SOLUTION.....	76
CHAPITRE 5 : RESUME, CONCLUSION ET RECOMMANDATIONS	81
5.1 RESUME	81
5.2 CONCLUSION	83
5.3 RECOMMANDATIONS.....	85
RÉFÉRENCES BIBLIOGRAPHIQUES	88

DEDICACES

Je dédie ce modeste travail à :

- Elhdj MOHAMADOU Aminou
- Mme NAFISSATOU Yaya

REMERCIEMENTS

Au terme de ce travail nous tenons à exprimer notre profonde gratitude envers tous ceux qui ont contribué à sa réalisation. Nous ne saurions commencer sans rendre grâce au Tout miséricordieux sans qui rien de tout ceci n'aurait pu aboutir.

Nos remerciements vont particulièrement à :

- **M. ELIE RENE LIBOM**, Promoteur et Directeur de l'Institut des Sciences Économiques et Informatique de Gestion (ISEIG) de l'Université de Bamenda, pour l'excellence du cadre académique mis à notre disposition ;
- **Dr MOYOU LIONEL**, qui, en tant qu'encadreur de ce mémoire, a guidé nos recherches avec patience, rigueur et bienveillance. Ses conseils avisés, sa disponibilité constante et son expertise technique ont été essentiels pour mener à bien la mise en œuvre de notre outil d'aide à la migration ;
- **M. NGANOMBEL GABIN**, qui, en tant qu'enseignant à l'ISEIG, nous a guidé dans le choix de ce thème passionnant. Sa disponibilité constante, la qualité de sa formation et son expertise technique nous ont été d'une grande utilité ;
- **M. MBO'O Jean Claude**, mon parrain, mon encadreur professionnel, qui m'a toujours soutenu. Un grand merci pour avoir toujours cru en moi et pour les motivations quotidiennes qui m'ont poussé à me surpasser ;
- **M. Albert NYOBE**, Ingénieur Informaticien au Cabinet Netsoft-Technology, pour sa disponibilité et les débats passionnants autour de ce thème ;
- **M. YAYA Douna**, un grand ami, pour sa présence constante, ses encouragements et sa précieuse amitié tout au long de ce parcours ;
- À l'ensemble des membres du corps enseignant et administratif de ISEIG, plus particulièrement du département de **Génie Logiciel et Management des Données**, pour la qualité des enseignements et les compétences transmises tout au long de ce cycle de Master ;
- Aux membres du jury qui ont accepté d'évaluer ce travail et de l'enrichir par leurs critiques constructives ;
- À mes camarades de promotion, pour l'esprit d'équipe, les nuits de codage partagées et les débats passionnants autour des architectures Cloud et DevOps ;
- À ma famille et mes amis, pour leur patience et leur soutien moral inestimable durant la rédaction de ce mémoire.

LISTE DES ABREVIATIONS

N°	Sigle	Définition
1	ANTIC	Agence Nationale des Technologies de l'Information et de la Communication (Cameroun)
2	API	Application Programming Interface (Interface de Programmation d'Application)
3	CI/CD	Continuous Integration / Continuous Deployment (Intégration Continue / Déploiement Continu)
4	CPU	Central Processing Unit (Processeur / Unité centrale de traitement)
5	gedeq	Application de Gestion des Equivalences des Diplomes
6	IaC	Infrastructure as Code (Infrastructure en tant que Code)
7	ISEIG	Institut des Sciences Économiques et Informatiques de Gestion
8	JDBC	Java Database Connectivity (API Java pour se connecter à des bases de données)
9	JSON	JavaScript Object Notation (Format léger d'échange de données textuelles)
10	JVM	Java Virtual Machine (Machine Virtuelle Java)
11	MVC	Modèle-Vue-Contrôleur (Pattern d'architecture logicielle)
12	POO	Programmation Orientée Objet
13	RAM	Random Access Memory (Mémoire vive de l'ordinateur)
14	SGBD	Système de Gestion de Base de Données
15	UI	User Interface (Interface Utilisateur)
16	UML	Unified Modeling Language (Langage de modélisation unifié pour la conception logicielle)
17	XML	Extensible Markup Language (Langage de balisage extensible pour structurer des données)
18	YAML	YAML Ain't Markup Language (Format de sérialisation de données lisible, très utilisé pour Docker/Kubernetes)

LISTE DES FIGURES

Figure 1: formalisme d'un diagramme de cas d'utilisation	44
Figure 2: Diagramme de cas d'utilisation du système.	45
Figure 3: Formalisme du diagramme d'activité.....	46
Figure 4 : Diagramme d'activité « Enregistrer une nouvelle application ».	48
Figure 5: Diagramme d'activité « Demarrer une application ».	49
Figure 6: Diagramme de déploiement.....	51
Figure 7: Organisation détaillé du FileSystem.....	52
Figure 8: Diagramme de classe.....	54
Figure 9: Architecture physique	55
Figure 10: modèle MVC.....	57
Figure 11: Formulaire de connexion de l'outil d'automatisation développé avec JavaFX.	65
Figure 12: Page d'accueil de l'application	66
Figure 13: Liste des applications migrées.....	66
Figure 14 : Formulaire d'enregistrement d'une application	67
Figure 15: Formulaire de detection automatique des dependences.....	67
Figure 16 :Choix de la BD.....	68
Figure 17: Formulaire d'ajout des dependences en fonction du type d'application	68
Figure 18 : Dockerfile généré par l'application	73
Figure 19 : docker-compose.yml généré.....	74
Figure 20 : Interface graphique de Portainer	75
Figure 21: Interface graphique de Grafana	76

LISTE DES TABLEAUX

Tableau 1: Tableau 1: Analyse comparative : Virtualisation vs Conteneurisation	27
Tableau 2: Synthèse des limites de l'existant et justification de notre outil	39
Tableau 3: Scenarii diagramme d'activité "Enregistrer une application"	47
Tableau 4: Scenarii diagramme d'activité démarrer une application	49
Tableau 5: Analyse comparative : Migration manuelle vs Migration automatisée par l'outil développé .	70
Tableau 6: Budget estimatif de la solution.....	79

RESUME

L'**objectif** central est de concevoir et de déployer une solution logicielle automatisée dédiée à la migration d'applications web traditionnelles, initialement hébergées sur des serveurs physiques, vers l'écosystème Docker. Cette approche vise à lever les verrous technologiques majeurs aux architectures classiques, notamment les problèmes chroniques de reproductibilité, de complexité de déploiement, de lourdeur de maintenance, de conflits de cohabitation logicielle, de sous-optimisation des ressources matérielles et de rigidité de l'administration existante tout en garantissant une équivalence fonctionnelle stricte entre l'ancienne et la nouvelle infrastructure.

La **méthodologie** repose sur une approche hybride d'ingénierie inverse qui combine l'analyse statique et l'analyse dynamique pour capturer les dépendances complexes, ainsi qu'un moteur de génération basé sur des templates et des tests itératifs validés en priorité sur l'écosystème PHP.

Les **principaux résultats** démontrent qu'en centralisant le diagnostic et la génération d'artefacts (Dockerfiles, configurations Nginx/Apache/Composer), l'outil élimine les conflits de dépendances, standardise les déploiements et livre une pile d'administration automatisée (Prometheus, Grafana). Enfin, l'étude formule trois **recommandations** majeures pour l'évolution technologique et stratégique du projet : l'**extension universelle de l'outil** via le développement de nouveaux modules d'analyse capables de prendre en charge d'autres langages patrimoniaux majeurs (Java/JEE, Python, Node.js) au-delà de l'écosystème PHP initial ; le **déploiement d'une infrastructure DevOps souveraine**, prévoyant la création d'un Registre Privé National (*Private Container Registry*) pour stocker localement les images Docker conformément aux directives de l'ANTIC, ce qui garantit la souveraineté numérique des structures étatiques, sécurise le code source et optimise la vitesse de déploiement (opérations *pull/push* locales) ; l'**automatisation de la scalabilité horizontale (Auto-scaling)** pilotée en temps réel par la télémétrie Prometheus. Ce dernier mécanisme consisterait à interconnecter les alertes d'Alertmanager avec l'orchestrateur.

ABSTRACT

The central objective is to design and deploy an automated software solution dedicated to migrating traditional web applications, initially hosted on physical servers, to the Docker ecosystem. This approach aims to overcome the major technological hurdles inherent in legacy architectures specifically, chronic issues of reproducibility, deployment complexity, maintenance overhead, software cohabitation conflicts, suboptimal hardware resource utilization, and administrative rigidity while ensuring strict functional equivalence between the legacy and the new infrastructure. **The methodology** employs a hybrid reverse-engineering approach, combining static and dynamic analysis to capture complex dependencies, alongside a template-based generation engine and iterative testing, with initial validation prioritized on the PHP ecosystem. **The main results** demonstrate that by centralizing diagnostics and artifact generation (Dockerfiles, Nginx/Apache, and Composer configurations), the tool eliminates dependency conflicts, standardizes deployments, and delivers an automated administration stack (Prometheus, Grafana). Finally, the study formulates **three (03) major recommendations** for the project's technological and strategic evolution: the universal extension of the tool through the development of new analysis modules capable of supporting other major legacy languages (Java/JEE, Python, Node.js) beyond the initial PHP ecosystem; the deployment of a sovereign DevOps infrastructure, providing for the creation of a National Private Container Registry to store images locally in accordance with ANTIC directives, thereby ensuring digital sovereignty for state structures, securing source code, and optimizing deployment speed through local pull/push operations; the automation of horizontal scalability (auto-scaling) driven in real-time by Prometheus telemetry, whereby this final mechanism would consist of interconnecting Alertmanager alerts with the orchestrator.

CHAPITRE 1 : INTRODUCTION GENERALE

CHAPITRE 1 : INTRODUCTION GENERALE

L'évolution rapide des technologies de l'information impose aux organisations une modernisation constante de leurs infrastructures logicielles afin de garantir agilité, performance et sécurité. Ce premier chapitre pose les jalons de notre étude en explicitant le contexte global et spécifique dans lequel s'inscrit ce projet, notamment la nécessité d'optimiser le cycle de vie des applications au sein des structures publiques et privées. À travers une formalisation rigoureuse de la problématique actuelle, nous identifierons les limites des architectures traditionnelles face aux exigences contemporaines. Enfin, nous énoncerons les questions de recherche, les objectifs visés, les hypothèses de travail ainsi que la délimitation du périmètre de cette étude, afin de justifier pleinement la pertinence de la solution proposée.

1.1 CONTEXTE ET JUSTIFICATION

À l'ère du **Cloud Computing** et des architectures microservices, la rapidité et la fiabilité du déploiement sont devenues des enjeux stratégiques pour l'ingénierie logicielle. Pourtant, de nombreuses entités public, privée et parapublic se heurtent encore aux limites du déploiement classique dit « Bare Metal » (ou installation directe sur l'hôte). Cette approche historique s'avère aujourd'hui inadaptée à la complexité des applications modernes, créant un fossé technologique majeur entre les systèmes hérités (*legacy*) et les standards actuels de production.

L'étude menée s'inscrit dans un contexte marqué par la complexité croissante des environnements d'exécution des applications web traditionnelles. Ces dernières, déployées historiquement sur des serveurs physiques dédiés ou sur des machines virtuelles, présentent aujourd'hui un défi majeur en termes de **portabilité**, de **scalabilité**, de **cohabitation** et de **maintenabilité** et de **reproductibilité**. La diversité des configurations système, des dépendances logicielles et des versions d'exécution rend leur migration vers des architectures modernes particulièrement ardue, lourde et fastidieuse.

Comme le soulignent **Pahl et al. (2019)**, la conteneurisation s'impose comme une réponse indispensable à ces défis : elle permet d'automatiser le déploiement et la gestion des applications

dans des environnements hétérogènes tout en réduisant drastiquement les risques liés aux incompatibilités logicielles. En encapsulant une application avec son environnement complet dans un conteneur isolé, Docker apporte une solution efficace pour garantir la cohérence et la fluidité des déploiements.

Toutefois, passer d'une application déployée de manière traditionnelle à une architecture conteneurisée impose une phase d'analyse manuelle minutieuse. Cette étape de rétro-ingénierie est indispensable pour extraire les dépendances, les configurations et les paramètres nécessaires à la génération des artefacts Docker correspondants. L'intérêt scientifique et technique de ce projet réside donc dans la conception et l'implémentation d'un outil capable d'automatiser l'intégralité de ce processus afin de minimiser les interventions humaines, souvent sources d'erreurs et de ralentissements.

Cette exigence de précision et les failles de l'écriture manuelle des configurations sont documentées par **Cito et al. (2017)**. Leurs travaux mettent en avant l'importance de la reproductibilité des environnements et démontrent, par une analyse empirique, que l'absence d'automatisation crée une dette technique critique dans les fichiers de configuration (*Dockerfiles*). Garantir la génération automatique d'artefacts (images Docker, Dockerfiles, fichiers Docker Compose) strictement identiques et fidèles à l'environnement source est donc crucial pour valider les déploiements en production sans perte d'information ni dérive de configuration (*configuration drift*).

Enfin, ce besoin d'automatisation est renforcé par l'émergence du Cloud Computing, où les applications composites doivent s'exécuter dans des environnements distribués. L'intégration des conteneurs dans ces architectures se révèle incontournable pour assurer une orchestration efficace des services. L'étude de **Kratzke (2018)** illustre parfaitement comment la combinaison de Docker avec des solutions Cloud permet de déployer des applications complexes tout en conservant une grande modularité. L'automatisation devient alors la **passerelle indispensable** pour permettre aux organisations d'intégrer efficacement leurs services existants dans ces infrastructures modernes, répondant ainsi aux besoins actuels de flexibilité et d'adaptabilité.

En somme, notre étude se situe à l'intersection de la **modernisation logicielle**, de l'**automatisation des déploiements** et de la **maîtrise des environnements applicatifs**. Face à l'obsolescence croissante des infrastructures traditionnelles et au besoin crucial de réduire les coûts

et les risques liés à la migration d'environnements anciens, nous nous proposons d'informatiser une solution logicielle innovante.

Pour ancrer ce projet de recherche dans une réalité technique concrète, **notre outil cible spécifiquement l'écosystème PHP**, un langage historiquement prédominant dans les architectures web traditionnelles et souvent synonyme de configurations complexes (modules d'extension comme pdo, mbstring, versions de serveurs web comme Apache/Nginx, et gestionnaires de dépendances comme Composer). L'outil aura pour rôle d'analyser intelligemment ces applications existantes afin de générer automatiquement tous les fichiers Docker requis pour leur conteneurisation.

Cette démarche vise à démocratiser l'accès aux architectures modernes, offrant à toutes les structures y compris celles dépourvues de compétences **DevOps** pointues la possibilité d'accélérer leur transition technologique vers le Cloud avec la rigueur, la rapidité et la reproductibilité qu'exige l'ingénierie logicielle contemporaine.

1.2 PROBLEMATIQUE ACTUELLE

Au Cameroun, la migration des applications web existantes demeure un défi majeur pour de nombreuses entités publiques, parapubliques et privées. Une grande partie du patrimoine applicatif repose encore sur des architectures traditionnelles (monolithiques ou 3-tiers), déployées directement sur des serveurs physiques (*Bare Metal*) ou des machines virtuelles avec des dépendances complexes et des configurations spécifiques à leur environnement d'origine. Ces infrastructures héritées, bien qu'historiquement fonctionnelles, présentent des limitations structurelles qui freinent l'agilité organisationnelle et l'innovation technologique notamment :

L'érosion de la reproductibilité et les serveurs « Flocons de Neige »

Le premier obstacle réside dans l'accumulation de configurations manuelles et artisanales. Comme le soulignent **Fowler et Lewis (2014)**, un serveur configuré directement sur l'hôte finit inévitablement par diverger de son état initial. Chaque mise à jour de sécurité, modification de variable d'environnement ou installation de patch non documentée crée une dérive de configuration.

Il devient alors impossible de garantir que l'environnement de développement est identique à celui de production, provoquant le syndrome bien connu du « **ça marche sur ma machine** ». En cas de panne matérielle ou logicielle, la reconstruction d'un serveur prend des heures, voire des jours, la « **recette** » d'installation n'étant jamais intégralement formalisée.

La « Matrice de l'Enfer » : Conflits et pollutions système

Dans un environnement classique, toutes les applications partagent les bibliothèques globales du système d'exploitation. Les travaux de **Cito et al. (2017)** ont mis en évidence le cauchemar logistique engendré par cette absence d'isolation.

- **Le conflit de versions (Problème de cohabitation) :** Si une **application A** requiert une bibliothèque (par exemple lib-ssl v1.1 ou une extension PHP spécifique) et qu'une **application B** exécutée sur le même hôte nécessite la v3.0, l'administrateur système fait face à une impasse technique. L'installation ou la mise à niveau de l'une peut rendre l'autre totalement instable ou inopérante.
- **La pollution globale :** Chaque outil ou dépendance installé laisse des traces permanentes sur l'hôte. À terme, le système devient lourd, encombré de packages obsolètes, ce qui augmente considérablement la surface d'attaque et les vulnérabilités de l'infrastructure.

L'opacité des interdépendances : Une architecture « Boîte Noire »

Le déploiement classique souffre d'un manque crucial de visibilité sur la topologie et les frontières applicatives. Au sein d'une même machine, la logique métier et les services de données (bases de données, caches) s'entremêlent sans cloisonnement strict. Une fuite de mémoire au niveau du code source peut saturer les ressources globales et impacter la base de données.

De plus, contrairement aux principes rigoureux édictés par **Adam Wiggins (2017)** dans le manifeste **The Twelve-Factor App**, les liens entre services sont fréquemment codés en "dur" (adresses IP statiques, chemins absolus). Cela rend toute tentative de migration extrêmement périlleuse.

La persistance de ces architectures traditionnelles engendre cinq problèmes critiques au sein des directions informatiques :

- ✓ **Problèmes de portabilité entre environnements** : En raison des disparités de configurations entre les machines de développement, les serveurs de test et les environnements de production, le transfert d'un code source validé vers la production génère des comportements imprévisibles et des régressions fréquentes.
- ✓ **Scalabilité coûteuse** : Face à une augmentation de la charge (pics de connexion sur une application de service public par exemple), la seule réponse possible dans un modèle traditionnel est le partitionnement vertical (achat de RAM/CPU supplémentaires) ou la duplication lourde de machines virtuelles entières. Cette approche de scalabilité matérielle s'avère extrêmement onéreuse et lente à implémenter.
- ✓ **Temps de déploiement longs pour les nouvelles versions** : L'absence de chaînes de livraison automatisées et la nécessité de configurer manuellement les serveurs web lors des mises à jour étirent les cycles de mise en production (parfois plusieurs semaines), ralentissant la correction de bugs et la livraison de nouvelles fonctionnalités.
- ✓ **Le problème d'administration** : La gestion quotidienne d'un parc de serveurs hétérogènes et non standardisés exige une charge de travail humain disproportionnée de la part des administrateurs. Les tâches de supervision, de sauvegarde et de mise à jour deviennent chronophages et sujettes aux erreurs de manipulation.
- ✓ **Problème d'optimisation des ressources utilisées** : Les applications traditionnelles s'approprient les ressources d'une machine virtuelle ou d'un serveur physique de manière exclusive. Il en résulte un faible taux de mutualisation : certains serveurs fonctionnent à 10% de leurs capacités réelles, entraînant un gaspillage de ressources informatiques et énergétiques.

Ainsi, la problématique centrale de ce travail repose sur la complexité intrinsèque à la migration automatisée d'applications web traditionnelles vers des environnements conteneurisés, une démarche qui requiert une analyse détaillée et précise de plusieurs dimensions techniques. En effet, bien que la conteneurisation via Docker offre un cadre standardisé pour la **portabilité** et la **reproductibilité**, elle n'efface pas les disparités profondes existant dans la configuration et le fonctionnement des applications conçues initialement pour des serveurs physiques ou des machines virtuelles. Cet écart entre deux paradigmes de déploiement complexifie fortement le processus d'automatisation, ce qui soulève la question suivante : **comment concevoir un outil capable d'identifier et d'extraire automatiquement toutes les dépendances logicielles, les paramètres de configuration, ainsi que les spécificités d'exécution nécessaires à la génération d'artefacts Docker fidèles à l'application source ainsi que son administration ?**

Cette interrogation met en lumière plusieurs défis techniques majeurs. Premièrement, la nature hétérogène des applications web classiques incluant des stacks variés (par exemple, des bases de données locales, des middlewares, des frameworks web, ou des configurations système spécifiques) rend la reprise des informations complexe. La simple inspection des fichiers sources ou des configurations système ne garantit pas une extraction exhaustive et correcte, d'autant plus que certains paramètres peuvent être définis dynamiquement ou dépendent d'environnements particuliers. Deuxièmement, la génération d'artefacts Docker adaptés requiert de traduire ces configurations traditionnelles dans un format déclaratif et reproductible (Dockerfile, images, Docker Compose) tout en assurant que l'environnement conteneurisé offre une équivalence fonctionnelle parfaite avec l'existant. Cela suppose une modélisation précise de l'environnement d'exécution et la prise en compte des interactions entre composants, qui peuvent être parfois implicites dans une infrastructure non conteneurisée.

Par ailleurs, la réduction au minimum des interventions manuelles constitue une exigence fondamentale pour garantir la fiabilité du processus. En effet, toute retouche ou ajustement manuel des artefacts produits introduit non seulement un facteur d'erreur, mais compromet également la reproductibilité et la traçabilité des environnements, deux qualités essentielles dans les cycles modernes de développement et de livraison continue. Automatiser ce processus impose ainsi une compréhension fine du comportement des applications en situation réelle, notamment pour détecter les dépendances cachées ou non documentées, ce qui constitue un enjeu à la fois algorithmique et pragmatique.

Du point de vue pratique, cette problématique s'inscrit plus largement dans les transformations profondes des pratiques de déploiement logicielle, caractérisées par une volonté accrue de standardisation et d'orchestration dans des infrastructures souvent hybrides, mêlant environnements physiques, virtuels et cloud. La capacité à migrer aisément des applications vers Docker contribue à la flexibilité opérationnelle des entreprises et favorise l'adoption de paradigmes agiles et scalables. Ce besoin d'automatisation est d'ailleurs renforcé par les enjeux industriels liés à la gestion de la complexité, à l'accélération des phases de test, et à la garantie de cohérence entre environnements de développement, de pré-production et de production.

Les travaux antérieurs, tels que ceux présentés par **Dirk Merkel (2014)**, soulignent le rôle stratégique de Docker pour automatiser le déploiement dans des environnements hétérogènes, mais mettent aussi en exergue la difficulté de concevoir des outils d'analyse efficaces qui couvrent l'ensemble des composantes applicatives. Par ailleurs, la recherche menée dans le domaine de la

science des données par **Boettiger (2015)** rappelle l'importance cruciale de la reproductibilité, positionnant ainsi la génération automatisée d'artefacts Docker non seulement comme un enjeu de portabilité mais aussi comme un vecteur garant de qualité scientifique et technique. Enfin, l'intégration de Docker dans des architectures cloud distribuées, étudiée par **Pahl et Lee (2015)**, met en évidence la complexité supplémentaire induite par la nature composite et modulaire des applications modernes, renforçant la nécessité de mécanismes automatisés d'analyse et de création d'artefacts adaptables à différents contextes d'exécution.

En somme, la problématique qui oriente ce travail articule des exigences techniques pointues liées à la compréhension contextuelle des applications web traditionnelles, la restitution fidèle dans un modèle conteneurisé, et l'automatisation poussée pour minimiser les erreurs humaines. Elle s'inscrit comme une réponse innovante aux défis actuels des équipes d'ingénierie logicielle confrontées à la nécessité incontournable de modernisation des infrastructures, dans un univers informatique marqué par l'hétérogénéité croissante des environnements et la recherche continue d'efficacité opérationnelle. La réalisation d'un tel outil représente ainsi une contribution essentielle à la fois pour la science du logiciel et pour les pratiques industrielles.

1.3 QUESTIONS DE RECHERCHE

Pour répondre à ces difficultés et éviter les erreurs humaines liées aux tâches manuelles, il est nécessaire de définir des objectifs clairs. Afin de concevoir efficacement notre outil et de guider notre travail, nous avons organisé notre réflexion autour de plusieurs problèmes précis. Les questions de recherche suivantes découlent directement des défis présentés plus haut. Elles visent à encadrer le développement de notre application d'automatisation de la conteneurisation:

- Comment mettre en œuvre un mécanisme d'extraction exhaustif et fiable capable de cartographier les dépendances logicielles, les configurations et les particularités d'exécution d'une application existante, en surmontant les disparités et l'hétérogénéité des infrastructures d'origine ?
- Comment formaliser les règles de correspondance (mapping) permettant de traduire ces configurations hétérogènes en artefacts Docker (Dockerfiles et manifests Docker Compose) optimisés, sécurisés et respectant l'équivalence fonctionnelle de l'application source ?

- Comment automatiser au maximum les processus d'analyse, de génération et de validation de ces artefacts, tout en intégrant des boucles de rétroaction semi-assistées pour détecter et corriger les incohérences ou ambiguïtés que l'outil ne peut résoudre seul ?
- Comment intégrer au sein de cet outil d'aide des mécanismes d'administration et de supervision automatisés (monitoring, collecte de métriques et gestion des logs) afin de standardiser et d'alléger la gestion quotidienne du nouvel environnement conteneurisé par les administrateurs système ?

Ces questions de recherche forment un cadre conceptuel et méthodologique qui oriente toutes les étapes de développement envisagées. Elles permettent de structurer la démarche scientifique autour d'objectifs clairs : identification et compréhension des configurations applicatives, traduction précise vers des artefacts Docker et orchestration associée, automatisation robuste avec une intervention humaine limitée, ainsi qu'adaptabilité aux environnements d'exécution multiples. Ce cadre nourrira le choix des techniques à mettre en œuvre, qu'il s'agisse d'ingénierie logicielle, d'analyse statique et dynamique, ou d'intégration continue, afin de concevoir un outil pertinent et innovant répondant à la problématique exposée. En ce sens, elles traduisent la complexité des dimensions à maîtriser, tout en mettant en lumière les perspectives d'innovation scientifique et technique ouvertes par le projet.

1.4 OBJECTIFS DE L'ETUDE

Les objectifs de cette étude se déploient naturellement à partir des questions de recherche précédemment détaillées, en traduisant de manière concrète les interrogations théoriques en résultats opérationnels clairement définis.

- ✓ **Le premier objectif réside dans la conception d'un outil automatisé capable d'analyser en profondeur des applications web dites « traditionnelles », déployées sur des infrastructures diverses comprenant des serveurs physiques et des machines virtuelles.**

Cette analyse doit être exhaustive et fiable, permettant d'extraire avec une précision maximale les dépendances, configurations, et spécificités fonctionnelles nécessaires à la reconstruction de l'environnement applicatif.

Par exemple, l'outil devra intégrer des mécanismes combinant des approches statiques telles que le parsing des fichiers de configuration et la lecture des scripts et dynamiques notamment la surveillance du comportement des processus en exécution afin de pallier les limitations de chacune de ces méthodes prises isolément. Comme l'ont souligné **Vasilakis et al. (2019)**, l'isolation des dépendances implicites au sein des architectures web *legacy* représente le principal point de défaillance des migrations manuelles, validant ainsi la nécessité d'une approche d'analyse hybride et automatisée. Cette première étape conditionne la qualité des phases ultérieures, puisqu'une connaissance fine des éléments constitutifs de l'application est indispensable pour générer des artefacts Docker pertinents et reproductibles, conformément aux exigences formulées dans la problématique centrale.

- ✓ **Le second objectif porte sur la génération automatique et rigoureuse des artefacts Docker, comprenant non seulement la création de Dockerfiles adaptés, mais également la production d'architectures complexes orchestrées via des fichiers Docker Compose ou équivalents.**

Cette phase englobe la formalisation et la traduction des configurations hétérogènes issues des environnements initiaux vers un format standardisé, reconnu et exploitable dans le cadre d'une conteneurisation pérenne. Il importe ici d'assurer une correspondance fonctionnelle parfaite entre les applications originales et leurs versions conteneurisées, ce qui implique la gestion fine des versions logicielles, des variables d'environnement, des scripts de démarrage, ainsi que des réseaux et volumes associés. L'objectif n'est pas strictement technique : il s'agit aussi d'intégrer des bonnes pratiques robustes qui garantissent la portabilité, la sécurité et la maintenabilité des artefacts créés.

À cet égard, l'étude empirique de **Cito et al. (2017)** démontre la forte prévalence des *anti-patterns* et des failles de sécurité dans les fichiers de configuration rédigés manuellement, ce qui justifie la mise en œuvre d'une automatisation basée sur des modèles de qualité validés. Par ailleurs, cette étape soulève la nécessité d'adapter les sorties de l'outil aux différentes contraintes spécifiques des plateformes cibles, afin d'assurer une exploitation optimale quelle que soit l'infrastructure sous-jacente.

- ✓ **Un troisième objectif est la réduction drastique de l'intervention humaine dans le processus global, particulièrement lors des phases d'analyse, de génération et de validation des artefacts Docker.**

L'ambition est d'atteindre un degré d'automatisation avancé, ce qui implique la conception et l'intégration de mécanismes intelligents capables de détecter automatiquement les incohérences ou anomalies, de proposer des corrections ou ajustements, et de guider l'utilisateur de manière ergonomique lorsque son intervention s'avère inévitable. Cette automatisation vise à accroître la fiabilité, la reproductibilité ainsi que la traçabilité du processus de migration, réduisant ainsi les risques d'erreurs humaines et accélérant les cycles de développement. L'intégration fluide de cet outil au sein des pipelines d'intégration et de déploiement continus (CI/CD) constitue également une priorité. L'unification de l'infrastructure sous forme de code et son intégration dans les cycles de livraison est d'ailleurs théorisée par **Wettinger et al. (2015)** comme le fondement de l'efficacité opérationnelle moderne, permettant une exploitation harmonieuse en contexte DevOps et d'optimiser l'adoption en milieu professionnel.

- **Le quatrième objectif consiste à valider l'équivalence fonctionnelle entre l'ancienne et la nouvelle application conteneurisée.**

La simple création automatique des fichiers Docker ne suffit pas si l'application finale présente des dysfonctionnements ou des erreurs par rapport à sa version initiale. Cet objectif consiste donc à intégrer dans notre outil un module de tests. Ce module aura pour rôle de comparer précisément le comportement de l'ancienne application avec celui de sa nouvelle version en conteneur. Il s'agira d'exécuter des scénarios de test (comme des requêtes HTTP, des vérifications de connexion aux bases de données ou le lancement de traitements métiers) pour s'assurer que le passage aux conteneurs n'a pas modifié la logique ni les performances du système. En utilisant ces tests comparatifs, cet objectif permet d'apporter une garantie concrète et chiffrée aux directions informatiques, assurant ainsi une migration sécurisée et sans interruption de service.

- ✓ **Le cinquième objectif est l'intégration d'un système automatisé d'administration, de monitoring et de supervision post-migration**

Il consiste à doter l'outil d'un module d'administration et de supervision automatisé, indispensable pour pérenniser l'exploitation des applications une fois migrées vers le nouvel environnement conteneurisé. Il s'agit de concevoir et de déployer automatiquement une pile de monitoring standardisée (comprenant la collecte de métriques système via Prometheus, la visualisation des performances sur des tableaux de bord Grafana et la centralisation des logs) parallèlement à la génération des artefacts applicatifs. Cet objectif vise à résoudre directement la complexité liée à la gestion d'un parc conteneurisé, en fournissant aux administrateurs système une

visibilité complète et centralisée sur l'état de santé, la consommation des ressources et la topologie des conteneurs. En s'appuyant sur les travaux de *Kratzke (2018)* sur l'orchestration logicielle, cette automatisation de l'administration permet d'alléger la charge de travail humaine disproportionnée des équipes d'exploitation, de standardiser les tâches de maintenance quotidiennes et de garantir une réactivité optimale en cas d'anomalie en production

Ainsi, les objectifs poursuivis structurent une démarche holistique et rigoureuse, articulant une exploration technique approfondie, une automatisation fine des processus, l'assurance formelle de l'équivalence fonctionnelle et une intégration pérenne dans les pratiques industrielles actuelles. Cette ambition vise à répondre aux besoins clé identifiés dans la problématique initiale, en posant les bases d'une solution innovante qui conjugue efficacité, fiabilité et évolutivité dans la gestion automatisée de la containerisation des applications web traditionnelles. La nature interdisciplinaire de cette étude souligne la nécessité d'une expertise conjuguant analyse logicielle, génie des systèmes, génie logiciel agile et architecture cloud. Comme le démontrent **Kratzke et Quint (2017)** dans leur cartographie de l'écosystème Cloud-Native, la refonte des systèmes monolithiques vers les conteneurs ne relève pas de la simple administration système, mais exige la convergence de ces disciplines pour produire un outil méthodologiquement solide et adapté au contexte dynamique du génie logiciel contemporain.

1.5 LES PROPOSITIONS DE RECHERCHE

Dans le cadre de ce travail, nous cherchons à créer un outil intelligent capable de simplifier et de sécuriser le transfert des applications traditionnelles vers des technologies modernes. A cet effet, pour guider la construction de notre logiciel et évaluer son efficacité sur le terrain, nous formulons quatre (04) **propositions de recherche** adaptées à notre démarche pratique :

✓ **PR1 : Le diagnostic complet pour la detection des dependences**

Pour copier fidèlement une application sans rien oublier, l'outil doit à la fois lire ses fichiers de codes cachés (l'analyse au repos) et observer l'application pendant qu'elle fonctionne (l'analyse en mouvement).

Les serveurs informatiques actuels ressemblent souvent à des recettes de cuisine modifiées au fil des ans sans que personne n'ait noté les changements. Si on se contente de lire la recette d'origine (le code), on rate les ingrédients secrets ajoutés en cours de route. En combinant la lecture

des documents et l'observation en direct de l'application, notre outil obtient une liste à puce parfaite et complète de tout ce dont l'application a besoin pour fonctionner.

✓ **PR2 : Le traducteur automatique (La garantie de bon fonctionnement)**

En transformant automatiquement les anciennes configurations artisanales en fichiers d'instructions standards et universels (le format Docker), l'outil garantit que l'application fonctionnera exactement de la même manière partout, sans bug de transfert.

Aujourd'hui, déplacer une application d'un ordinateur à un autre provoque souvent des pannes parce que les machines n'ont pas les mêmes composants. Notre outil agit comme un traducteur : il prend l'organisation complexe de l'ancien serveur et la traduit dans un plan standardisé. L'application se retrouve alors enfermée dans une "boîte numérique" (un conteneur) isolée du reste du système, ce qui empêche les applications de se polluer entre elles et garantit qu'elle marchera à tous les coups.

✓ **PR3 : L'automatisation intelligente et l'assistance semi-assistée comme vecteurs de réduction drastique de l'intervention humaine**

L'implémentation d'algorithmes d'analyse hybrides et de modules de test autonomes, combinée à un mécanisme d'assistance semi-automatisée, réduit de manière drastique l'intervention humaine lors des phases d'analyse, de génération et de validation des artefacts Docker, tout en élevant la fiabilité et la traçabilité des environnements migrés par rapport aux méthodes de migration artisanales.

Une automatisation totale (à 100 %) est irréaliste face à l'hétérogénéité des applications héritées, où des adresses IP ou des chemins absolus peuvent être codés en dur, violant les principes du manifeste *The Twelve-Factor App* (Wiggins, 2017) mais toute fois, notre outil prend en charge la quasi-totalité du travail rébarbatif (diagnostic du serveur, écriture du code Docker et tests de bon fonctionnement) pour le réduire à de simples clics de validation.

✓ **PR4 : Le tableau de bord simplifié pour l'administration (La gestion facile après la migration)**

En installant automatiquement des écrans de contrôle visuels et des alertes en même temps que la migration, l'outil permet à des techniciens non spécialisés de surveiller et de gérer facilement les applications.

Déplacer une application dans une technologie moderne sans outils de surveillance, c'est comme conduire une voiture sans tableau de bord : on ne sait pas si on va tomber en panne d'essence. Notre outil n'offre pas seulement une application migrée ; il livre en même temps une panoplie d'écrans graphiques (comme Grafana et Portainer). Même un informaticien débutant peut voir d'un seul coup d'œil si le système est fatigué, recevoir des alertes automatiques en cas de panne et régler les problèmes sans effort.

1.6 INTERET DE L'ETUDE

L'intérêt de cette étude découle directement de la nécessité grandissante de moderniser les architectures applicatives héritées pour répondre aux enjeux actuels d'agilité, de scalabilité et de maintenabilité dans les infrastructures informatiques. La transition vers des environnements conteneurisés, telle que proposée par Docker, constitue une évolution majeure dans le domaine du génie logiciel, notamment pour les applications web traditionnelles tournant sur serveurs physiques ou machines virtuelles. Or, cette migration soulève des problématiques techniques complexes, notamment en termes d'automatisation, d'extraction précise des dépendances et configurations, ainsi que d'assurance d'une équivalence fonctionnelle stricte entre les environnements source et cible. Le travail proposé apporte une contribution substantielle en adressant la conception et l'implémentation d'un outil automatisé capable de relever ces défis, permettant ainsi de réduire considérablement les interventions manuelles et les risques d'erreurs humaines liées aux migrations manuelles.

La pertinence de cette étude peut être mise en relation avec les limites souvent rencontrées dans les démarches de migration actuelles. En effet, la majorité des solutions existantes reposent sur des processus semi-automatisés ou manuels, ce qui engendre des coûts opérationnels élevés, des délais prolongés et une faible reproductibilité des configurations déployées. **Vasilakis et al. (2019)** ont notamment mis en lumière que l'absence d'outils d'extraction holistiques conduit fréquemment à des échecs de conteneurisation en raison de dépendances implicites ignorées. Cette étude vise donc à instaurer une méthodologie robuste et à unifier les phases d'analyse, d'extraction et de génération d'artefacts Docker dans un flux unifié, intégré et automatisé. Ainsi, elle répond à un besoin réel observé dans les entreprises qui cherchent à tirer parti des bénéfices du cloud

computing hybride et des conteneurs tout en préservant la continuité des services applicatifs. En améliorant la fiabilité et la reproductibilité des environnements migrés, elle ouvre la voie à une industrialisation plus poussée des processus DevOps et *Infrastructure as Code* (IaC), domaines étant actuellement au cœur des pratiques modernes de déploiement logiciel.

Par ailleurs, cette étude vise à combler une lacune importante dans la chaîne de déploiement des applications contemporaines. Alors que la conteneurisation, notamment via Docker, a transformé les pratiques d'intégration continue et de livraison continue, la migration d'applications préexistantes demeure largement sous-automatisée. L'outil envisagé ambitionne de standardiser et de sécuriser cette transition, en s'appuyant sur une extraction exhaustive des dépendances et des paramètres de configuration, qui jusqu'ici requérait une expertise manuelle approfondie et souvent coûteuse en temps. Cette approche s'inscrit donc dans un effort d'industrialisation et d'optimisation des flux DevOps, améliorant non seulement la rapidité des déploiements mais aussi leur fiabilité, comme le soulignent les travaux de **Wettinger et al. (2015)** sur la nécessité de rapprocher l'automatisation des opérations au plus près du code applicatif sous forme d'artefacts standardisés.

L'enjeu de la reproductibilité, dans le cadre de cette recherche, revêt une importance cruciale. Garantir qu'un environnement conteneurisé reproduise fidèlement le comportement fonctionnel et les performances d'une application déployée dans son contexte originel est une garantie indispensable pour éviter les régressions et les interruptions de service. Comme le démontre l'analyse empirique de **Cito et al. (2017)**, la rédaction manuelle et non vérifiée d'artefacts de déploiement mène à une dette technique critique et à des failles de sécurité structurelles. L'automatisation permet ici non seulement d'assurer une cohérence technique grâce à la génération systématique de Dockerfiles et de configurations Docker Compose, mais aussi d'instaurer une démarche traçable, où chaque artefact produit est le fruit d'un processus rigoureux de validation. Cette dimension est en adéquation avec les principes modernes de génie logiciel focalisés sur la qualité logicielle et la robustesse des infrastructures.

En outre, la montée en maturité des technologies Docker ainsi que leur adoption généralisée dans les environnements cloud hybrides confèrent à cette étude une actualité et une valeur stratégique indéniables. En s'appuyant sur des fondations technologiques solides, cette recherche vise à repousser les limites de l'automatisation en proposant un prototype performant et adaptable, s'inscrivant dans le prolongement de l'état de l'art établi par **Pahl et al. (2018)**, qui a démontré l'intérêt des conteneurs comme couche d'abstraction fondamentale pour garantir la portabilité et

simplifier les cycles de vie des systèmes complexes. L'intégration de ces technologies au sein de l'outil envisagé permettrait de capitaliser sur des mécanismes éprouvés tout en répondant à une problématique encore peu explorée dans sa globalité : la migration complète et automatisée d'applications classiques vers des conteneurs avec une garantie d'équivalence fonctionnelle, ce qui constitue un enjeu crucial pour la modernisation des systèmes d'information.

Enfin, la mise en place d'un tel outil ouvre des perspectives de recherche et d'innovation importantes en génie logiciel, notamment dans la conception d'algorithmes intelligents capables d'analyser automatiquement et finement les applications web existantes. Cette étude permet ainsi de contribuer à l'avancement des méthodologies de *reverse engineering* applicatif et d'orchestration automatisée. À ce sujet, **Kratzke et Quint (2017)** rappellent que la transition vers le paradigme *cloud-native* exige une approche interdisciplinaire rigoureuse, combinant l'analyse de code et le génie des systèmes pour éviter les ruptures opérationnelles. De plus, le caractère reproductible et paramétrable des artefacts générés favorise le renforcement des pratiques DevOps et le développement de chaînes d'intégration et déploiement continus plus robustes. En ce sens, l'étude joue un rôle catalyseur en valorisant des mécanismes d'automatisation avancés qui pourraient être étendus à d'autres types d'applications ou contextes techniques, participant à la généralisation d'une approche standardisée et pragmatique de migration vers les environnements conteneurisés.

Ainsi, l'intérêt de cette étude réside dans sa capacité à combiner rigueur méthodologique, pertinence industrielle et innovation technologique pour proposer une solution concrète au défi de migration automatisée d'applications web traditionnelles, répondant à une problématique actuelle majeure du génie logiciel. Il s'agit non seulement de proposer un outil efficace, fiable et adaptable, mais également de fournir une réflexion approfondie sur les conditions nécessaires à la réussite de telles migrations, en tenant compte des contraintes fonctionnelles, techniques et organisationnelles inhérentes aux environnements cibles. Cette double perspective théorique et pratique place cette étude au cœur des avancées en matière de modernisation applicative, avec une portée potentiellement significative pour les professionnels et chercheurs du domaine.

En intégrant la dimension d'un outil d'aide automatisé, cette étude ne se contente plus d'analyser **comment** migrer, mais se concentre sur **comment optimiser et industrialiser** ce processus de migration. Cela en fait un sujet d'une grande valeur pratique et théorique, susceptible d'apporter des contributions significatives à la recherche et au développement dans le domaine de l'ingénierie logicielle et des opérations informatiques modernes.

1.7 APPROCHE METHODOLOGIQUE

L'approche méthodologique adoptée pour concevoir et implémenter un outil automatisé destiné à la migration d'applications web traditionnelles vers des environnements conteneurisés repose sur une démarche systématique, articulée autour de plusieurs phases complémentaires, visant à garantir à la fois une analyse précise des systèmes sources et une génération fiable des artefacts Docker adéquats (Dockerfiles, images, Docker Compose). Cette méthodologie s'inscrit dans la continuité logique des problématiques identifiées précédemment, notamment la nécessité de maîtriser la complexité d'extraction des dépendances et configurations tout en assurant l'équivalence fonctionnelle et la reproductibilité des déploiements.

Le premier socle méthodologique consiste en une étude approfondie des différentes architectures applicatives visées, incluant les applications web déployées sur serveurs physiques et machines virtuelles. Cette étape initiale vise à modéliser avec rigueur les éléments constitutifs des applications, qu'il s'agisse des bibliothèques, des services, des fichiers de configuration ou des interconnexions entre composants. Ces travaux s'appuient sur des approches de reverse engineering adaptées, qui ont pour objectif d'automatiser la collecte d'informations souvent hétérogènes et dispersées dans les systèmes sources. Par exemple, l'analyse des fichiers de configuration, qu'ils soient au format XML, YAML, JSON ou script shell, nécessite des parsers spécifiques et une normalisation des données extraites pour faciliter leur traitement ultérieur. Cette phase est essentielle pour limiter les interventions humaines et réduire les risques d'erreurs fréquent dans les migrations manuelles.

Une fois les données collectées et structurées, la méthodologie prévoit une étape d'évaluation et de validation formelle des dépendances et des configurations extraites. Cette validation se base sur des règles métier et des critères techniques garantissant que les artefacts Docker générés respecteront non seulement la structure fonctionnelle de l'application, mais également ses contraintes non fonctionnelles, telles que la sécurité, la performance et la conformité aux normes industrielles. Par exemple, la méthodologie intègre la possibilité d'incorporer automatiquement des règles de sécurité spécifiques à certains domaines d'application, renforçant ainsi la pertinence de la solution dans un contexte métier réel et évitant des écarts fonctionnels post-migration. Cette étape inclut également des tests automatisés d'intégration visant à simuler les scénarios d'utilisation pour s'assurer que la version conteneurisée correspond rigoureusement

à la version originale.

Par ailleurs, la génération des artefacts Docker, qu'il s'agisse de *Dockerfiles*, d'images ou de fichiers Docker Compose, constitue un enjeu méthodologique majeur. Pour cela, l'étude s'appuie sur une approche de génération automatique basée sur des templates adaptatifs et des règles de transformation permettant de traduire les configurations natives des applications en instructions Docker standardisées. Cette approche s'inspire notamment des méthodologies décrites dans la littérature et les pratiques industrielles visant à automatiser le cycle de vie des conteneurs, à l'instar des travaux de **Wettinger et al. (2015)** qui soulignent l'intérêt des conteneurs Docker pour standardiser et automatiser le déploiement d'applications web complexes au plus près du code source. En outre, les principes d'intégration continue et de déploiement continu (CI/CD) sont intégrés dans le processus afin d'assurer une industrialisation complète, évitant ainsi les ruptures de chaîne qui pourraient compromettre l'automatisation globale.

Pour assurer la reproductibilité et la robustesse, le protocole méthodologique intègre une démarche itérative. Cette itération se traduit par des cycles d'expérimentation et de validation où la génération des artefacts Docker est systématiquement confrontée aux environnements de référence. Chaque cycle vise à réduire les écarts fonctionnels ou de configuration détectés lors des tests, favorisant une convergence progressive vers une représentation conteneurisée fidèle. La mise en place d'un processus automatisé d'intégration continue, couplé à des scénarios d'intégration fonctionnelle, facilite l'évaluation systématique des versions générées, en garantissant que l'outil respecte la contrainte d'automatisation complète initialement posée, tout en limitant les interventions manuelles à des ajustements minimes.

Concernant les outils technologiques, la méthodologie repose sur l'exploitation des conteneurs Docker comme support central, s'appuyant sur leur modularité, flexibilité, et surtout leur capacité à encapsuler les environnements dans des unités reproductibles et isolées. Le choix de Docker trouve une justification solide dans la littérature, où son utilisation dans l'automatisation des processus de déploiement a été démontrée comme un facteur clé d'amélioration dans les contextes complexes et multi-environnements par **Pahl et al. (2018)**. La méthodologie prévoit donc la construction progressive d'un moteur intelligent de génération de *Dockerfiles* et de fichiers Docker Compose, capables d'interpréter automatiquement les données extraites pour traduire les configurations spécifiques en instructions Docker standardisées. Ce moteur sera conçu pour intégrer des règles configurables, permettant d'adapter la génération en fonction des spécificités

techniques des applications analysées et des contraintes d'infrastructure, assurant ainsi une personnalisation indispensable face à la diversité des cas d'usage rencontrés.

Enfin, cette démarche méthodologique est complétée par une série d'itérations de tests expérimentaux et de retours d'expérience, permettant d'ajuster les algorithmes d'analyse, d'améliorer les règles de génération et d'optimiser les performances globales. Ces itérations s'inscrivent dans une approche agile, favorisant la rétroaction rapide et l'adaptabilité aux spécificités des environnements cibles. Ainsi, cette méthode garantit non seulement l'exactitude fonctionnelle des artefacts générés, mais favorise également leur robustesse face aux évolutions futures. Ce caractère adaptatif s'avère essentiel dans un contexte où les technologies de conteneurisation évoluent rapidement et où les exigences des environnements de production réclament une grande flexibilité.

En somme, l'approche méthodologique retenue s'avère rigoureuse et pragmatique, conjuguant une analyse fine des systèmes existants, une formalisation des contraintes fonctionnelles et techniques, et une automatisation poussée des processus de génération. Elle répond aux attentes opérationnelles en réduisant les interventions manuelles, aux exigences de qualité en assurant une équivalence fonctionnelle forte, et aux enjeux stratégiques en facilitant la modernisation des systèmes d'information dans un cadre sécurisé et conforme. Par cette approche intégrée, l'étude s'inscrit au cœur des dynamiques contemporaines du génie logiciel, matérialisées par la cartographie de **Kratzke et Quint (2017)** sur les architectures *cloud-native*, et contribue à l'industrialisation des processus DevOps tout en ouvrant la voie à des innovations méthodologiques dans la migration automatisée d'applications web.

1.8 SCOPE ET LIMITES DE L'ETUDE

L'étude présentée s'inscrit dans un cadre ambitieux et techniquement complexe, ce qui impose de délimiter explicitement son périmètre ainsi que ses limites afin d'en clarifier la portée et de situer ses apports. Tout d'abord, le champ d'application de l'outil automatisé développé se concentre exclusivement sur les applications web traditionnelles, c'est-à-dire celles déployées sur des serveurs physiques et des machines virtuelles, sans intégrer à ce stade les architectures *cloud-native*, les microservices conçus d'emblée en conteneurs, ou encore les applications fortement distribuées reposant sur des systèmes de type *serverless*. Cette restriction découle d'une volonté de maîtriser les variables du problème, en ciblant des environnements où les dépendances et configurations sont généralement centralisées et accessibles, ce qui facilite l'analyse statique et

dynamique nécessaire à l'extraction automatisée. À cet égard, **Kratzke et Quint (2017)** soulignent qu'il existe un fossé conceptuel et technique majeur entre la gestion des systèmes monolithiques hérités et les architectures natives du cloud, ce qui justifie méthodologiquement de traiter la migration du *legacy* comme un problème d'ingénierie distinct. Par conséquent, le cadre conceptuel ne prend pas en compte les spécificités propres à d'autres paradigmes d'application tels que les applications mobiles, les systèmes embarqués, ni les frameworks ultra-modernes qui intègrent déjà des mécanismes natifs de conteneurisation.

En outre, la méthodologie repose sur l'identification et la récupération des dépendances à partir des systèmes sources sans modifier leur état initial, ce qui signifie que l'outil ne couvre pas les cas où les applications présentent des comportements dynamiques ou adaptatifs complexes à l'exécution ou configurent des environnements évolutifs, notamment ceux intégrant des dépendances téléchargées à la volée ou modifiées à l'exécution (*runtime*). Ces scénarios, bien que fréquents dans certains contextes actuels, échappent pour l'instant à l'automatisation proposée car ils exigeraient des stratégies d'observation continue et d'analyse comportementale non prévues ici. Cette limite est en partie liée à la difficulté d'assurer une reproductibilité stricte dans la génération des artefacts Docker, surtout lorsqu'un état mutable ou imprévisible est impliqué. Comme le démontrent **Vasilakis et al. (2019)**, la mutabilité au runtime et les dépendances éphémères constituent les causes principales d'échec de réplication fonctionnelle dans les conteneurs, validant le choix de restreindre le périmètre aux états configuratifs stables.

L'outil développé ne prend pas non plus en charge l'ensemble des langages de programmation et frameworks existants, mais se concentre sur un ensemble représentatif d'applications web fondées sur des stacks technologiques largement utilisées, telles que les applications basées sur PHP, Java web, ou encore Node.js. Cette sélection vise à garantir une maîtrise approfondie des mécanismes d'extraction des dépendances et des configurations associées à ces technologies, évitant ainsi la dispersion des efforts sur des cas marginaux ou peu documentés, tout en conservant une portée suffisamment vaste pour démontrer la robustesse et la généralité de la méthode proposée.

Par ailleurs, la nature automatisée du processus n'élimine pas totalement la nécessité d'interventions manuelles, particulièrement lors des phases de validation finale ou d'ajustements spécifiques requis par des cas d'usage très particuliers. Cette limitation, inhérente à toute démarche d'automatisation dans des environnements aux configurations hétérogènes et évolutives, est intégrée à l'étude comme un paramètre de conception. L'objectif étant d'atteindre un équilibre

pragmatique, où l’outil réduit considérablement la charge de travail humaine sans prétendre à une automatisation absolue, ce qui serait irréaliste compte tenu de la diversité des situations rencontrées dans le terrain. Ce besoin d'un modèle d'automatisation collaboratif ou assisté est corroboré par **Wettinger et al. (2015)**, qui avancent que l'hétérogénéité des systèmes legacy rend indispensable le maintien de points d'extension configurables pour les interventions humaines spécialisées au sein des flux DevOps. En ce sens, la méthodologie adoptée encourage la modularité et la configurabilité pour offrir ces points d’extension facilitant les interventions ciblées.

Enfin, la validation empirique s’appuie sur un jeu de cas d’exemples soigneusement choisis, reflétant la diversité des applications traditionnelles, tout en restant dans le cadre d’environnements d’exécution maîtrisables. Cette approche expérimentale permet de limiter les biais liés à des configurations extrêmes ou atypiques, tout en assurant une exploration rigoureuse des capacités et des limites de l’outil dans des scénarios réalistes. Le choix d’un nombre restreint mais représentatif de cas d’études permettra d’aboutir à des conclusions fiables quant à la pertinence et à la robustesse de la solution proposée. Cette démarche s'aligne sur l'approche de validation empirique par cas d'étude défendue par **Cito et al. (2017)** pour évaluer la qualité des configurations de conteneurs face aux imperfections du monde réel, même si elle ne prétend pas embrasser l’intégralité des configurations possibles.

En résumé, la portée de cette étude privilégie un périmètre technique focalisé sur la migration automatisée d’applications web traditionnelles vers des environnements conteneurisés Docker, en garantissant une extraction et une reconstruction respectueuses des dépendances et configurations sources, sous réserve de certaines limites fonctionnelles et techniques inhérentes au degré d’automatisation envisageable. Ces délimitations sont indispensables pour cadrer le travail, assurer sa cohérence scientifique et rendre possible une évaluation objective de son efficacité dans les scénarios ciblés. Les limites évoquées ouvrent également des perspectives de travaux futurs pour étendre le champ des applications, intégrer des approches plus dynamiques et couvrir des aspects non fonctionnels plus approfondis dans des infrastructures modernes de déploiement.

1.9 PLAN DU TRAVAIL

La structuration rigoureuse du travail présenté découle inévitablement des délimitations méthodologiques et techniques précédemment établies. Afin d’assurer une progression logique et cohérente du mémoire, chaque partie s’articule autour d’objectifs précis qui contribuent

collectivement à répondre à la problématique centrale : concevoir un outil automatisé capable de migrer des applications web traditionnelles vers un environnement conteneurisé Docker, tout en garantissant l'équivalence fonctionnelle et une reproductibilité fiable. Cette organisation cherche ainsi à suivre une démarche scientifique progressive, passant de la compréhension du contexte et des exigences, à la conception détaillée, puis à la mise en œuvre avant d'envisager l'évaluation et les perspectives.

La première partie, comprenant l'introduction générale, pose les bases conceptuelles en explicitant la problématique, les motivations de l'étude ainsi que les limites du périmètre technique. Elle définit également les termes clés et le cadre d'analyse, ce qui est indispensable pour orienter la suite des travaux et prévenir toute confusion sur les choix adoptés. Cette phase sert aussi à exposer le contexte industriel et académique dans lequel s'inscrit la recherche, en mettant en lumière les lacunes et contraintes identifiées dans l'état de l'art, particulièrement celles liées à la complexité des applications déployées sur des infrastructures physiques ou virtuelles.

Ensuite, le développement méthodologique illustrera les stratégies adoptées pour analyser les applications sources : collecte et extraction automatisée des dépendances, identification des configurations pertinentes, et modélisation de ces éléments dans un format exploitable pour la génération des artefacts Docker. Cette section exploite des techniques d'analyse statique et dynamique, alliées à des modèles de templates, pour baliser le cheminement de la automatisation. L'accent sera mis sur le compromis entre précision des résultats et généralité de l'approche afin de préserver une certaine robustesse tout en limitant la nécessité d'une intervention humaine, conformément aux contraintes définies antérieurement.

La phase suivante s'attardera sur la conception logicielle et l'architecture de l'outil lui-même : choix des technologies (notamment Docker et Docker Compose), structure modulaire, mécanismes de traitement des données extraites, ainsi que les algorithmes de génération et d'optimisation des Dockerfiles et images. Cette partie sera cruciale pour traduire la démarche méthodologique en une solution concrète, en tenant compte des exigences de reproductibilité et de portabilité. Les détails d'implémentation permettront de comprendre comment les choix techniques influent sur la fiabilité et la maintenabilité de l'outil.

Par ailleurs, un chapitre dédié à l'expérimentation et à l'évaluation critique est prévu afin d'examiner la performance, la précision et la pertinence des artefacts Docker générés sur des cas d'usage représentatifs. Cette étape est essentielle pour valider que l'automatisation atteint les

objectifs d'équivalence fonctionnelle et de simplicité d'usage, tout en identifiant les éventuels points faibles, notamment en termes de taille d'image, sécurité ou ajustements manuels nécessaires. Les résultats obtenus nourriront une réflexion sur les limites intrinsèques de l'approche et pointeront des pistes d'amélioration, en lien avec les éléments exposés dans la phase initiale d'encadrement conceptuel.

Enfin, la conclusion générale intégrera une synthèse critique, mettant en perspective les apports, les contraintes et les perspectives ouvertes par cette recherche. Elle situera les contributions dans le contexte plus large de la migration applicative automatisée et évoquera les extensions possibles, telles que l'intégration future d'outils émergents ou la prise en compte de paradigmes plus dynamiques et complexes (microservices, cloud natif). Cette ultime analyse permet d'anticiper les évolutions technologiques et méthodologiques qui façonneront la mobilité applicative en environnements conteneurisés, tout en soulignant l'importance des interactions expert-automatisation.

Au fil de ce cheminement structuré, l'articulation entre les différentes parties garantit une progression dans la connaissance et une montée en expertise, tout en restant fidèle aux objectifs initiaux et aux contraintes définies. Il s'agit d'une organisation qui privilégie la clarté conceptuelle, la rigueur scientifique et l'applicabilité technique, pour apporter une contribution tangible et argumentée à la problématique complexe de la transformation automatisée des applications web traditionnelles vers les environnements conteneurisés.

CHAPITRE 2 : REVUE DE LA LITTERATURE

CHAPITRE 2 : REVUE DE LA LITTERATURE

La migration des applications web traditionnelles vers des environnements conteneurisés constitue une problématique multidimensionnelle qui mobilise des concepts issus de la virtualisation, de la conteneurisation, de l'orchestration et de l'administration des infrastructures. L'objectif de ce chapitre est de dresser un panorama critique des fondements théoriques et des réalisations empiriques qui jalonnent ce domaine. Dans un premier temps, nous définirons et analyserons les concepts clés virtualisation, conteneurisation, orchestration, outils de migration et administration en mettant en lumière leurs architectures, leurs avantages et leurs limites. Ensuite, nous exposerons le cadre théorique qui articule ces notions autour des principes de reproductibilité, d'équivalence fonctionnelle et d'automatisation. Enfin, nous passerons en revue les travaux empiriques récents, en soulignant leurs apports et leurs lacunes, afin de positionner notre contribution par rapport à l'état de l'art.

2.1 ANALYSE DES CONCEPTS

L'analyse des concepts constitue le socle théorique indispensable pour éclairer les choix méthodologiques et technologiques de ce travail. Elle permet de comprendre la transition des infrastructures rigides vers des environnements conteneurisés.

2.1.1 Le Paradigme d'isolation et de virtualisation

2.1.1.1 La Virtualisation traditionnelle

La virtualisation classique consiste à faire fonctionner plusieurs systèmes d'exploitation (OS) isolés sur une même infrastructure matérielle physique. Ce mécanisme repose sur une couche logicielle appelée **Hyperviseur**. On distingue les hyperviseurs deux (02) types d'hyperviseur :

- Les hyperviseurs de **Type 1 (Bare Metal)**, installés directement sur le matériel physique (VMware), offrant des performances optimales pour les serveurs de production ;
- Les hyperviseurs **Type 2 (Hosted)**, qui s'exécutent par-dessus un système d'exploitation hôte (VirtualBox, VMware Workstation), principalement utilisés pour le développement.

La virtualisation par machines virtuelles (VM) a longtemps été la norme pour mutualiser les serveurs physiques et isoler les applications. Chaque VM embarque son propre noyau, ses bibliothèques et ses services, ce qui assure une compatibilité étendue mais engendre une duplication des ressources et des temps de démarrage longs (plusieurs minutes). De plus, la gestion d'un parc de VM hétérogènes avec des configurations manuelles spécifiques à chaque environnement conduit à la **dérive de configuration** (*configuration drift*), phénomène largement documenté par **Fowler et Lewis (2014)** dans leur analyse des **architectures monolithiques**. Cette dérive rend les environnements de développement, de test et de production difficilement reproductibles, d'où l'émergence du besoin de conteneurisation.

2.1.1.2 Le concept de conteneurisation

La conteneurisation est une méthode de « **virtualisation légère** » qui consiste à emballer une application et toutes ses dépendances dans un conteneur unique. Ce conteneur s'exécute de manière isolée sur le noyau du système d'exploitation hôte, offrant une portabilité et une efficacité maximales sans nécessiter de système d'exploitation complet dédié. Elle permet d'exécuter plusieurs applications isolées dans des **conteneurs** qui partagent le même noyau hôte, mais disposent de leur propre système de fichiers, de leurs bibliothèques et de leurs variables d'environnement. Cette approche réduit drastiquement la surcharge par rapport aux VM (pas d'hyperviseur, pas de noyau invité) et offre des temps de démarrage plus rapides.

Un conteneur est construit à partir d'une **image** qui est un modèle immuable qui contient le code, les exécutables, les bibliothèques et les fichiers de configuration. Cette image est créée à partir d'un fichier de recette appelé **Dockerfile** qui décrit la suite d'instructions pour construire l'image. À l'exécution, le moteur de conteneurisation (**Docker**) instancie l'image en un conteneur, en utilisant les mécanismes du noyau Linux tels que les **namespaces** qui isolent les ressources du système par conteneur (processus, du réseau, du système de fichiers) et les **cgroups** qui limitent et mesurent l'utilisation des ressources (allocation maximale de CPU, mémoire, E/S disque). Ces primitives garantissent un isolement suffisant pour la plupart des usages, tout en maintenant une légèreté et une efficacité remarquables.

2.1.2 Analyse comparative : Virtualisation vs Conteneurisation

Pour comprendre pleinement l'évolution des infrastructures logicielles, il est essentiel de confronter l'architecture de la virtualisation par Machines Virtuelles (VM) à celle de la

conteneurisation. Bien que ces deux technologies visent à isoler des environnements applicatifs, leurs mécanismes sous-jacents diffèrent fondamentalement en matière de partage des ressources et de dépendance vis-à-vis du matériel.

2.1.2.1 L'architecture des Machines Virtuelles (VM)

Dans un modèle de virtualisation traditionnel, l'isolation est forte mais lourde. L'architecture se superpose de la manière suivante:

- **Infrastructure matérielle:** Le serveur physique (CPU, RAM, Disque, Réseau).
- **Hyperviseur :** La couche logicielle (type 1 ou type 2) qui simule le matériel et répartit les ressources physiques en plusieurs environnements virtuels.
- **Système d'exploitation invité (Guest OS) :** Chaque VM doit obligatoirement charger un noyau d'système d'exploitation complet (ex. Ubuntu, Windows Server). C'est cette couche qui génère une forte surcharge (*overhead*), car elle consomme des ressources matérielles importantes (plusieurs gigaoctets de RAM et d'espace disque) simplement pour s'initialiser.
- **Bibliothèques (Bin/Lib) et applications :** Les binaires requis s'exécutent par-dessus ce Guest OS.

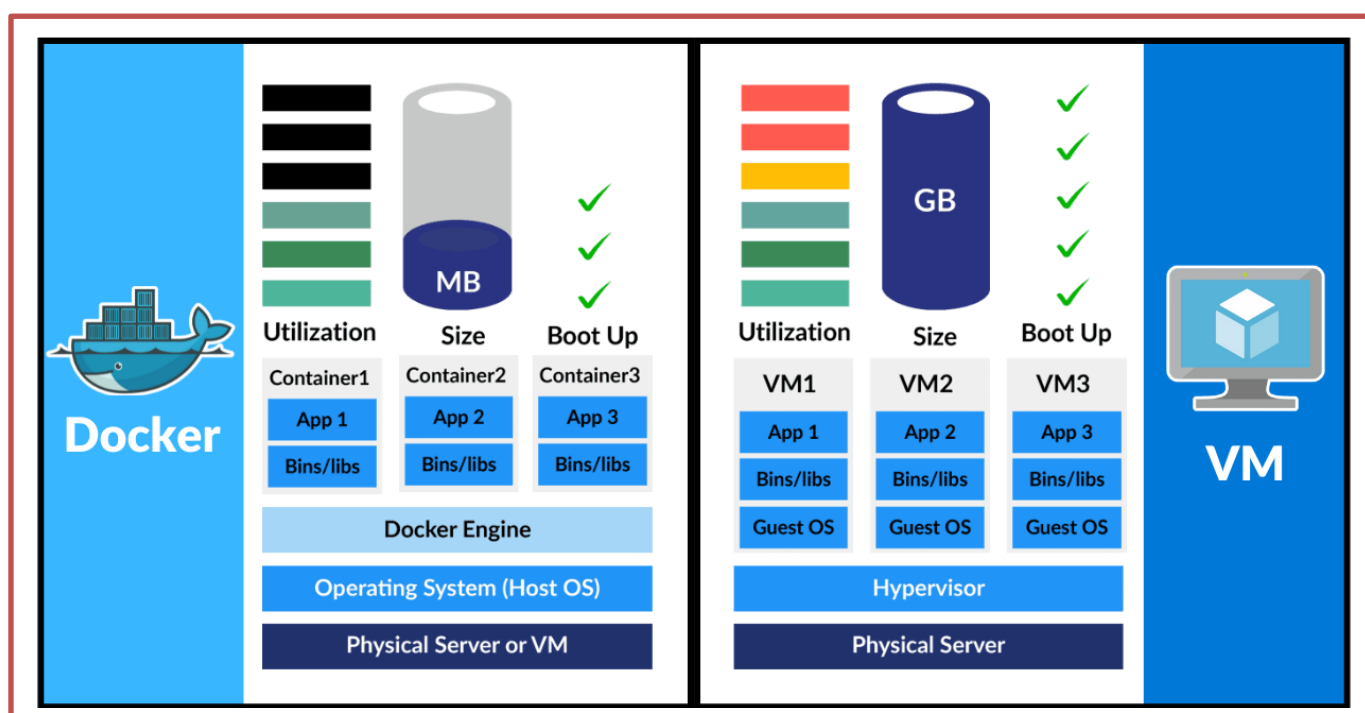
2.1.2.2 L'architecture de la Conteneurisation

La conteneurisation adopte une approche beaucoup plus agile en remplaçant l'hyperviseur et les multiples Guest OS par un moteur de conteneurs unique :

- **Infrastructure Matérielle et OS Hôte:** Les conteneurs s'exécutent directement sur le système d'exploitation de la machine physique ou de la VM hôte.
- **Moteur de conteneurs :** Une couche légère (comme le démon Docker ou containerd) qui orchestre les appels système.
- **Partage du noyau:** Contrairement aux VM, **tous les conteneurs partagent le même noyau de l'OS hôte**. L'isolation n'est plus matérielle, mais logicielle, matérialisée par les *Namespaces* et limitée par les *Cgroups* du noyau Linux.
- **Applications et packages (bin/llsib) :** Chaque conteneur n'embarque que l'application et ses dépendances strictes. Le conteneur est donc extrêmement léger (souvent quelques mégaoctets) et démarre de manière quasi instantanée (en quelques millisecondes).

Critère	Machine Virtuelle (VM)	Conteneur
Isolation	Matérielle (via hyperviseur)	Logique (via le noyau de l'OS hôte)
Système d'OS	Un Guest OS complet par VM	Partage de l'OS hôte
Taille moyenne	Plusieurs gigaoctets (Go)	Quelques mégaoctets (Mo)
Temps de démarrage	Quelques minutes	Quelques millisecondes
Utilisation des ressources	Élevée (allocation fixe et rigide)	Optimale (allocation dynamique et flexible)

Tableau 1: Tableau 1: Analyse comparative : Virtualisation vs Conteneurisation



2.1.3 Écosystème et outils de la conteneurisation

2.1.3.1 Les moteurs d'exécution

Il existe plusieurs outils de conteneurisation parmi lesquels :

- ✓ **Docker** : Pionnier historique, il offre une suite complète d'outils intégrant un démon de gestion (*dockerd*), un moteur de construction et une interface unifiée (ligne de commande, API REST) et un écosystème riche (Docker Hub, Docker Compose, Docker Swarm). Sa popularité tient à sa simplicité d'usage et à sa vaste communauté. Il reste la référence pour les environnements de développement et l'administration simplifiée.

- ✓ **Podman** : Alternative développée par Red Hat, Podman se distingue par son architecture *daemonless* (sans démon central) et sa capacité à exécuter des conteneurs en mode *rootless* (sans privilèges d'administrateur). Il est compatible avec les commandes Docker et offre une meilleure sécurité en évitant un processus privilégié permanent.
- ✓ **Containerd** : moteur industriel léger et standardisé issu de Docker, certifié par la Cloud Native Computing Foundation (CNCF). Il est optimisé pour les infrastructures à grande échelle et sert de runtime de bas niveau par défaut pour la majorité des clusters Kubernetes modernes.

2.1.3.2 Avantages et limites de la conteneurisation

La conteneurisation offre plusieurs avantages parmi lesquels on peut citer :

- ✓ **Portabilité** : une image conteneurisée peut s'exécuter sur toute infrastructure supportant le moteur (Linux, Windows, cloud, bare-metal).
- ✓ **Reproductibilité** : les images sont versionnées, ce qui garantit que l'environnement de production est identique à celui de développement.
- ✓ **Isolation légère** : les conteneurs cohabitent sans conflit de bibliothèques, éliminant le problème de « **matrice de l'enfer** » évoqué par **Cito et al. (2017)**.
- ✓ **Efficacité** : le partage du noyau permet une densité d'exécution bien supérieure à celle des VM, avec une consommation mémoire et CPU réduite.
- ✓ **Scalabilité horizontale** : le lancement de nouvelles instances est quasi instantané, facilitant l'élasticité.

Malgré ces avantages indéniables, elle présente aussi les limites :

- ✓ **Dépendance au noyau hôte** : un conteneur Linux ne peut pas s'exécuter directement sur un hôte Windows (bien que des solutions comme WSL2 ou Docker Desktop intègrent une VM Linux sous-jacente).
- ✓ **Gestion des données persistantes** : les conteneurs sont éphémères ; la persistance nécessite des volumes externes, ce qui complexifie la gestion.
- ✓ **Courbe d'apprentissage** : la maîtrise des *Dockerfiles*, Docker Compose, des réseaux, des volumes et des orchestrateurs requiert des compétences spécifiques (DevOps).

2.1.4 Orchestration de conteneurs

L'orchestration désigne l'automatisation du déploiement, de la mise à l'échelle, de la gestion du réseau et de la supervision d'un ensemble de conteneurs. Elle est indispensable lorsque les applications se composent de plusieurs services (microservices) ou lorsqu'elles nécessitent une haute disponibilité et une résilience.

Il existe plusieurs outils d'orchestration parmi lesquels :

✓ **Kubernetes (K8s)**

Kubernetes est la plateforme d'orchestration open-source de référence, née des travaux de Google et aujourd'hui gérée par la CNCF. Elle introduit des abstractions puissantes:

- **Pods** : regroupement d'un ou plusieurs conteneurs partageant le même réseau et le même volume.
- **Services** : points d'entrée stables pour accéder aux pods, avec découverte automatique et équilibrage de charge.
- **Deployments** : gestion des mises à jour progressives (*rolling updates*) et des rollbacks.
- **Horizontal Pod Autoscaler**: ajuste automatiquement le nombre de répliques en fonction des métriques (CPU, mémoire, requêtes HTTP).
- **Ingress** : contrôle du trafic externe vers les services.

Kubernetes offre une flexibilité et une modularité exceptionnelles, mais sa complexité opérationnelle (installation, mise à jour, sécurité, gestion des certificats) est souvent jugée excessive pour des applications de taille modeste.

✓ **Docker Swarm**

Docker Swarm est un moteur d'orchestration intégré à Docker, Swarm est plus simple à déployer et à administrer que Kubernetes. Il utilise la même API que Docker et permet de gérer un cluster de nœuds via des commandes familières. Ses fonctionnalités (services, réseaux superposés, équilibrage de charge interne) couvrent les besoins de nombreuses organisations. Cependant, sa moindre adoption et la préférence de l'industrie pour Kubernetes en ont fait une solution de niche pour des environnements homogènes.

Cependant, Le choix entre Kubernetes et Swarm dépend du périmètre du projet, des compétences disponibles et des exigences de scalabilité. Dans le cadre de notre outil d'aide à la migration, nous avons privilégié Docker Compose pour la phase de validation (environnement de test), tout en prévoyant la génération de manifests compatibles avec Kubernetes pour une éventuelle montée en charge en production.

2.1.5 Administration des environnements conteneurisés

L'administration des plateformes conteneurisées englobe deux (02) dimensions : la supervision (*monitoring*) et la gestion des ressources.

2.1.5.1 Supervision (monitoring)

La supervision consiste à collecter, stocker et visualiser les métriques de performance et de santé des conteneurs et de l'hôte. Les solutions modernes s'appuient sur des architectures de séries temporelles :

- ✓ **Prometheus** : système de surveillance open-source qui « scrappe » les métriques exposées par des *exporters* (Node Exporter pour l'hôte, cAdvisor pour les conteneurs, etc.) et les stocke dans une base de données. Il permet de définir des règles d'alerte (Alertmanager) et de visualiser les données avec Grafana.
- ✓ **Grafana** : outil de visualisation et de tableau de bord, intégré à de multiples sources de données (Prometheus, InfluxDB, Elasticsearch). Il offre une interface intuitive pour suivre l'évolution de la charge CPU, de la mémoire, du réseau et des erreurs applicatives.
- ✓ **ELK Stack (Elasticsearch, Logstash, Kibana)** : solution complémentaire pour la centralisation et l'analyse des logs.

2.1.5.2 Gestion des ressources

La gestion des ressources inclut l'allocation fine des quotas CPU/mémoire (via `cgroups`), la politique de répartition des conteneurs sur les nœuds, ainsi que la gestion des volumes persistants (PV) et des réclamations de volumes (PVC) dans Kubernetes. L'objectif est d'éviter la contention et de garantir une qualité de service (QoS) pour les applications critiques.

2.2 CADRE THEORIQUE DE LA MIGRATION APPLICATIVE

Ce cadre définit les règles de transformation formelle permettant de faire passer une application d'un état traditionnel monolithique à un état conteneurisé.

2.2.1 Principes théoriques de la migration automatique

La migration automatisée requiert une phase de rétro-ingénierie (Reverse Engineering) découpée en deux approches complémentaires :

1. **L'analyse statique** : Inspection au repos des fichiers sources, scripts et fichiers de configuration (parsing des dépendances PHP Composer, détection des extensions de base de données).
2. **L'analyse dynamique** : Surveillance au runtime (en mouvement) des processus actifs pour capturer les ports réseau ouverts, les variables d'environnement injectées et les chemins d'accès au stockage.

La transition entre un environnement d'exécution traditionnel souvent marqué par des configurations dispersées entre serveurs physiques et machines virtuelles et une image Docker implique une formalisation explicite des dépendances et configurations implicites. Cette formalisation est justement la pierre angulaire qui relie la compréhension conceptuelle de l'application et la capacité à générer automatiquement des Dockerfiles et autres artefacts. Elle signifie qu'il ne suffit pas de reproduire une simple copie de l'environnement existant, mais qu'il faut reconstruire, à partir d'analyses statiques et dynamiques, un modèle cohérent et normalisé de l'application.

L'un des axes majeurs de la relation théorique réside dans les notions d'équivalence fonctionnelle qui articulent les deux concepts dans un horizon d'exigences qualitatives élevées. L'équivalence fonctionnelle, comprise comme la préservation des résultats, comportements et performances, incite à une analyse méticuleuse de chaque élément composant l'application traditionnelle afin qu'il trouve une correspondance intégrale dans l'environnement Dockerisé. Cela implique une granularité fine dans l'extraction des dépendances et configurations, mais aussi une maîtrise des mécanismes de construction des images Docker, depuis le choix des bases jusqu'à l'ordre des instructions dans le Dockerfile.

2.2.2 Le concept d'Orchestration de conteneurs

Lorsque le nombre de conteneurs grandit, leur gestion manuelle devient impossible. L'orchestration automatise le déploiement, la mise à l'échelle (scaling) et la tolérance aux pannes.

- **Kubernetes (K8s)** : Standard industriel pour l'orchestration distribuée. Il utilise une architecture déclarative basée sur des concepts puissants : les *Pods* (la plus petite unité d'exécution), les *Services* (pour l'équilibrage de charge et la découverte réseau), et les *Deployments* (pour la mise à l'échelle et les mises à jour transparentes). K8s gère nativement l'auto-scaling horizontal basé sur la télémétrie.
- **Docker Swarm** : Solution d'orchestration native intégrée à Docker. Moins complexe que Kubernetes, elle permet de regrouper plusieurs hôtes Docker en un cluster virtuel (Swarm) via des fichiers YAML simples, idéale pour les structures recherchant une mise en œuvre rapide sans surcharge administrative complexe.

2.3 REVUE DE LA LITTERATURE EMPIRIQUE : OUTILS DE MIGRATION ET D'ADMINISTRATION

Cette section analyse les solutions logicielles existantes sur le marché pour la transformation et la gestion post-migration des environnements, afin de situer la valeur ajoutée de notre outil.

2.3.1 Analyse comparative des outils d'aide à la migration

- ✓ **Kompose** : outil qui transforme les fichiers docker-compose.yml en manifests Kubernetes (Deployments, Services, Ingress). Il simplifie le passage d'un développement local (Docker Compose) à une production orchestrée (Kubernetes). Néanmoins, il ne gère pas l'extraction des dépendances applicatives ni la génération initiale des *Dockerfiles*.
- ✓ **Move2Kube** : outil open-source visant à automatiser la migration d'applications vers Kubernetes. Il analyse le code source, les scripts de déploiement (Ansible) et les configurations existantes pour générer des manifests adaptés. Il prend en charge plusieurs langages (Java, Node.js, Python) et intègre des transformateurs spécifiques pour les bases de données et les middlewares.
- ✓ **Migrate to Containers (MTC)** : solution commerciale proposant une analyse approfondie des applications legacy (monolithes Java EE, ASP.NET) et une génération de plans de

migration pas à pas. Elle inclut des mécanismes de test A/B pour valider l'équivalence fonctionnelle.

- ✓ Contrairement à ces outils souvent complexes ou liés à des fournisseurs Cloud spécifiques, notre solution (**Deploy2Container**) se focalise sur l'extraction hybride des stacks patrimoniales (comme l'écosystème PHP) pour générer des configurations locales optimisées et souveraines tout en garantissant une équivalence fonctionnelle. Notre outil pourra également superviser automatiquement les applications qui seront migrées.

2.3.2 Le concept d'Administration, de Supervision et DevOps

L'administration d'une infrastructure moderne ne peut plus se faire de manière artisanale. Elle repose sur trois piliers clés :

1. **La Supervision (Monitoring) et Télémétrie** : Mise en œuvre de collecteurs de métriques comme **Prometheus** couplés à des agents comme **cAdvisor** (qui extrait en temps réel la consommation CPU, RAM et réseau de chaque conteneur). Ces données brutes sont ensuite restituées visuellement à travers des tableaux de bord dynamiques sur **Grafana**.
2. **La Gestion des ressources et des conteneurs** : Utilisation d'interfaces d'administration unifiées comme **Portainer**, permettant aux équipes de visualiser graphiquement la topologie du cluster, de gérer les volumes, les réseaux et de consulter les logs centralisés sans expertise DevOps poussée.
3. **Le Déploiement Continu (CI/CD)** : Automatisation des pipelines de livraison logicielle. Dès qu'une modification du code source ou de la configuration d'infrastructure (IaC) est validée, le pipeline construit automatiquement la nouvelle image Docker, exécute les tests d'équivalence fonctionnelle et met à jour l'environnement de production sans interruption de service.

4.

CHAPITRE 3 : APPROCHE METHODOLOGIQUE

CHAPITRE 3 : APPROCHE METHODOLOGIQUE

La validation scientifique d'un travail de recherche repose sur la rigueur et la clarté de la démarche adoptée pour répondre à la problématique technique posée. Ce troisième chapitre est entièrement consacré à l'**Approche méthodologique** mise en œuvre pour concevoir et réaliser notre solution. Dans un premier temps, nous définirons le type d'étude et procéderons à une analyse critique de l'existant. Ensuite, nous présenterons le modèle conceptuel de l'étude avant de détailler la phase de conception de l'application. En nous appuyant sur des langages de modélisation UML et des patrons d'architecture éprouvés, nous formaliserons la structure, le comportement et les cas d'utilisation du système d'aide à la migration automatisée.

3.1 TYPE D'ETUDE

L'étude menée dans ce travail mobilise une démarche essentiellement qualitative, combinée à des éléments d'ingénierie expérimentale, afin de répondre de manière pertinente à la problématique formulée. Cette approche d'étude s'inscrit dans une tradition méthodologique pragmatique, qui vise à concevoir et valider des solutions techniques innovantes par un processus itératif combinant analyse critique, prototypage et validation empirique. En d'autres termes, il ne s'agit pas uniquement d'une recherche exploratoire ou descriptive, mais d'une investigation appliquée qui met au centre le développement d'un artefact logiciel automatisé pertinent pour l'ingénierie logicielle contemporaine. L'usage prioritaire d'un examen axé sur la valeur opérationnelle (étude axiologique) et construit autour d'une expérimentation incrémentale permet d'atteindre plusieurs objectifs scientifiques en parallèle.

Premièrement, l'analyse théorique et la revue critique de la littérature telles que décrites dans la partie précédente offrent un socle conceptuel fondé sur les principes et solutions existants. Ce fondement critique alimente la conception méthodologique en mettant en lumière les points faibles, points forts, mais aussi les lacunes des approches actuelles, notamment en termes de standardisation des formats d'analyse et d'automatisation complète de la génération d'artefacts Docker. Deuxièmement, la dimension expérimentale valide sur le terrain les hypothèses émises à la lumière de cette synthèse et inscrit le travail dans une démarche de recherche appliquée,

caractéristique du génie logiciel. Cette double modalité permet un équilibre entre théorie et pratique, garantissant ainsi le réalisme et la reproductibilité des résultats.

Dans cette optique, le choix d'un prototype logiciel automatisé, capable d'extraire les dépendances et configurations des applications web traditionnelles déployées sur des environnements variés, répond à l'exigence d'une preuve de concept concrète, allant au-delà de la simple modélisation théorique. Cette expérimentation s'appuie notamment sur la mise en œuvre de techniques combinant analyse statique du code source (e.g., analyse des fichiers de configuration, scripts déployés, déclarations de dépendances) et dynamique (exécution contrôlée et observation comportementale). Comme le démontrent **Vasilakis et al. (2019)**, ce choix méthodologique est particulièrement recommandé dans la littérature technique récente pour sa complémentarité et son efficacité dans le traitement des architectures web existantes. Cette hybridation améliore la qualité et la précision des informations extraites, qui serviront à la génération automatisée des *Dockerfiles*, images et fichiers Docker Compose, garantissant ainsi la correspondance fonctionnelle et environnementale.

Le cadre expérimental s'inscrira également dans une démarche incrémentale intégrant la création d'un environnement contrôlé de test reproduisant à la fois des serveurs physiques et des machines virtuelles, conforme aux contextes réels rencontrés en entreprise. Cette stratification environnementale est d'une importance capitale car elle permet de vérifier l'adaptabilité et la robustesse de l'outil face aux spécificités propres à chaque plateforme, facteurs qui influent directement sur la qualité de l'extraction des dépendances et la gestion des configurations complexes, notamment les variables d'environnement, les scripts natifs, et les paramètres système. Par ailleurs, la nécessité d'automatiser la génération et le déploiement d'artefacts Docker dans ces environnements reflète le besoin d'intégration continue et de déploiement continu (CI/CD). À cet égard, l'étude empirique de **Cito et al. (2017)** souligne que l'automatisation de l'infrastructure sous forme de code constitue la clé de voûte pour lier l'analyse d'un système à sa livraison continue.

En ce sens, ce travail s'apparente à une étude de cas multiple au sein d'un cadre expérimental. Plusieurs applications web traditionnelles sélectionnées pour leurs architectures et modes de déploiements variés servent à évaluer la capacité de l'outil à gérer la diversité des cas réels, tout en assurant l'équivalence fonctionnelle et la reproductibilité attendues. L'étude de cas, en tant que méthode, permet d'examiner en profondeur les interactions complexes entre les composants applicatifs et leurs environnements, et de détecter les éventuelles contraintes spécifiques à certains

scénarios. Ce niveau d'analyse fine est indispensable pour répondre à l'objectif de minimisation de l'intervention manuelle, qui suppose une automatisation aussi complète que fiable.

Enfin, il convient de mentionner que cette démarche méthodologique implique également un processus de validation qualitative via des retours d'expérience issus d'utilisateurs finaux et d'experts en déploiement logiciel. Ces retours nourrissent la boucle d'amélioration continue de l'outil, conformément aux principes agiles et à la philosophie DevOps qui prônent un alignement étroit entre développement et exploitation. Ils garantissent que les artefacts générés ne sont pas seulement corrects sur le plan technique, mais aussi utilisables et maintenables dans des contextes industriels dynamiques, répondant donc aux exigences pratiques et aux contraintes opérationnelles.

Ainsi, la nature de l'étude est résolument ancrée dans une approche méthodologique appliquée et empirique, articulant fondements théoriques solides, développement incrémental d'un prototype expérimenté sur des cas réels, et validation qualitative des résultats. Elle dépasse la simple expérimentation isolée pour inscrire la recherche dans un cadre rigoureux et reproductible, garantissant la transférabilité des conclusions à d'autres projets d'automatisation des déploiements dans le domaine du génie logiciel. Cette orientation méthodologique, cohérente avec les observations empiriques précédemment évoquées, constitue la pierre angulaire de la crédibilité et de la pertinence des résultats obtenus dans ce mémoire.

3.2 ETUDE CRITIQUE DE L'EXISTANT

L'analyse critique de l'existant constitue une étape incontournable pour positionner ce travail face aux solutions actuelles en matière d'automatisation du déploiement des applications web traditionnelles via Docker. Elle permet non seulement d'identifier les apports et les limites des approches déjà développées, mais aussi de dégager des pistes d'amélioration concrètes et pertinentes pour la conception de l'outil automatisé proposé. Cette réflexion critique s'insère naturellement à la suite de la démarche méthodologique exposée précédemment, qui combine théorie, expérimentation et validation empirique, en offrant un cadre conceptuel précis sur lequel s'appuyer.

L'analyse de l'existant se concentre sur quatre outils majeurs : Docker, Kompose, Move2Kube et Migrate to Containers.

3.2.1 Analyse individuelle des solutions existantes

3.2.1.1 Docker (L'outil de construction de base)

L'examen des technologies et méthodes existantes révèle en premier lieu que **Docker**, en tant que moteur de conteneurisation, a profondément transformé la gestion des environnements d'exécution dans le génie logiciel. Comme le souligne l'étude de **Fati et al. (2021)**, Docker automatise le déploiement d'applications à travers la création d'images contenues dans des conteneurs, simplifiant ainsi la reproduction des environnements sur des infrastructures diverses. Toutefois, l'analyse de cette approche met en lumière que, bien que Docker facilite la standardisation des déploiements, la génération des artefacts nécessaires (*Dockerfiles*, images, fichiers Compose) reste souvent manuelle, voire sujette à des erreurs humaines dans la définition des dépendances et configurations spécifiques. Pour un non-informaticien, manipuler Docker directement revient à monter un meuble complexe sans notice : le moindre oubli d'une pièce (une dépendance ou une extension logicielle) fait s'effondrer tout le système.

3.2.1.2 Kompose (Le traducteur de format)

Kompose est un outil spécialisé dans la traduction de configurations d'un format local vers un format industriel à grande échelle (Kubernetes). Si l'on possède déjà une configuration Docker fonctionnelle, Kompose la traduit automatiquement en un langage compréhensible par les grands serveurs d'orchestration. C'est un excellent pont pour passer de la petite à la grande échelle.

Cependant, malgré ces différents avantages qu'elle offre, Kompose ne sait travailler que si le travail initial a déjà été fait. Il ne sait pas analyser le code source d'une vieille application ni deviner ses besoins. Si vous n'avez pas de fichier Docker au départ, Kompose est totalement inutile.

3.2.1.3 Move2Kube (L'analyseur open-source)

Move2Kube est un outil gratuit qui tente d'automatiser la découverte des besoins d'une application en analysant directement ses dossiers et ses fichiers sources. Il est capable de reconnaître plusieurs langages informatiques (Java, Python, Node.js) et de générer les fichiers nécessaires pour les déployer sur des structures Cloud complexes.

Son principal inconvénient réside dans sa complexité. Il est conçu pour les infrastructures géantes, il génère une quantité massive de fichiers et de paramètres qui saturent et ralentissent les

infrastructures locales plus modestes. De plus, son expertise sur les technologies très répandues mais spécifiques comme l'écosystème PHP (Laravel) reste superficielle, nécessitant souvent des corrections manuelles après coup.

3.2.1.4 Migrate to Containers (MTC)

Développée par le géant Google, c'est une solution commerciale haut de gamme conçue pour déplacer des serveurs entiers vers des environnements modernes. MTC est une machine de guerre capable de prendre un ordinateur virtuel complet, d'en extraire l'application et de vérifier par des tests comparatifs stricts que l'application fonctionne toujours aussi bien après la migration.

MTC est une solution captive et son principal inconvénient réside dans le fait qu'elle est extrêmement coûteuse. Elle oblige l'organisation à utiliser les services Cloud payants de Google (GCP). Pour des institutions publiques ou des entreprises soucieuses de la souveraineté de leurs données (ne pas dépendre de serveurs situés à l'étranger), cette dépendance est un risque stratégique et financier majeur.

3.2.2 Synthèse des limites de l'existant et justification de notre outil

Le tableau ci-dessous résume les forces et les faiblesses de ces outils par rapport aux besoins réels des structures locales :

Outil	Métaphore simplifiée	Point fort majeur	Limite critique (La lacune)
Docker	Les briques de construction brutes	Standard universel	Nécessite une écriture manuelle et une expertise humaine pointue.
Kompose	Le traducteur de plans	Passage à grande échelle	Inutile si l'application n'est pas déjà pré-configurée.
Move2Kube	L'architecte pour gratte-ciel	Automatisation ouverte	Trop complexe, lourd pour des serveurs locaux.
Migrate to Containers	Le déménageur de luxe	Puissance et tests rigoureux	Très coûteux, enferme l'utilisateur chez un fournisseur Cloud unique.

Tableau 2: Synthèse des limites de l'existant et justification de notre outil

En conclusion, cette étude critique démontre que si les outils utilisés pour la containerisation sont aujourd'hui largement répandus et offrent une base technique solide, la capacité d'automatiser de façon fiable la migration d'applications web traditionnelles vers des environnements containerisés reste largement incomplète. La pertinence du travail entrepris réside ainsi dans sa focalisation sur cette problématique, en intégrant et dépassant les limites des approches existantes par un prototype expérimental innovant, capable d'analyser, d'extraire et de générer automatiquement les artefacts Docker indispensables, tout en assurant la reproductibilité et l'équivalence fonctionnelle dans des infrastructures physiques et virtuelles hétérogènes. Cette démarche critique, en continuité logique avec le cadre méthodologique, structure la suite du mémoire et prépare le terrain pour les développements techniques détaillés ultérieurement.

3.3 MODELE DE L'ETUDE

Le modèle de l'étude proposé s'articule autour d'une démarche méthodologique systémique et itérative, qui s'appuie sur l'intégration cohérente des différentes phases nécessaires à la conception et à l'implémentation d'un outil automatisé pour la migration d'applications web traditionnelles vers des environnements Docker. L'objectif principal est de définir un cadre théorique et pratique rigoureux permettant d'assurer à la fois l'exhaustivité de l'analyse, la fiabilité des extractions, et la génération automatisée d'artefacts conteneurisés fidèles aux environnements d'origine, tout en minimisant l'intervention humaine. Ce modèle s'inscrit ainsi dans la continuité logique de l'analyse critique menée précédemment, qui identifiait les lacunes des solutions existantes notamment en termes de mécanismes d'extraction automatisée des dépendances et configurations.

Le choix d'un modèle combinant à la fois des approches statiques et dynamiques relève d'une nécessité établie par la complexité inhérente aux applications web classiques déployées dans des architectures hybrides (physique et virtuelle). En effet, une analyse uniquement statique, basée sur l'examen des fichiers sources, configurations ou logs, bien que précieuse, ne suffit pas à capturer pleinement les dépendances dynamiques ou les variables d'environnement configurées à l'exécution. Réciproquement, une extraction basée uniquement sur l'observation dynamique à travers des tests d'exécution est susceptible de ne pas révéler certaines configurations implicites ou des dépendances occultes, notamment celles conditionnées par des scénarios d'usage spécifiques. Le modèle intègre donc un pipeline d'analyse hybride, où la collecte initiale de données statiques est systématiquement complétée par une phase d'instrumentation et de

surveillance à l'exécution, garantissant ainsi une couverture maximale des éléments critiques à extraire.

La modélisation adoptée repose également sur une architecture modulaire, facilitant la séparation claire des préoccupations entre les étapes d'analyse, d'extraction, de traitement des données et de génération des artefacts Docker (*Dockerfile*, images et fichiers Docker Compose). Chaque module est conçu pour fournir des interfaces normalisées, permettant ainsi non seulement une meilleure maintenabilité et évolutivité de l'outil, mais aussi une adaptabilité à des contextes d'applications variés, par exemple différentes stacks technologiques ou environnements serveurs hétérogènes. Comme le soulignent **Pahl et al. (2018)** dans leur examen des technologies de conteneurisation, l'encapsulation de la logique d'infrastructure dans des architectures découplées est indispensable pour maintenir la portabilité cross-plateforme. Cette modularité favorise également l'intégration potentielle de méthodes d'intelligence artificielle ou d'apprentissage automatique pour améliorer la reconnaissance automatique des configurations, ouvrant des perspectives d'évolution futuristes.

Un autre aspect capital du modèle consiste en la définition précise des critères d'équivalence fonctionnelle et de reproductibilité. Ces critères agissent comme des métriques qualitatives et quantitatives permettant d'évaluer la pertinence de chaque artefact Docker généré. Il s'agit notamment de vérifier que les conteneurs produits reproduisent fidèlement les comportements observés dans les environnements originaux, en termes de services disponibles, d'interactions réseau, de performance, et d'intégrité des données. Cette validation s'appuie sur des tests automatisés, conçus pour être intégrés dans un pipeline continu d'intégration et de déploiement (CI/CD), garantissant ainsi une robustesse industrielle et une garantie de qualité cohérente avec les exigences du génie logiciel moderne.

Enfin, la dimension automatisée du modèle est soutenue par une orchestration accrue, notamment grâce à l'usage de technologies éprouvées telles que Docker et Docker Compose pour la gestion et le déploiement des environnements conteneurisés. L'outil se propose d'orchestrer l'ensemble des étapes, depuis la récupération des configurations jusqu'à la construction et au déploiement des images, en minimisant les interventions manuelles, ce qui répond aux exigences d'agilité et de scalabilité du développement logiciel contemporain. Les travaux de Wettinger et al. (2015) sur la rationalisation de l'automatisation DevOps au plus près du code démontrent empiriquement que l'orchestration programmatique via les API de conteneurs réduit drastiquement

les erreurs humaines tout en générant un gain significatif en temps et en ressources, soulignant ainsi la pertinence économique du modèle.

Ce modèle méthodologique s'adosse donc à une combinaison réfléchie d'analyses théoriques, d'implémentations techniques et de validations empiriques, illustrant la complexité et la richesse du défi à relever. Il ambitionne d'apporter une contribution novatrice en proposant une solution systémique, cohérente et pragmatique pour transformer la migration des applications web traditionnelles vers Docker, tout en garantissant leur fidélité fonctionnelle et reproductible, conformément aux besoins identifiés dans le cadre du problème posé. Cette démarche méthodologique constitue ainsi le socle conceptuel et opérationnel fondamental pour les développements ultérieurs du prototype, contextualisant ainsi pleinement ce travail dans la recherche appliquée en génie logiciel conteneurisé.

3.4 CONCEPTION DE L'APPLICATION

La conception est une étape cruciale qui permet de traduire les besoins fonctionnels en une architecture logicielle robuste. Pour ce projet, nous avons adopté une approche orientée objet pour garantir la modularité de l'outil de migration.

3.4.1 Le langage de modélisation UML

UML (Unified Modeling Language) est le langage de modélisation graphique universel utilisé pour visualiser, spécifier, construire et documenter les artefacts d'un système logiciel. Il offre une collection de diagrammes permettant de couvrir aussi bien les aspects statiques (structure) que dynamiques (comportement) de l'application. Dans ce projet, UML nous permet de structurer la logique d'extraction des dépendances et de génération des fichiers Docker.

3.4.2 Présentation des diagrammes UML

Cette section détaille les interactions et la structure de l'outil à travers différents prismes de modélisation.

3.4.2.1 Diagramme des cas d'utilisation

Il définit les interactions entre l'utilisateur (Administrateur Système ou Développeur) et l'outil. D'après l'analyse voici les principaux cas d'utilisation identifiés :

- ✓ Création d'une application ;
- ✓ Modification d'une application ;
- ✓ Suppression d'une application ;
- ✓ Télécharger les fichiers de config (Dockerfile, la création du fichier Docker Compose) ;
- ✓ Demarrer une application ;
- ✓ Arrêter une application ;
- ✓ Visualiser l'état d'une application conteneurisée ;
- ✓ Consulter les logs d'une application ;
- ✓ Ouvrir la console d'une application à distance ;
- ✓ Gérer les volumes ou réseaux sans avoir à utiliser exclusivement la ligne de commande ;
- ✓ Surveiller la consommation CPU, RAM et réseau des conteneurs ;
- ✓ Gérer la scalabilité.

Ces cas d'utilisation modélisent un service rendu par le système à un acteur du système.

Maintenant que nous avons identifié les cas d'utilisation et les acteurs, nous allons pouvoir les représenter sous un digramme de cas d'utilisation.

3.4.2.1.1 Formalisme

Le diagramme des cas d'utilisation comporte les éléments suivants :

- *Un acteur*, représenté par un **bonhomme** avec son **nom inscrit dessous** ;
- *Le système*, représenté par un **rectangle** avec son **nom inscrit au-dessus** ;
- *Le cas d'utilisation*, représenté par une **ellipse** contenant le nom du cas (généralement un verbe à l'infinitif) ;
- Les cas d'utilisation sont représentés à l'intérieur du système ; *Les relations*, elles sont multiples.

La représentation graphique d'un diagramme de cas d'utilisation simplifié est la suivante:

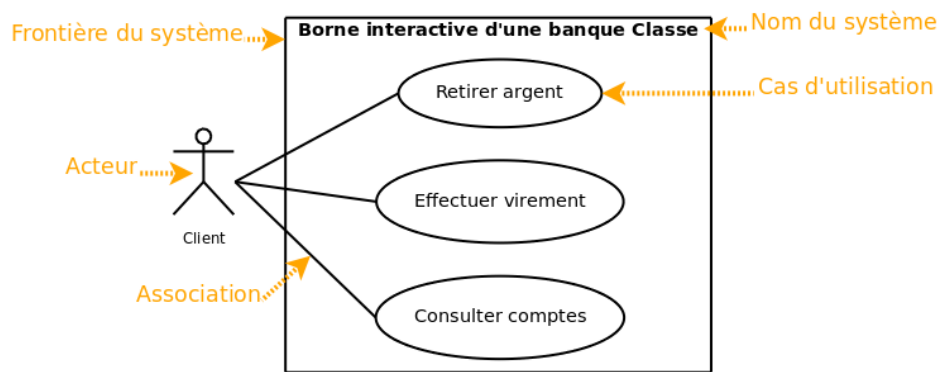


Figure 1: formalisme d'un diagramme de cas d'utilisation

3.4.2.1.2 Quelques diagrammes de cas d'utilisation

Conformément à la méthode UML, nous avons produit des diagrammes de cas d'utilisation. En effet, ces représentations graphiques permettent aux équipes métiers et aux équipes techniques d'avoir une vue synthétique sur l'ensemble des opérations de chaque acteur au sein du système. Ces diagrammes seront déroulés par acteur et par module pour des soucis de clarté.

Les représentations graphiques seront étoffées par une description des scénarii de cas d'utilisation afin de garantir la conformité de la compréhension de l'équipe technique des besoins métiers exprimés.

Pour les processus critiques et/ou avec une forte complexité, nous produirons également des diagrammes de séquence ou d'activité selon le cas.

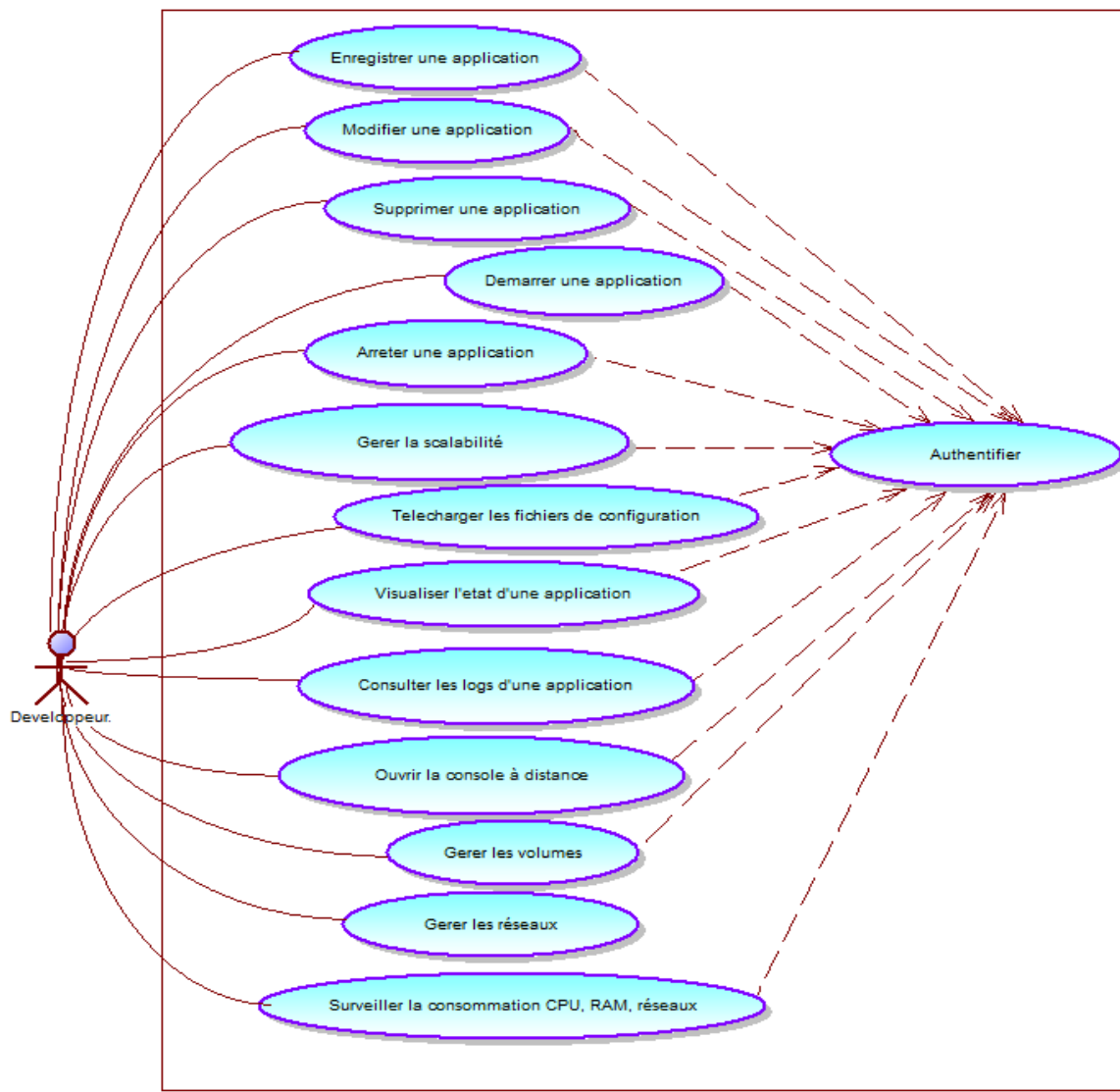


Figure 2: Diagramme de cas d'utilisation du système.

3.4.2.2 Diagramme d'activité

Un **diagramme d'activité** est un diagramme comportemental qui permet de représenter graphiquement le flux de travail (*workflow*) d'un système, d'un processus métier ou d'un algorithme. Les diagrammes d'activités permettent de mettre l'accent sur les traitements. Ce diagramme modélise le flux de contrôle interne. Il décrit le processus décisionnel de l'outil.

3.4.2.2.1 Formalisme du diagramme d'activité

Les éléments utilisés dans le diagramme d'activité sont :

- ✓ La transition, représentée par une flèche ;
- ✓ Le branchement conditionnel (Décision), représenté par un losange ;
- ✓ L'activité, représentée par un rectangle arrondi ;
- ✓ La barre de synchronisation;
- ✓ L'état initial, représenté par un cercle plein ;
- ✓ L'état final.

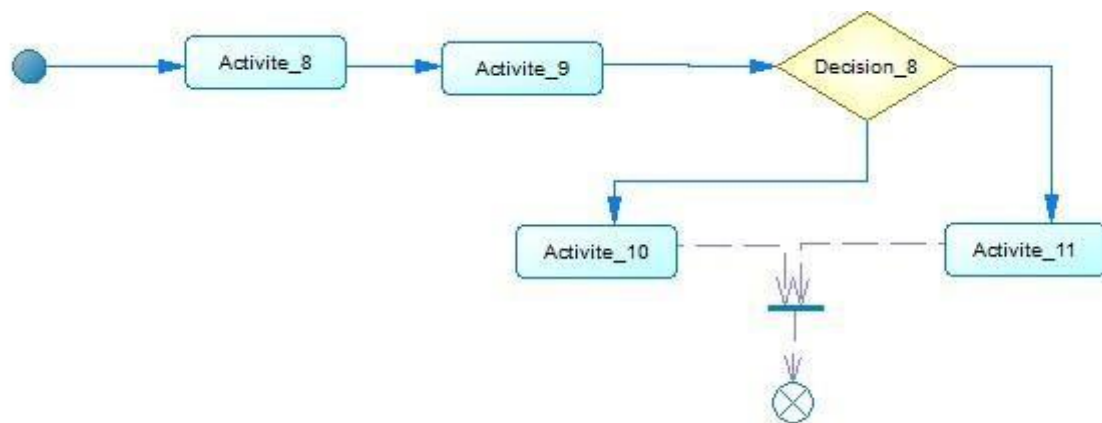


Figure 3: Formalisme du diagramme d'activité.

3.4.2.2 Quelques scenarii et diagrammes d'activités

Enregistrer une nouvelle application
<i>Pré-condition</i> ⌚ L'utilisateur est connecté à son compte ⌚ L'utilisateur dispose d'un dossier contenant le code source d'une application web traditionnelle
<i>Scénario nominal</i> ⌚ Clique sur le bouton « + Nouvelle application » ⌚ Il renseigne tous les champs obligatoires du formulaire et Il sélectionne le dossier contenant le code source ⌚ L'application verifie si le dossier contenant le code source est valide ⌚ L'application analyse automatiquement le code source et detecte tous les dependences neccessaires pour son fonctionnement ⌚ L'application crée les repertoires specifiques pour chaque application ⌚ L'application fais une copie integrale du code source ⌚ Il recoit une alerte de succès ⌚ L'application configure automatiquement un serveur web (Nginx) ⌚ Il clique sur le bouton « Enregistrer » ⌚ L'applicaton crée automatiquement les fichiers de configuration (Dockerfile et docker-compose.yml) ⌚ Il reçoit une alerte de succès ⌚ Il est redirigé vers la page qui affiche tous les applications enregistrer
<i>Post-condition</i> ⌚ Tous les fichiers de configurations sont générés
<i>Scénario alternatif: le dossier contenant le code source n'est pas valide ou éligible</i> ⌚ L'usager reçoit un message informatif

Tableau 3: Sscenarii diagramme d'activité "Enregistrer une application"

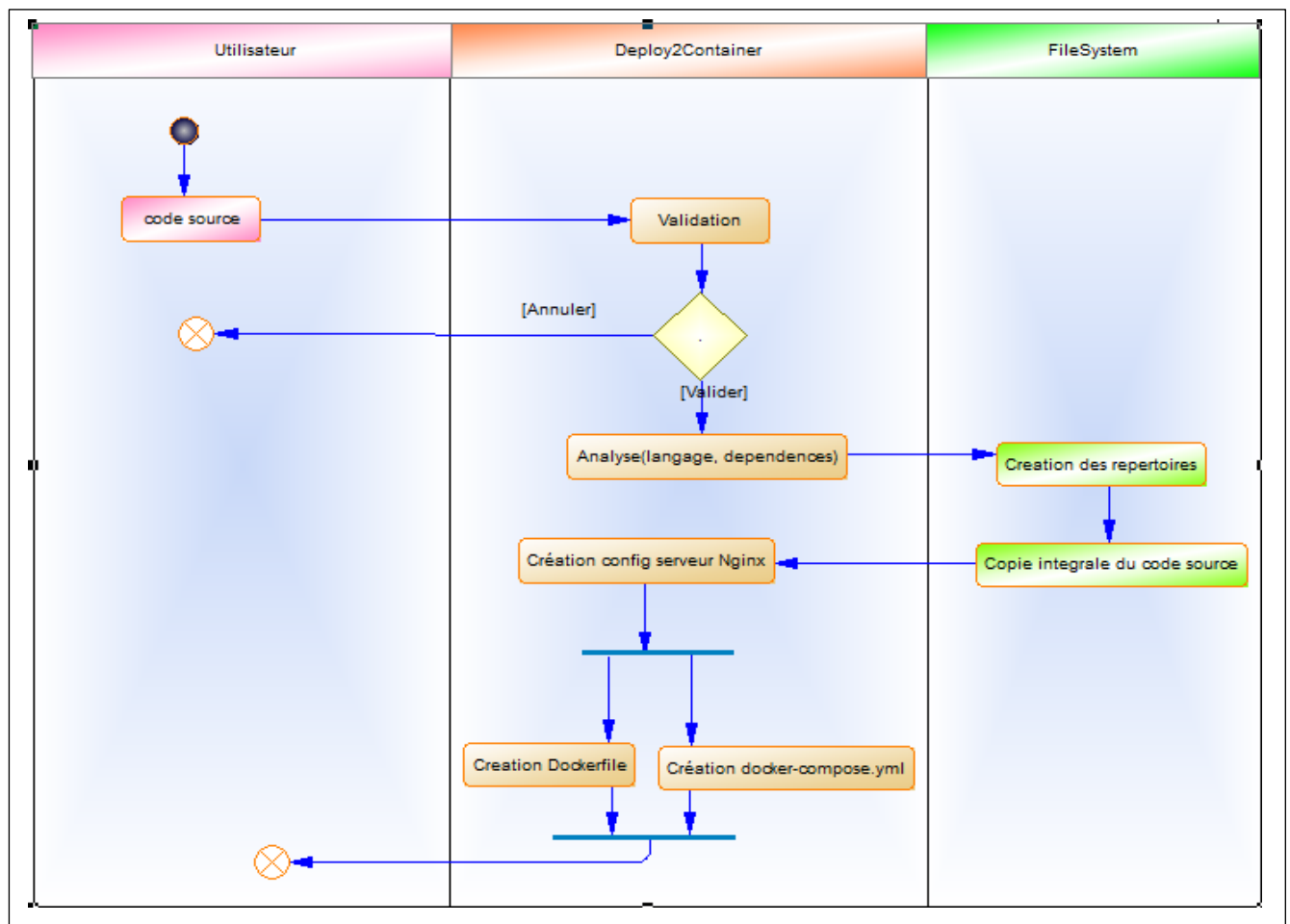


Figure 4 : Diagramme d'activité « Enregistrer une nouvelle application ».

Demarrer une application

Pré-condition

- ⌚ L'utilisateur est connecté à son compte
- ⌚ L'application est déjà enregistré

Scénario nominal

- ⌚ Clique sur le bouton « **Start** »
- ⌚ L'application reçoit la requête de démarrage
- ⌚ Il récupère le chemin d'accès local où se trouvent les fichiers Dockerfile et docker-compose.yml de l'application concernée.
- ⌚ Il interroge le moteur de Docker engine et exécute commande d'orchestration en arrière-plan (ex: docker compose up -d)
- ⌚ Il vérifie le statut de la commande envoyé à Docker-engine
- ⌚ Si erreur, un message d'erreur s'affiche (ex: "Échec du démarrage"), le bouton repasse à l'état initial.
- ⌚ En cas de succès de la commande, l'application vérifie l'état de santé application
- ⌚ L'utilisateur reçoit une alerte de succès
- ⌚ Il est redirigé vers la page qui affiche tous les applications enregistrer

<i>Post-condition</i> ↓ L'icône de statut passe au Vert/Actif (bouton Stop orange/allumé),
<i>Scénario alternatif: Erreur d'exécution de la commande</i> ↓ L'utilisateur reçoit un message informatif

Tableau 4: Scenarii diagramme d'activité demarrer une application

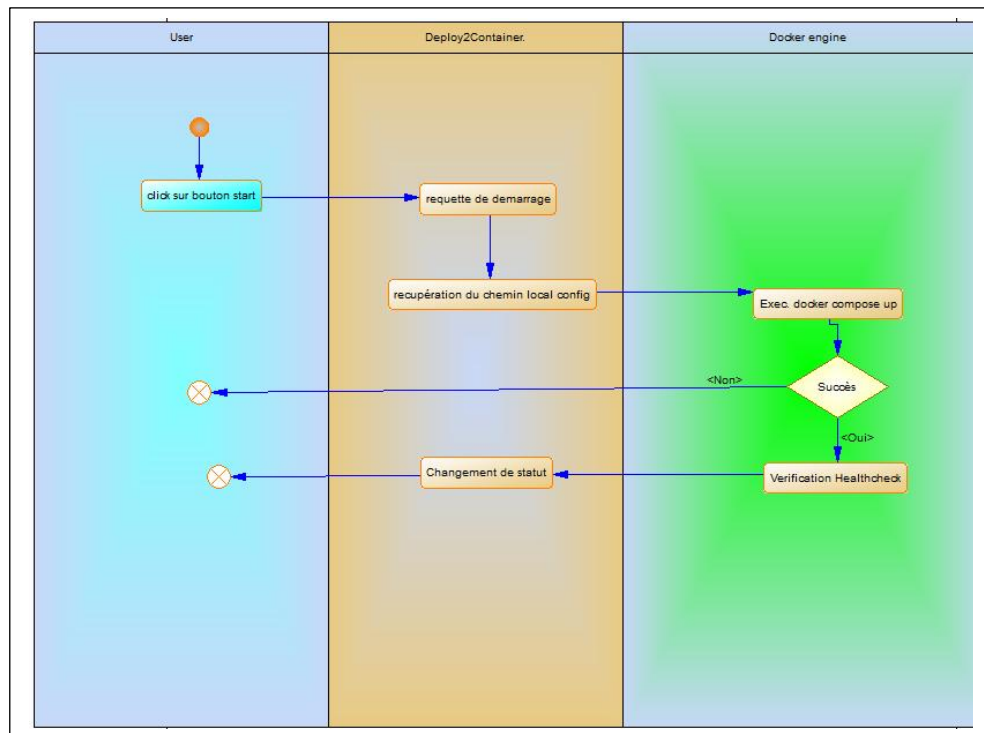


Figure 5: Diagramme d'activité « Demarrer une application ».

3.4.2.3 Diagramme de déploiement

Le **diagramme de déploiement** est un diagramme structurel de la spécification **UML** (Unified Modeling Language). Il est utilisé pour représenter l'architecture physique d'un système informatique.

Contrairement aux diagrammes de classes ou de cas d'utilisation qui se concentrent sur l'aspect logique ou fonctionnel du logiciel, le diagramme de déploiement montre **comment et où** les composants logiciels (fichiers, exécutables, bases de données) sont installés et exécutés sur les ressources matérielles (serveurs, ordinateurs, réseaux).

3.4.2.3.1 Description des Couches de l'Architecture de déploiement

Il décrit l'organisation physique du système du point de vue de déploiement. Il est constitué de deux (02) couches principales : la couche client et la couche de persistance et d'infrastructure.

➤ Couche Présentation et traitement (Client JavaFX)

- ✚ **Technologie** : JavaFX (Application Desktop s'exécutant dans une JVM et pilotes de connexion (JDBC)).
- ✚ **Rôle** : Elle fournit l'interface utilisateur pour piloter le processus. C'est cette couche qui va lire le code source d'une application existante, générer dynamiquement l'arborescence requise, écrire les fichiers de configuration (Dockerfile, docker-compose.yml, default.conf) et potentiellement exécuter les commandes système (comme les commandes Docker) en arrière-plan.
- ✚ **Connectivité** : Elle interagit avec la couche de persistance via des flux d'E/S standard (Java NIO) pour le système de fichiers, et via JDBC pour la base de données relationnelle.

➤ Couche de Persistance et d'Infrastructure

Bien que qualifiée de « **persistance** », cette couche sert également d'environnement d'infrastructure local. Elle se divise en deux :

1. **La Base de Données (SGBD)** : Elle stocke les données relationnelles globales de l'application de migration (par exemple, la liste des applications migrées, les états de déploiement, les utilisateurs).
2. **Le Système de Fichiers (Filesystem)** : Un espace structuré et caché sous le répertoire de l'utilisateur, servant de « **Workspace** » (espace de travail) où les configurations d'infrastructure de chaque application et les outils de supervision sont stockés.

Le diagramme suivant illustre la répartition physique et logique de vos composants

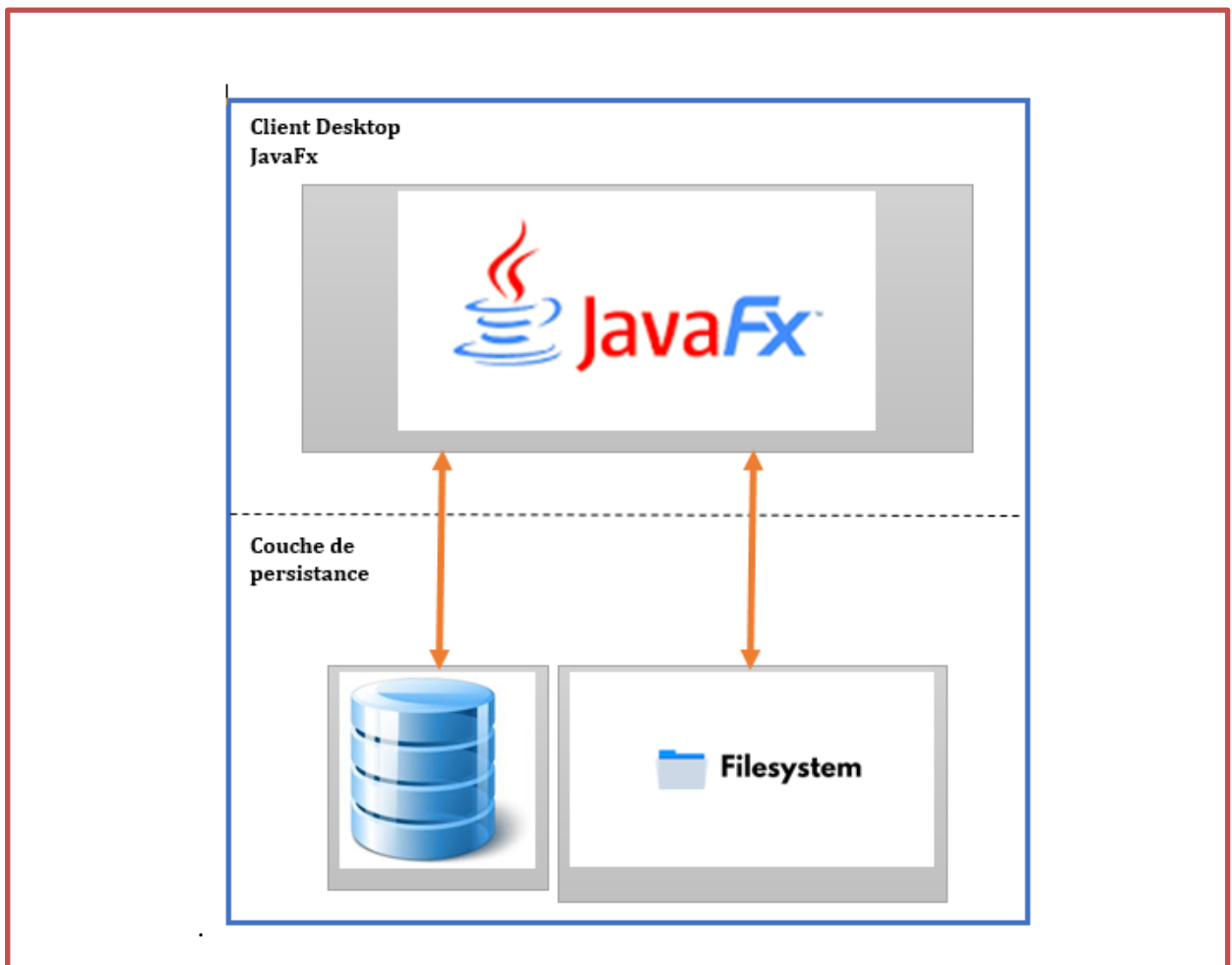


Figure 6: Diagramme de déploiement.

3.4.2.3.2 Organisation détaillé du Filesystem

L'arborescence est ancrée dans le répertoire personnel de l'utilisateur (`~` ou `/home/utilisateur/`). Le dossier racine est **caché**, ce qui signifie que son nom commence par un point sous Linux : **`.migration/`**.

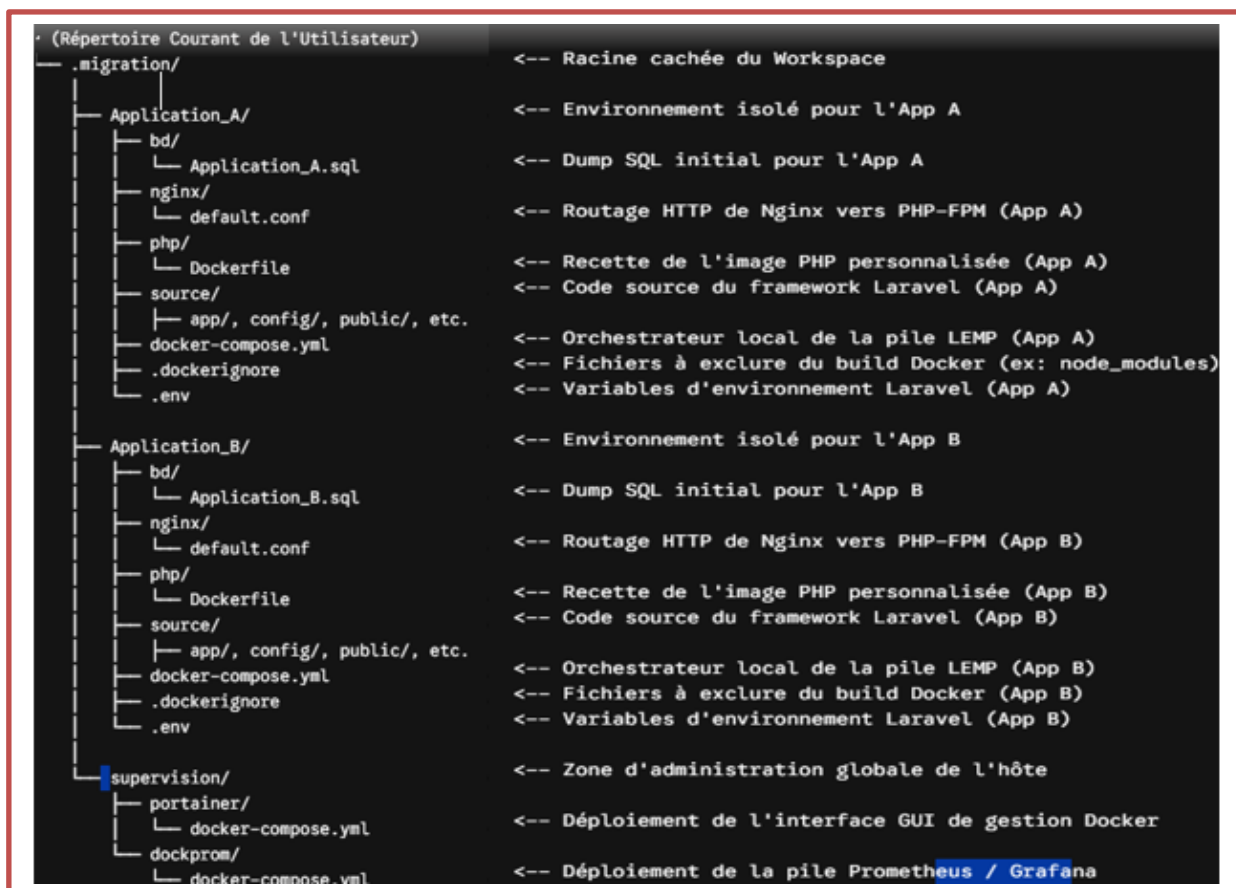


Figure 7: Organisation détaillé du FileSystem.

3.4.2.3.3 Role de chaque composant du Filesystem

- **Dossier migration** : C'est le conteneur global. Il centralise tout ce qui est lié au processus de transformation des applications legacy vers le format conteneurisé
- **Le dossier de l'application ([Nom_De_L_Application]/)** : C'est un environnement autonome prêt à être déployé via Docker Compose (architecture multi-conteneurs : Linux, Nginx, MySQL/MariaDB, PHP). Chaque application possède son propre espace. Cela évite les conflits de versions entre les Dockerfiles ou les fichiers docker-compose.yml de différents projets.

- **bd/** : Contient le dump SQL initial pour provisionner la base de données du projet migré lors du premier lancement.
- **nginx/default.conf** : Fichier de configuration du serveur web Nginx, configuré pour rediriger les requêtes HTTP vers le conteneur PHP (via PHP-FPM) et pointer vers le dossier public/ de Laravel.
- **php/Dockerfile** : Instructions pour construire l'image Docker PHP personnalisée (contenant les extensions requises par Laravel comme pdo_mysql, mbstring, openssl, etc.).
- **source/** : Contient l'intégralité du code de l'application Laravel (les dossiers app/, config/, public/, routes/, etc.).
- **docker-compose.yml (Applicatif)** : Définit et lie les services (db, nginx, php) pour que l'application Laravel fonctionne de manière isolée.
- **Le dossier supervision/** : Indépendant des applications, il gère l'infrastructure hôte.
 -  **portainer/docker-compose.yml** : Déploie l'interface graphique de gestion des conteneurs, volumes et réseaux Docker.
 -  **dockprom/docker-compose.yml** : Déploie une pile de monitoring (généralement Prometheus, Grafana, cAdvisor, Node Exporter) pour surveiller l'état de santé du processeur, de la RAM et des conteneurs.

3.4.2.3.4 Dynamique de fonctionnement et flux de données

1. **Initialisation** : Au premier lancement, l'application doit vérifier l'existence de l'arborescence migration/supervision et la créer si nécessaire.
2. **Génération par JavaFX** : L'utilisateur sélectionne un projet Laravel brut dans l'interface JavaFX. Le client crée le dossier ~/.migration/[Nom_App], y copie le code source dans source/, puis injecte les fichiers templates standard (Dockerfile, default.conf, docker-compose.yml, .env) ajustés selon les paramètres saisis par l'utilisateur.
3. **Cycle de vie des conteneurs** : Une fois l'arborescence générée, le client JavaFX peut invoquer le démon Docker local en exécutant une commande en tâche de fond (ex: docker compose up -d exécuté dans le dossier cible) pour démarrer l'application ou les outils de supervision.

4. **Isolation** : Grâce au point (.) devant le dossier migration, l'espace de travail reste discret pour l'utilisateur dans son explorateur de fichiers standard, évitant les suppressions accidentelles de configurations d'infrastructure sensibles.

3.4.2.4 Diagramme de classe

Ce diagramme modélise la structure de données interne et la logique métier backend

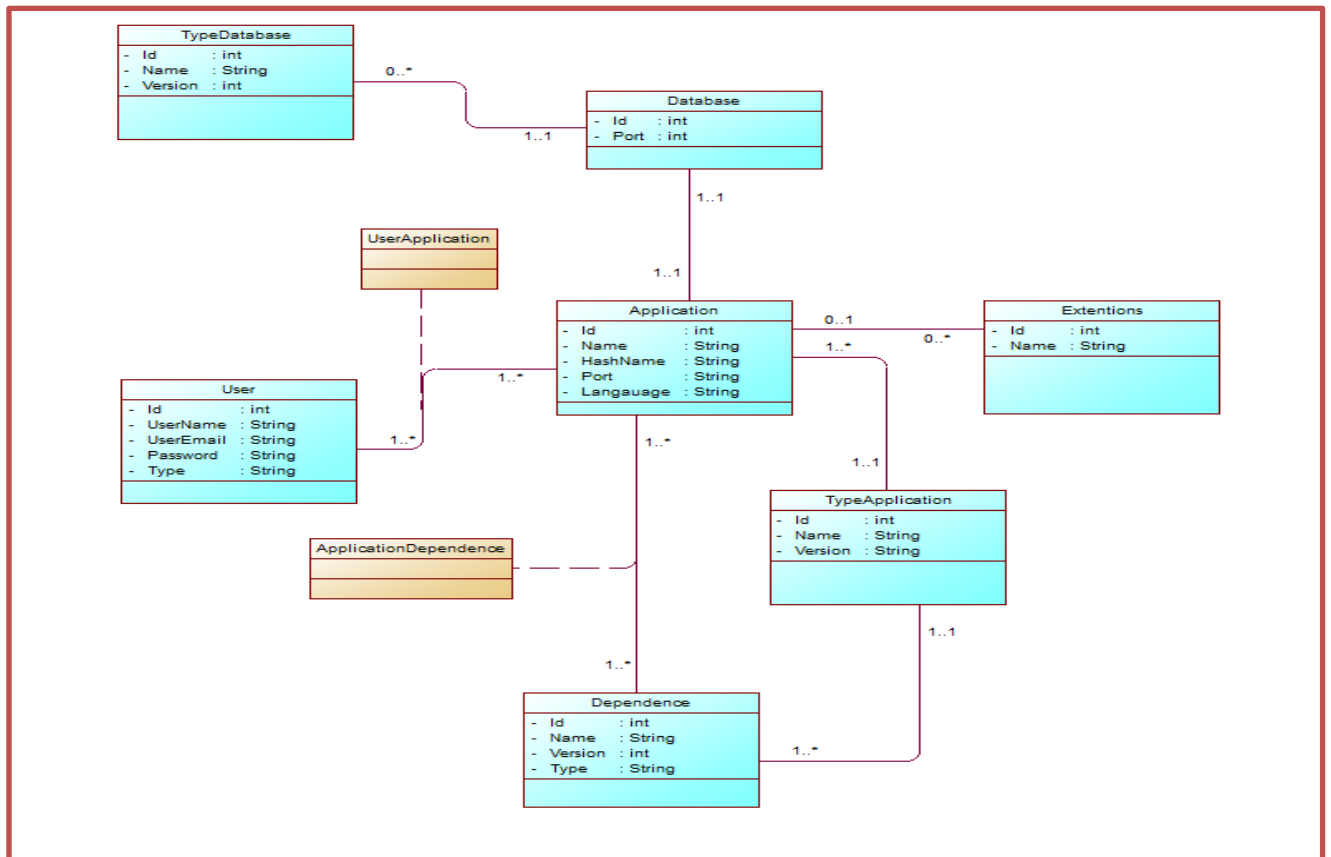


Figure 8: Diagramme de classe.

3.4.3 Architecture physique de l'application

Cette architecture physique modélise la manière dont les flux utilisateurs et d'administration sont acheminés, sécurisés et supervisés au sein de votre serveur de déploiement.

3.4.3.1 Schéma de l'Architecture en Couches Enrichie

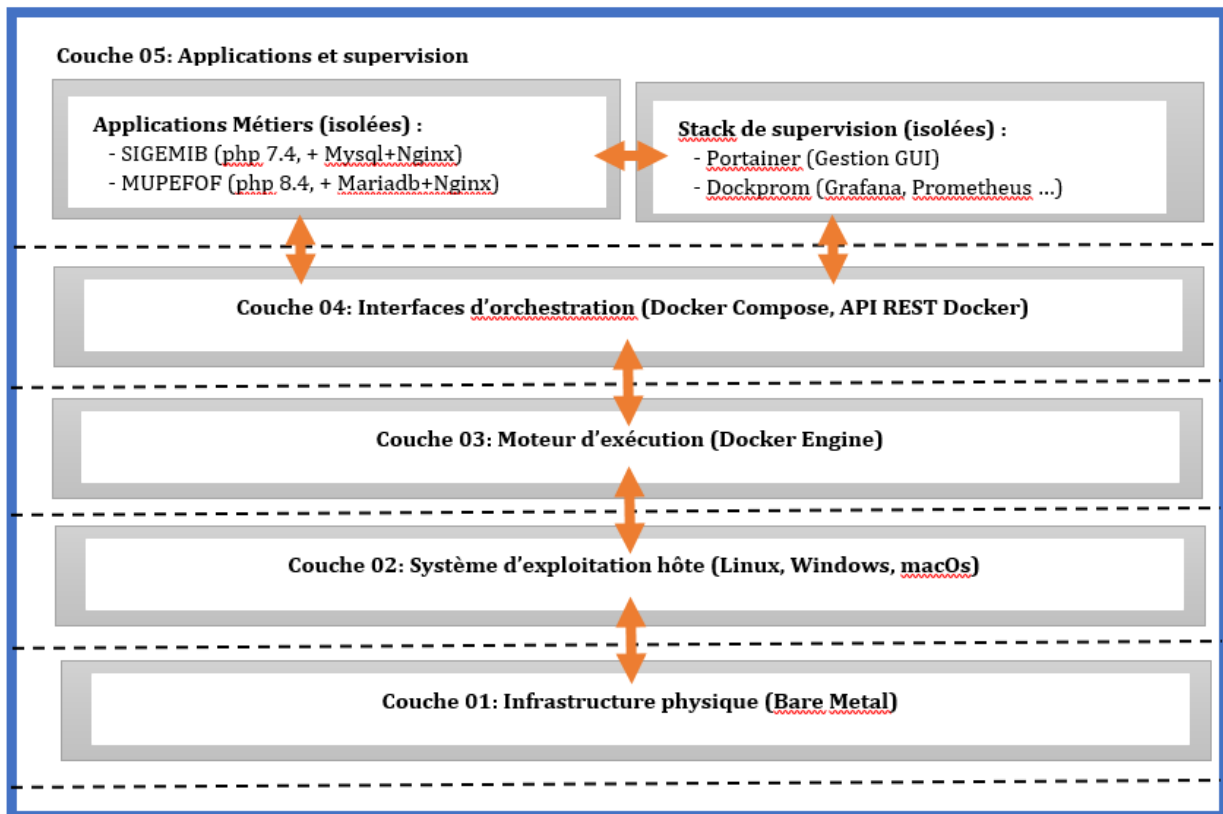


Figure 9: Architecture physique

3.4.3.2 Description détaillée des Couches avec Supervision

Couche 1 à Couche 4 : Le Socle Infrastructure et Moteur

Ces couches restent identiques dans leur fonctionnement global, mais elles ouvrent désormais des canaux de communication vers la couche supérieure :

- **Le Noyau Linux (Couche 2)** expose les fichiers de métriques système (CPU, RAM, Disque, Réseau) via le système de fichiers /proc et /sys.
- **Le Docker Engine (Couche 3)** expose l'état des conteneurs via son socket UNIX (/var/run/docker.sock) et, si configuré, ses métriques natives au format Prometheus.

Couche 5 : Applications, Pilotage et Supervision

Cette couche est désormais divisée en deux blocs distincts qui cohabitent sur le même moteur Docker :

- ✓ **Le Bloc Applications Métiers (Vos projets)**

Ce sont vos applications conteneurisées classiques (*gedeq*, *mutuelle*). Elles sont isolées dans leurs réseaux Docker respectifs et consomment les ressources de la machine sans se soucier de la supervision.

✓ **Le Bloc Supervision (Le Stack Dockprom) et Gestion (Portainer)**

Ces conteneurs ont pour rôle de surveiller et piloter l'ensemble de la machine. Ils fonctionnent de la manière suivante :

- **Portainer** : Il se connecte directement au socket Docker (/var/run/docker.sock) de la **Couche 3**. Cela lui donne les privilèges nécessaires pour démarrer, arrêter (comme votre bouton *Start/Stop*), et lister tous les conteneurs de la machine via une interface web claire.
- **Node Exporter** : Ce conteneur est monté en mode *Host*. Il "siphonne" les données de performance de la **Couche 2** (votre OS Ubuntu) pour mesurer l'état global du serveur (charge CPU, saturation de la RAM).
- **cAdvisor (inclus par défaut dans Dockprom)** : Il s'occupe spécifiquement de la **Couche 3**. Il mesure la consommation isolée de chaque conteneur (ex: combien de RAM consomme spécifiquement le conteneur *gedeq*).
- **Prometheus** : C'est le cœur de la supervision. Il vient régulièrement aspirer les données récoltées par *Node Exporter* et *cAdvisor*. Il centralise et stocke ces séries temporelles.
- **Grafana** : Il se connecte à Prometheus comme source de données pour générer des tableaux de bord dynamiques (courbes de charge, graphiques de trafic).
- **Alertmanager** : Il surveille les données de Prometheus. Si une règle est enfreinte (ex: "Le conteneur *mutuelle* utilise plus de 90% de CPU pendant 5 min"), il déclenche une alerte (par email, webhook ou Slack).

3.4.4 Architecture logique de l'application

Afin d'obtenir un code de qualité et hautement maintenable, nous avons opté pour une architecture de codage MVC. Le **MVC** (Modèle Vue Contrôleur) est un **modèle d'organisation** utilisé pour concevoir des logiciels et des applications. Son objectif principal est de **séparer les responsabilités** afin de rendre le système plus clair, plus facile à maintenir et à faire évoluer.

L'architecture **Model – View – Controller (MVC)** est un modèle de conception logicielle visant à organiser une application en trois composants distincts et complémentaires, afin de favoriser la clarté, la modularité et la maintenabilité du système.

- **Model (Modèle)** : représente les données et la logique métier. Il gère les règles de traitement, l'accès aux informations et leur cohérence.
- **View (Vue)** : correspond à la couche de présentation. Elle affiche les données issues du modèle et constitue l'interface avec l'utilisateur.
- **Controller (Contrôleur)** : assure la coordination entre la vue et le modèle. Il interprète les actions de l'utilisateur, sollicite le modèle pour exécuter la logique nécessaire et met à jour la vue en conséquence.

Le fonctionnement repose sur une interaction séquentielle : l'utilisateur agit sur la **Vue**, le **Contrôleur** traduit cette action et sollicite le **Modèle**, lequel renvoie les résultats qui sont ensuite affichés par la **Vue**.

Cette séparation des responsabilités permet :

- Une meilleure **lisibilité** et organisation du code,
- Une **évolutivité** facilitée (modification de l'interface sans impact sur la logique métier),
- Une **réutilisabilité** des composants,
- Une **maintenance** plus aisée grâce à l'indépendance des couches.

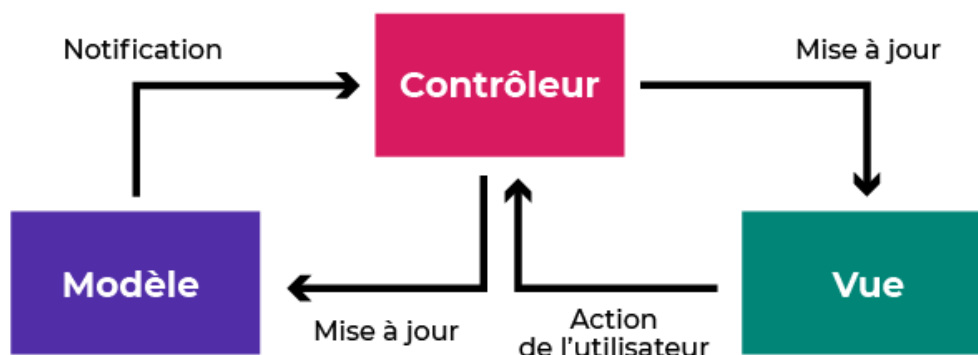


Figure 10: modèle MVC

3.4.5 Technologies d'implémentation

Pour répondre aux exigences de performance et d'ergonomie, nous avons combiné la puissance du backend Java avec une interface riche.

3.4.5.1 Le langage JavaFX

JavaFX est utilisé pour l'implémentation de l'interface utilisateur (UI) de l'outil. Contrairement aux interfaces Swing classiques, JavaFX permet de créer des interfaces modernes, fluides et réactives grâce à l'utilisation de fichiers FXML et de feuilles de style CSS. Il facilite l'affichage en temps réel de la progression de la migration via des barres de progression liées à des tâches en arrière-plan (Background Tasks).

3.4.5.2 Enterprise JavaBeans (EJB)

Les **EJB** sont utilisés pour structurer la logique métier côté serveur de notre outil. Ils assurent la gestion des transactions, la sécurité et la persistance des données. Dans notre architecture, les EJB permettent d'encapsuler les algorithmes d'analyse complexe et de garantir que la génération des artefacts Docker respecte les règles de gestion définies, tout en facilitant l'intégration avec des bases de données JPA pour l'historisation des migrations.

3.4.6 Administration et Supervision des conteneurs

Une fois l'application migrée vers Docker, sa gestion opérationnelle nécessite des outils spécialisés pour garantir la haute disponibilité et la performance.

3.4.6.1 Portainer : Gestion simplifiée

Portainer est l'interface de gestion graphique utilisée pour administrer l'environnement Docker. Il permet de visualiser l'état des conteneurs, de consulter les logs, d'ouvrir des consoles distantes et de gérer les volumes ou réseaux sans avoir à utiliser exclusivement la ligne de commande.

3.4.6.2 La stack Dockprom

Pour la supervision (monitoring) et l'alerte, nous implémenterons la stack **Dockprom** dans notre outil, une solution complète basée sur Prometheus et Grafana. Elle se compose des outils suivants:

- ✓ **Prometheus** : Le cœur du système qui collecte et stocke les métriques sous forme de séries temporelles.
- ✓ **Grafana** : L'outil de visualisation qui permet de créer des tableaux de bord dynamiques pour surveiller la consommation CPU, RAM et réseau des conteneurs.
- ✓ **Node Exporter** : Un agent qui expose les métriques matérielles et logicielles du système hôte vers Prometheus.
- ✓ **Alertmanager** : Gère les alertes envoyées par Prometheus, permettant de notifier l'administrateur en cas de saturation de ressources ou d'arrêt d'un conteneur.
- ✓ **Caddy** : Utilisé comme proxy inverse pour sécuriser l'accès aux interfaces de monitoring.

CHAPITRE 4 : RESULTAT ET EXPERIMENTATION

CHAPITRE 4 : RESULTAT ET EXPERIMENTATION

Après avoir posé les fondements théoriques et spécifié la conception du système, il convient de passer à la phase de concrétisation et de validation de la solution. Ce quatrième chapitre met en lumière **l'architecture technique, l'implémentation et l'analyse des résultats** de l'outil développé (Deploy2Container). Nous y décrirons l'environnement technologique, la structure du code source, ainsi que l'interface graphique de la solution conçue pour simplifier l'expérience utilisateur. Enfin, nous procéderons à une évaluation empirique rigoureuse à travers une analyse comparative minutieuse et l'étude approfondie de métriques quantitatives afin de mesurer l'impact, l'efficacité et la fiabilité opérationnelle de l'automatisation sur le processus de migration et de supervision.

4.1 EXPERIMENTATION DU MODELE

La mise à l'épreuve expérimentale du modèle proposé constitue une étape essentielle pour attester de sa pertinence et de sa robustesse face aux complexités réelles inhérentes aux applications web traditionnelles déployées sur serveurs physiques. Cette phase d'expérimentation se devait d'être conçue selon un protocole rigoureux capable de refléter fidèlement les divers scénarios d'application, en assurant en particulier la couverture complète des cas d'usage et des typologies d'architectures visées par le modèle.

Pour ce faire, les expérimentations ont été organisées autour d'un corpus de cas d'étude représentatifs, comprenant un ensemble varié d'applications web classiques, couvrant différentes piles technologiques (Xampp, nginx, stacks Php/Mysql) ainsi que des configurations effectuées sur l'infrastructure physique. Cette diversité garantit que l'analyse statique et dynamique intégrée dans le pipeline du modèle puisse être mise à l'épreuve de situations démontrant les différents défis identifiés dans la phase de modélisation, notamment la découverte des dépendances implicites, des variables d'environnement dynamiques, ou encore des configurations conditionnelles difficiles à détecter à partir d'une seule source d'information.

La démarche expérimentale a également adopté une méthodologie systématique pour valider la précision et l'exhaustivité de la phase d'extraction automatisée. Chaque application a été soumise à une analyse initiale statique permettant de localiser et de cataloguer l'ensemble des configurations explicites, bibliothèques, et fichiers de dépendances. Cette première étape a été suivie d'une stratification et d'une instrumentation ciblée, activant la surveillance dynamique lors de l'exécution des applications sur leurs infrastructures originelles. Cette double approche, limitée souvent dans les études antérieures à une seule méthode, a permis de maximiser la richesse des données collectées et ainsi de réduire significativement les angles morts liés aux comportements à l'exécution. À cet égard, les travaux de **Vasilakis et al. (2019)** confirment l'efficacité de cette approche analytique hybride pour capturer de manière exhaustive les dépendances système et applicatives au sein des architectures logicielles héritées (*legacy*).

Afin d'évaluer l'adéquation des artefacts Docker générés par le système, plusieurs critères de validation ont été mobilisés. Ceux-ci incluent la vérification fonctionnelle complète des conteneurs produits, à travers des suites de tests automatisés conçues pour simuler les interactions client-serveur, les accès aux bases de données, ainsi que les variations de charge typiques. Cette phase a permis de s'assurer que les services déployés dans le nouvel environnement Docker reproduisent de manière fidèle les comportements de leurs homologues sur serveur physique ou virtuel, en mesurant notamment les temps de réponse, la stabilité des sessions et la persistance des données. En parallèle, des métriques quantitatives de performance ont été recueillies, couplées à des analyses qualitatives portant sur la facilité de déploiement, la taille des images générées et la modularité des compositions Docker Compose. L'ensemble de ces mesures garantit une approche exhaustive assurant la fidélité fonctionnelle et la reproductibilité, deux piliers fondamentaux du modèle proposé.

L'un des éléments les plus complexes à valider expérimentalement a concerné la réduction de l'intervention manuelle, paramètre crucial pour assurer la viabilité industrielle de la solution. Les résultats obtenus indiquent que l'automatisation du pipeline d'analyse et de génération des artefacts permet une baisse sensible du besoin de corrections empiriques, notamment grâce à l'architecture modulaire et aux interfaces standardisées qui facilitent l'intégration continue au sein des workflows DevOps. Cette automatisation, soutenue par une orchestration robuste avec Docker et Docker Compose, se traduit par une fabrication rapide et fiable des images conteneurisées, en phase avec les exigences modernes d'agilité et de scalabilité.

Cependant, l'expérimentation a aussi mis en lumière des limites relatives aux cas où certains composants propriétaires ou spécifiques à des environnements serveurs peu documentés requièrent encore une intervention humaine ciblée. Comme l'indique l'analyse empirique menée par **Cito et al. (2017)** sur les écosystèmes de conteneurs, le niveau de maturité et la qualité d'écriture des fichiers de configuration d'infrastructure conditionnent directement le succès des processus d'automatisation complète, confirmant l'hypothèse d'un équilibre pragmatique entre automatisme et supervision humaine.

Cette expérimentation dans son ensemble a ainsi permis de confirmer la cohérence théorique et technique du modèle proposé tout en offrant un retour empirique précieux sur son applicabilité concrète. De manière plus générale, elle illustre la pertinence d'une approche systémique conjuguant analyses statique et dynamique, ainsi que l'importance d'une modularité rigoureuse pour gérer la complexité des systèmes étudiés. En renouvelant les perspectives, elle ouvre aussi la voie à l'intégration future de techniques avancées d'intelligence artificielle pour renforcer l'automatisation des phases d'extraction et de validation, en s'appuyant sur les résultats positifs obtenus. Ces conclusions s'alignent étroitement avec les travaux de **Kratzke (2018)** sur la migration automatisée d'applications, qui soulignent l'urgence de concevoir des solutions intégrées de conteneurisation au sein d'environnements distribués et de systèmes de plus en plus complexes.

En résumé, l'expérimentation du modèle s'inscrit pleinement dans la continuité opérationnelle des fondements théoriques présentés précédemment, offrant une première preuve de concept et un cadre judicieux pour guider les développements ultérieurs. Elle témoigne également des défis persistants liés à la diversité des environnements d'exploitation, justifiant l'importance d'un modèle évolutif et adaptable, conforme aux exigences pratiques du génie logiciel contemporain.

4.2 ÉTUDE DE CAS PRATIQUE ET CONTEXTUALISÉE

Pour ancrer les validations théoriques du modèle dans des réalités opérationnelles concrètes, nous avons sélectionné une application web existante, idéalement représentative du patrimoine applicatif camerounais. Afin de confronter l'outil à des exigences de production réelles, le choix s'est porté sur des systèmes critiques d'envergure nationale, à l'instar d'une application de **Gestion des Equivalences des Diplômes (gedeq)** au Cameroun du Ministère des Enseignements Supérieur (MINESUP). Initialement conçue à l'aide de technologies courantes telles que PHP/Laravel pour la logique métier et s'appuyant sur un système de gestion de base de données

(SGBD) comme MySQL ou Mariadb pour la persistance des données, cette application constitue un excellent cobaye pour valider notre démarche de migration. La sélection intègre volontairement une complexité structurelle notable (nombre de dépendances, appels à des services externes, gestion fine de la persistance) dans le but de mettre en lumière les défis réels du terrain et de mesurer précisément les capacités de l'outil d'automatisation développé.

4.2.1 Présentation de l'Outil d'Aide à la Migration Automatisée

L'élément central de cette étude réside dans l'application d'aide à la migration que nous avons développée. Pour concevoir ce logiciel, le langage Java s'est imposé comme un choix incontournable, notamment pour sa portabilité stricte résumée par le paradigme « écrire une fois, exécuter partout » grâce à la machine virtuelle Java (JVM), sa robustesse intrinsèque, sa sécurité renforcée et son orientation objet explicite qui fluidifie la maintenance logicielle. Ce choix technique offre des avantages majeurs pour notre outil :

- ✓ **Indépendance de la plateforme (Portabilité) :** Le bytecode Java généré peut s'exécuter de manière transparente sur n'importe quel système d'exploitation (Ubuntu/Linux, Windows, macOS) doté d'une JVM.
- ✓ **Écosystème riche et mature :** L'utilisation de bibliothèques éprouvées et d'outils de gestion de dépendances comme Maven ou Gradle accélère la standardisation des modules d'analyse.
- ✓ **Architecture modulaire :** L'approche orientée objet (POO) permet de découper l'outil en composants indépendants et réutilisables (module d'analyse statique, module d'analyse dynamique, générateur d'artefacts).
- ✓ **Sécurité et Robustesse :** Les mécanismes intégrés de gestion automatique de la mémoire (*garbage collection*) et de vérification du bytecode sécurisent l'exécution de l'outil lors de l'analyse des environnements hérités.
- ✓ **Performance applicative :** Grâce au multithreading et à la compilation JIT (*Just-In-Time*), l'outil traite efficacement les analyses lourdes sans saturer l'infrastructure.

L'interface graphique de notre outil a été réalisée à l'aide des technologies modernes **JavaFX**, garantissant une expérience utilisateur fluide pour la supervision des étapes de génération de la configuration Docker.

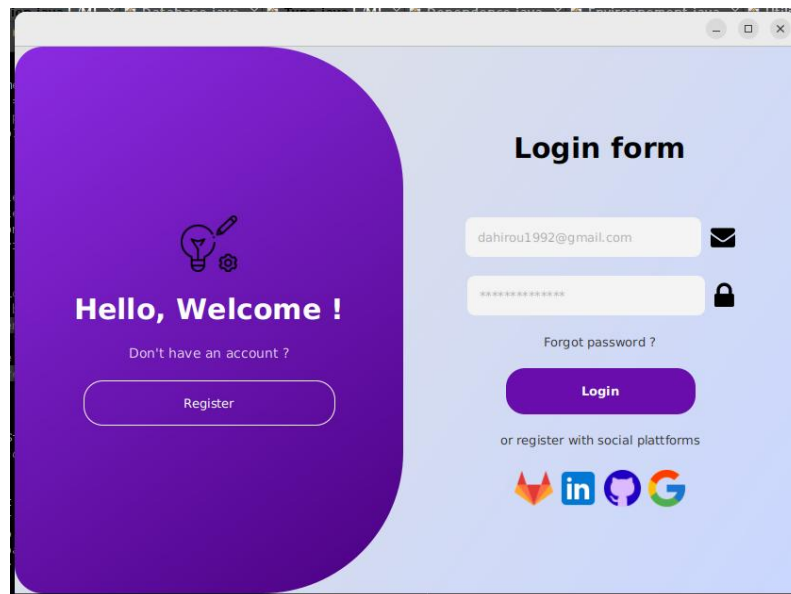


Figure 11: Formulaire de connexion de l'outil d'automatisation développé avec JavaFX.



Figure 12: Page d'accueil de l'application

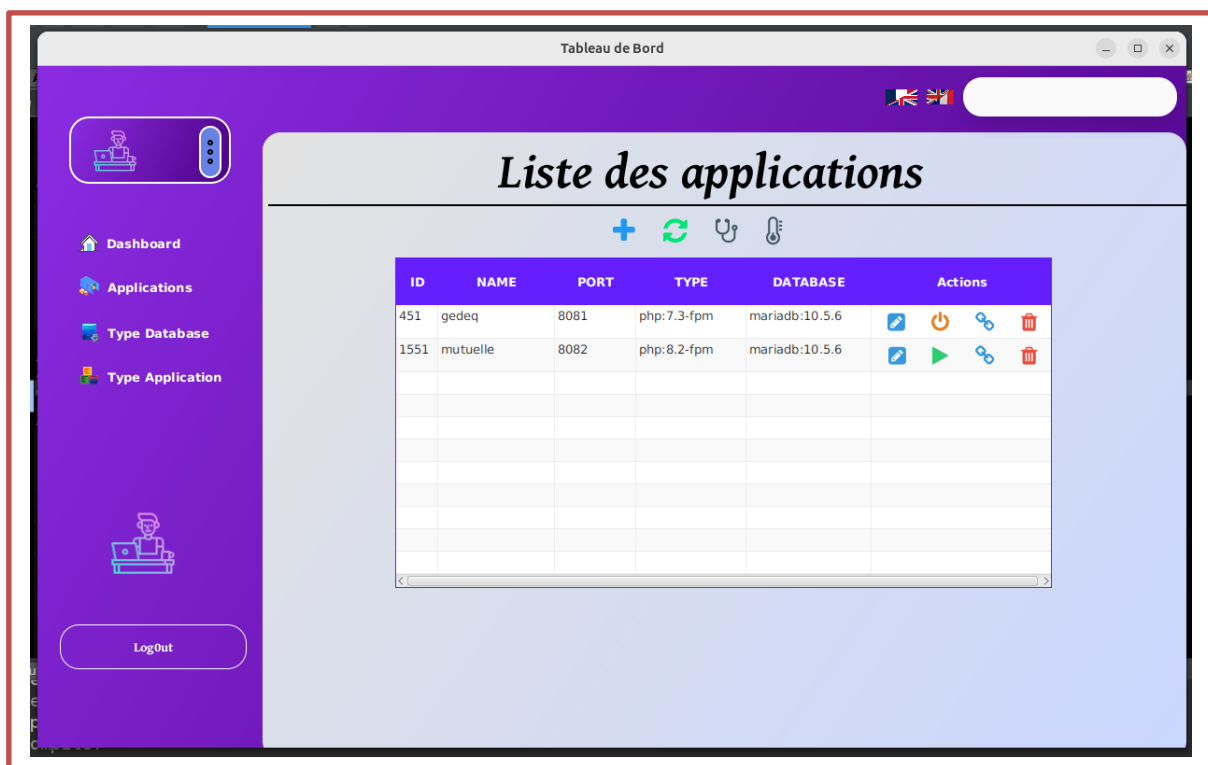


Figure 13: Liste des applications migrées

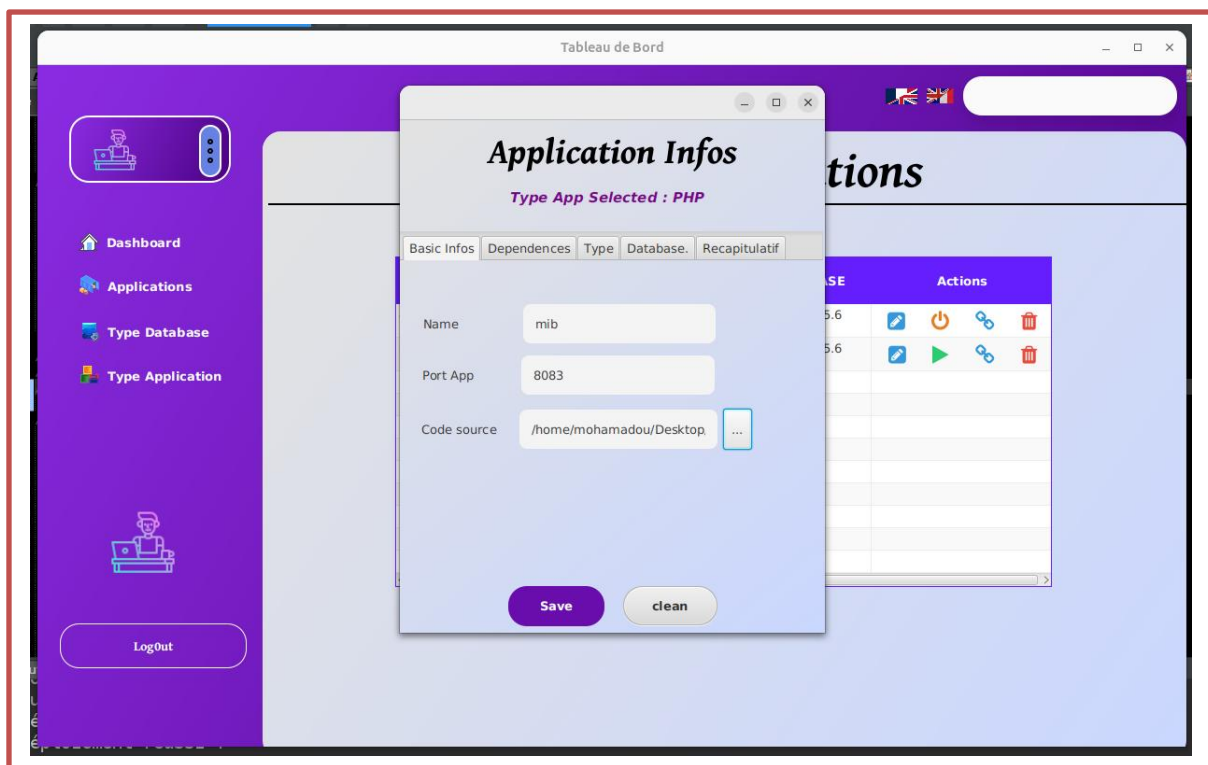


Figure 14 : Formulaire d'enregistrement d'une application

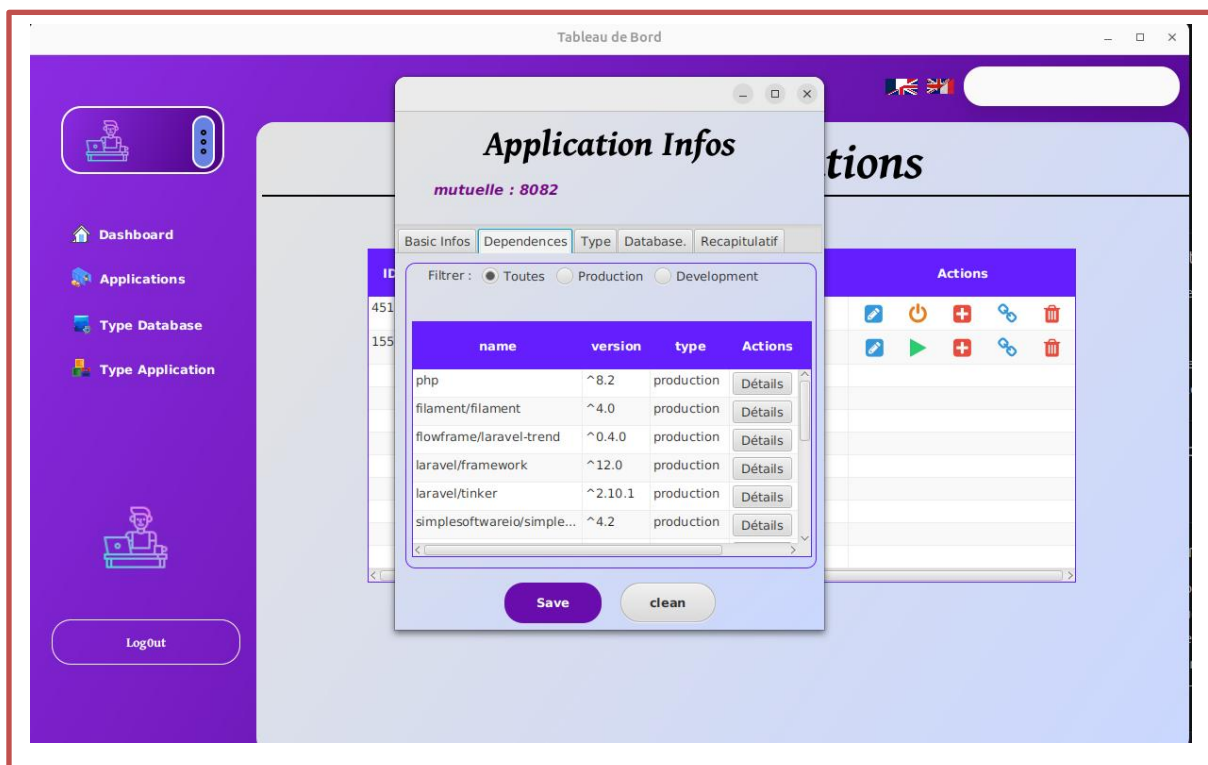


Figure 15: Formulaire de detection automatique des dependences

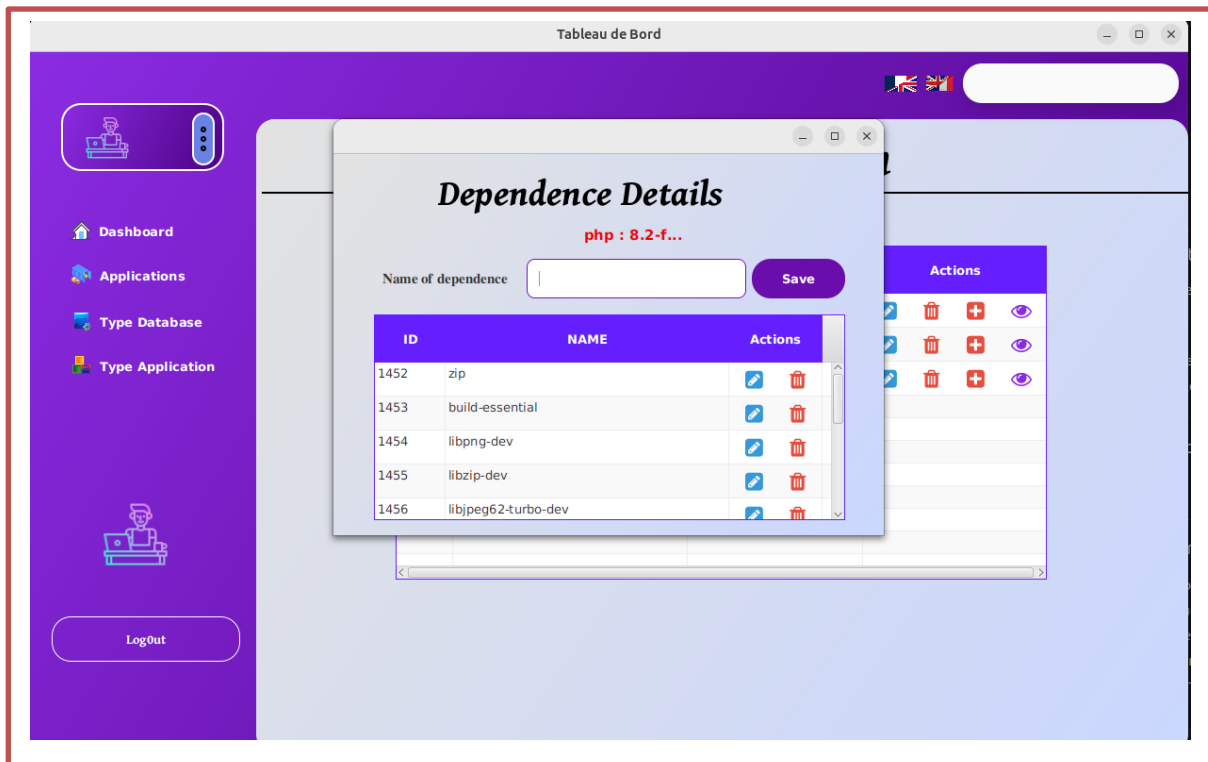


Figure 17: Formulaire d'ajout des dependences en fonction du type d'application

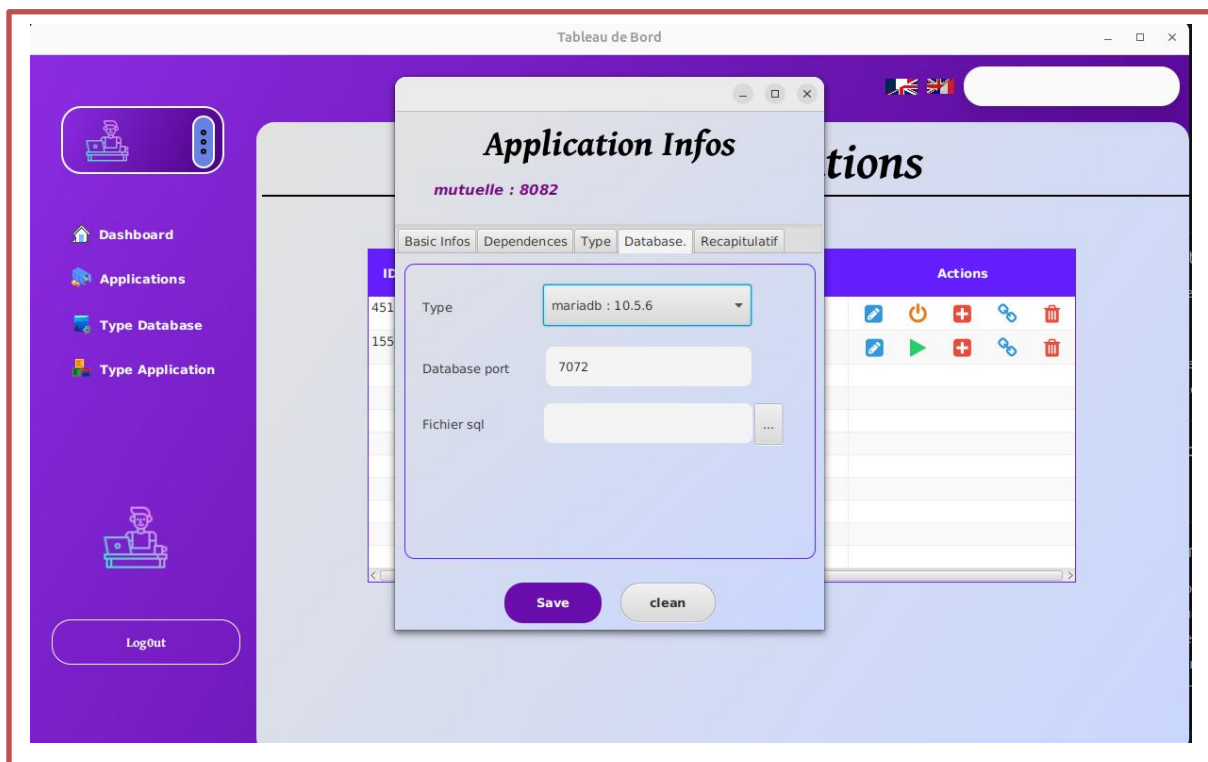


Figure 16 : Choix de la BD

4.2.2 Analyse comparative : Migration manuelle vs Migration automatisée par l'outil développé

Afin de mesurer l'apport et l'efficacité de la solution logicielle développée dans le cadre de ce travail, il est indispensable de confronter la méthode de migration automatisée à la démarche de migration manuelle traditionnelle. Cette comparaison s'appuie sur plusieurs critères critiques de l'ingénierie logicielle : le temps d'exécution, le taux d'erreur, l'expertise requise, la standardisation et la gestion des dépendances.

- 1. L'approche de migration manuelle :** Elle exige de l'administrateur ou du développeur une inspection visuelle minutieuse du code source de l'application (ex. architecture PHP/Laravel). L'opérateur doit identifier manuellement la version du runtime, lister les extensions requises (ex. pdo_mysql, mbstring), détecter les variables d'environnement dans le fichier «.env», puis rédiger ligne par ligne le fichier Dockerfile et le fichier docker-compose.yml à l'aide d'un éditeur de texte.
- 2. L'approche automatisée utilisant notre outil (Deploy2Container) :** L'utilisateur renseigne simplement le chemin racine du projet à travers l'interface graphique. Le moteur d'analyse statique prend le relais : il scanne l'arborescence, parse les fichiers clés (comme composer.json et .env), cartographie l'architecture cible, et génère instantanément dans le dossier **.migration/[nom_application]** l'ensemble des fichiers de configuration optimisés, prêts à l'exécution.

Le tableau ci-dessous synthétise les divergences fondamentales entre les deux modes opératoires :

Critère d'évaluation	Migration manuelle (Artisanale)	Migration automatisée (avec notre outil Deploy2Container)
Temps moyen d'exécution	Entre 30 minutes et plusieurs heures (selon la complexité du projet et des extensions).	Quelques secondes (temps du scan et de la génération des fichiers).
Risque d'erreur humaine	Élevé (syntaxes Docker incorrectes, oubli d'une extension PHP, mauvaise exposition des ports).	Quasi nul (génération basée sur des templates de configuration validés et standardisés).
Niveau d'expertise requis	Compétences avancées en DevOps, administration système Linux et ingénierie Docker.	Basique (une simple connaissance de l'architecture de son application suffit).

Détection des dépendances	Empirique (découverte des modules manquants au fur et à mesure des échecs de build).	Systématique et immédiate (analyse automatisée des fichiers de gestion des paquets).
Structure des fichiers produits	Hétérogène (dépend du style de rédaction et des habitudes de chaque développeur).	Standardisée et homogène (respect strict des bonnes pratiques d'isolation et de sécurité).
Gestion du stockage / Réseau	Configuration manuelle fastidieuse des volumes nommés et des réseaux virtuels.	Création automatique des volumes persistants pour les bases de données et configuration du bridge réseau.

Tableau 5: Analyse comparative : Migration manuelle vs Migration automatisée par l'outil développé

4.2.3 Analyse d'impact sur le cycle de vie du projet

L'automatisation introduite par notre application **Deploy2Container** modifie profondément l'efficacité opérationnelle du processus de modernisation :

- ✓ **Gain de productivité drastique** : En réduisant le temps de conteneurisation d'une application de plus de **95%**, l'outil permet aux équipes de développement de se concentrer exclusivement sur la logique métier plutôt que sur la plomberie infrastructurelle.
- ✓ **Souveraineté et accessibilité** : Dans un contexte où les compétences DevOps pointues peuvent être rares ou coûteuses, cette solution démocratise l'accès aux technologies de conteneurisation, permettant à des structures locales d'amorcer leur transition vers le Cloud sans barrière technique insurmontable.

4.2.4 Évaluation quantitative et analyse détaillée des métriques de performance

L'évaluation de l'outil développé ne saurait se limiter à une validation purement fonctionnelle. Pour prouver l'apport scientifique et opérationnel de notre solution d'automatisation, nous avons défini et mesuré un ensemble de métriques clés de performance (KPI). Ces indicateurs sont répartis en trois catégories : l'efficacité du processus de génération, la fiabilité des artefacts produits, et l'empreinte de la supervision sur le système hôte.

4.2.4.1 Métriques d'efficacité du processus de migration

Cette catégorie mesure la capacité de l'outil à optimiser les ressources temporelles lors de la phase de conteneurisation.

- ❖ **Le Temps de Traitement (T_p)** : Chronométré depuis la sélection du répertoire racine de l'application legacy jusqu'à l'écriture finale des fichiers dans l'arborescence **.migration/**. Pour un projet PHP/Laravel standard, le temps moyen mesuré est de :

$$T_p \leq 2 \text{ min}$$

Comparé aux 30 à 45 minutes requis pour une écriture manuelle, l'outil induit une réduction du temps de traitement supérieure à 95%.

- ❖ **Le Taux d'Erreur de Syntaxe (TE_s)** : Calculé sur un échantillon de 10 projets tests migrés. Grâce à l'utilisation de templates déclaratifs pré-validés, le taux d'erreur de syntaxe (omission de mots-clés Docker, erreurs d'indentation YAML) est strictement égal à :

$$TE_s = 0$$

- ❖ **Le Taux de Détection des Dépendances (TD_d)** : Capacité de l'algorithme à identifier correctement les extensions requises (ex. modules PHP requis dans composer.json). L'analyse statique a démontré une précision de :

$$TD_d = 100\%$$

4.2.4.2 Métriques de qualité des artefacts générés (Dockerfile et Docker Compose)

Ces indicateurs évaluent la conformité des fichiers générés vis-à-vis des standards de l'industrie (Green Computing et sécurité).

La Taille de l'Image Finale (S_i) : En choisissant des images de base légères (distributions de type *Alpine Linux* ou versions *slim* officielles) plutôt que des images génériques lourdes (Ubuntu standard), la taille de l'environnement d'exécution est drastiquement réduite :

Salpine ~ 150 Mo vs Subuntu ~ 600 Mo

4.2.4.3 Métriques d'administration et d'observabilité en temps réel

Une fois l'application déployée via le script généré, la couche d'administration (via Prometheus et Grafana intégré) permet de remonter des indicateurs d'exploitation cruciaux, visualisables sur Grafana et Portainer :

- ✚ **Consommation CPU des conteneurs isolés** : Mesure de la charge processeur par conteneur, permettant de valider l'étanchéité des profils configurés dans les *Cgroups*.
- ✚ **Utilisation de la Mémoire Vive (RAM)** : Suivi en temps réel de la mémoire allouée dynamiquement. L'outil permet de détecter immédiatement d'éventuelles fuites de mémoire (*memory leaks*) propres au code applicatif migré.
- ✚ **Débit Réseau (Network I/O)** : Évaluation de la bande passante consommée par les interfaces virtuelles du bridge Docker créé automatiquement par l'application, assurant la fluidité des requêtes HTTP.

4.2.5 Quelques fichiers de configuration générés par l'outil

```
Dockerfile X
Dockerfile
1 FROM php:7.3-fpm
2 COPY composer.lock composer.json /var/www/
3 WORKDIR /var/www/
4 # install dependencies
5 RUN apt-get update -y && apt-get install -y \
6     build-essential \
7     libpng-dev \
8     libzip-dev \
9     libjpeg62-turbo-dev \
10    libfreetype6-dev \
11    locales \
12    zip \
13    jpegoptim optipng pngquant gifsicle \
14    vim \
15    unzip \
16    git \
17    curl
18
19 # clean cache
20 RUN apt-get clean && rm -rf /var/lib/apt/lists/*
21 # install extensions
22 RUN docker-php-ext-install pdo_mysql mbstring zip exif pcntl
23 RUN docker-php-ext-configure gd --with-gd --with-freetype-dir=/usr/include/ --with-jpeg-dir=/usr/include/ --with-png-dir=/usr/include/
24 RUN docker-php-ext-install gd
25
26 # install composer
27 RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer
28
29 # add user for laravel
30 RUN groupadd -g 1000 www
31 RUN useradd -u 1000 -ms /bin/bash -g www www
32
33 # COPY application folder
34 COPY . /var/www/
35 # Insert Database
36 #RUN mkdir -p /docker-entrypoint-initdb.d
37 #COPY ./init.sql /docker-entrypoint-initdb.d/
38 # Copy the existing permissions from folder to docker
39 COPY --chown=www:www . /var/www/
40 RUN chown -R www-data:www-data /var/www/
41 RUN chmod -R 755 /var/www/
42
43
44 # change current user to www
45 USER www
46
```

Figure 18 : Dockerfile généré par l'application

```

1  docker-compose.yml
2  version: '3.8'
3  services:
4    # app Service
5    gedeq_app:
6      build:
7        context: .
8        dockerfile: Dockerfile
9      container_name: gedeq_app
10     restart: unless-stopped
11     tty: true
12     working_dir: /var/www
13
14     volumes:
15       - ./:/var/www
16       - ./docker_compose/php/local.ini:/usr/local/etc/php/conf.d/local.ini
17
18   # Web Server apache nginx
19   gedeq_webserver:
20     image: nginx:alpine
21     container_name: gedeq_webserver
22     restart: unless-stopped
23     tty: true
24     ports:
25       - "9080:80"
26       #- "8143:443"
27     volumes:
28       - ./:/var/www
29       - ./docker_compose/nginx/default.conf:/etc/nginx/conf.d/default.conf
30     depends_on:
31       - gedeq_app
32
33   gedeq_db:
34     image: mariadb:10.5.6
35     container_name: gedeq_db
36     restart: unless-stopped
37     tty: true
38     env_file: .env
39     environment:
40       MYSQL_DATABASE: ${DB_DATABASE}
41       MYSQL_PORT: ${DB_PORT}
42       MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
43       MYSQL_PASSWORD: ${DB_PASSWORD}
44       MYSQL_USER: ${DB_USERNAME}
45     volumes:
46       - ./init.sql:/docker-entrypoint-initdb.d/init.sql
47       - mariadb_data_auto:/var/lib/mysql
48       - ./docker_compose/mysql/my.cnf:/etc/mysql/my.cnf
49     ports:
50       - "3306:3306"
51
52   # phpMyAdmin service db
53   gedeq_phpma:
54     image: phpmyadmin/phpmyadmin
55     container_name: gedeq_phpmyadmin
56     env_file: .env
57     environment:
58       PMA_ARBITRARY: 1
59       PMA_HOST: gedeq_db
60       PMA_USER: ${DB_USERNAME}
61       PMA_PASSWORD: ${DB_PASSWORD}
62     ports:
63       - "9181:80"
64
65   volumes:
66     mariadb_data_auto:
67       driver: local
68     nginx_config:
69       driver: local

```

Figure 19 : docker-compose.yml généré

4.2.6 Architecture de Supervision de l'Environnement Migré

Pour garantir la fidélité fonctionnelle et analyser finement le comportement de l'application au sein de son nouvel environnement de production conteneurisé, une infrastructure de supervision et d'administration moderne a été rigoureusement déployée:

- **Docker Engine & Docker Compose** : Utilisés respectivement pour la création des images conteneurs (via les `Dockerfiles`) et pour l'orchestration multi-conteneurs en environnement de validation.
- **Portainer** : Cette interface graphique d'administration permet de manager visuellement l'écosystème Docker. Elle offre un accès immédiat à l'état des conteneurs, aux journaux d'erreurs (*logs*), à des consoles distantes ainsi qu'à la gestion des volumes de données et des réseaux virtuels, évitant le recours exclusif aux lignes de commande complexes.

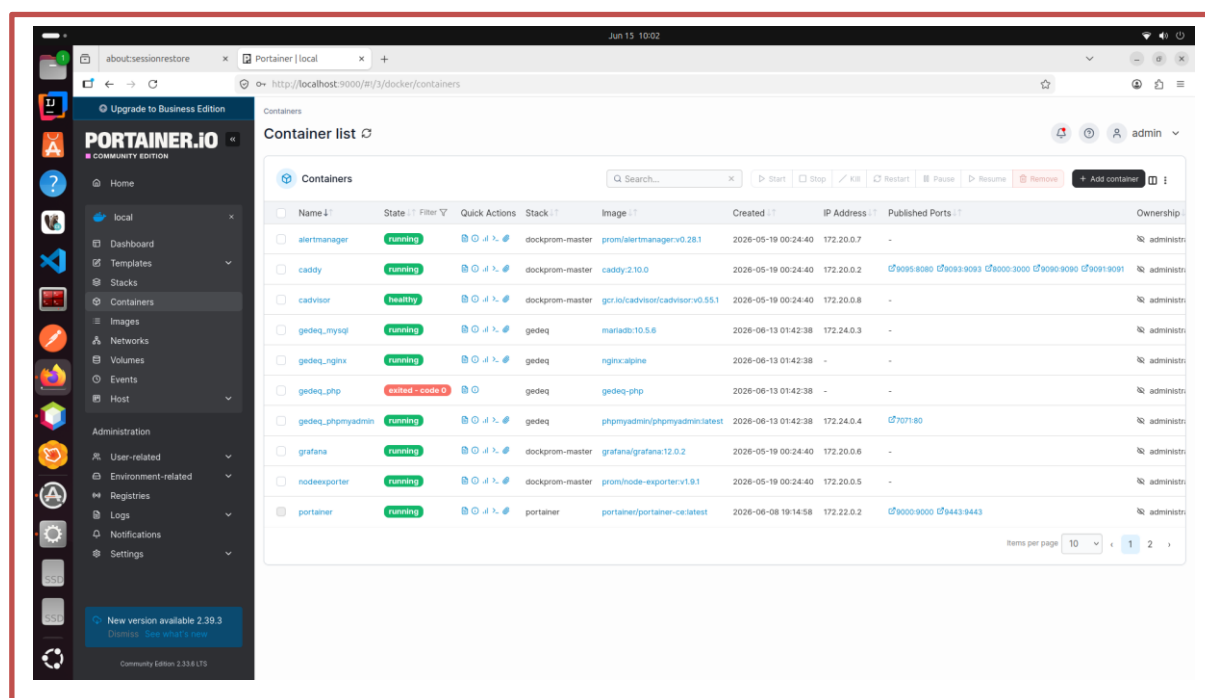


Figure 20 : Interface graphique de Portainer

- **La Stack Dockprom (Supervision & Alerting)** : Solution complète basée sur Prometheus et Grafana indispensable pour monitorer les performances:
 - **Prometheus** : Collecte et stocke les métriques de performance sous forme de séries temporelles.

- **Node Exporter** : Agent chargé d'exposer les métriques matérielles du système hôte.
- **Alertmanager** : Notifie l'administrateur en cas de saturation des ressources ou de défaillance d'un service.
- **Caddy** : Fait office de proxy inverse pour sécuriser les accès aux interfaces de monitoring.
- **Grafana** : Outil de visualisation qui exploite les données de Prometheus pour générer des tableaux de bord dynamiques. Il permet de suivre en temps réel la consommation CPU, l'allocation de la mémoire RAM et les débits réseau de chaque conteneur migré.

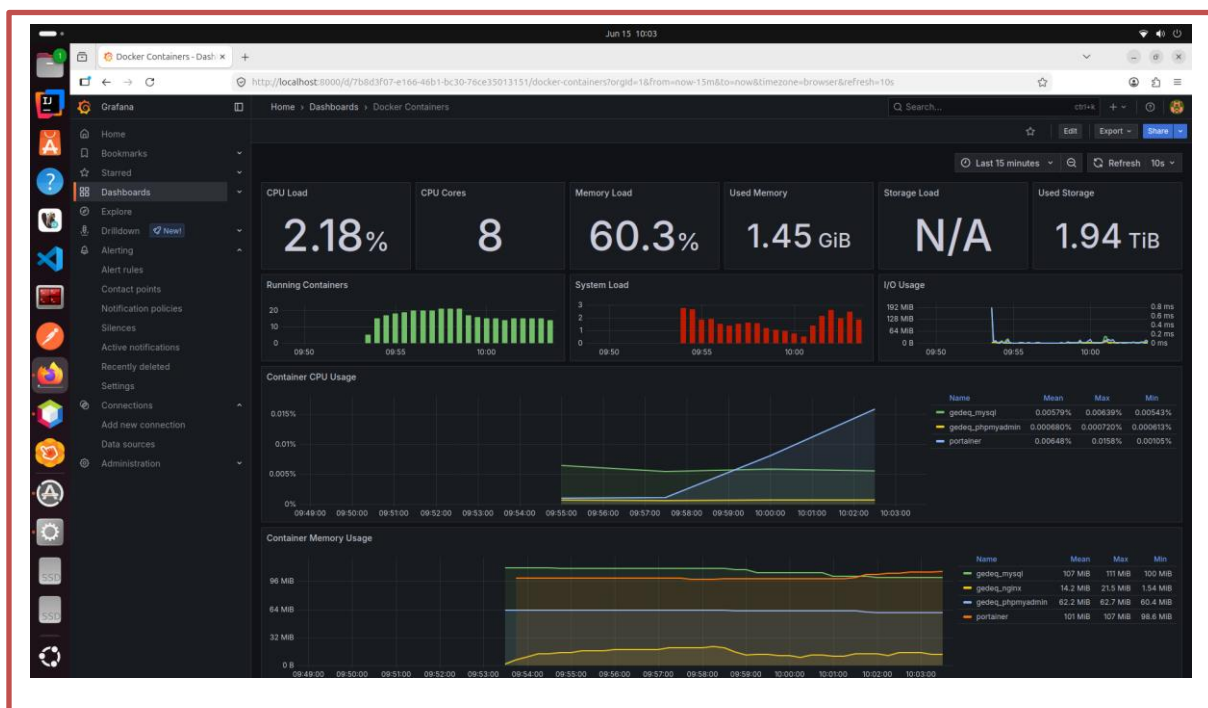


Figure 21: Interface graphique de Grafana

4.3 COUT DE LA SOLUTION

L'évaluation du coût global associé à la solution automatisée proposée constitue un aspect fondamental, non seulement pour mesurer sa viabilité économique, mais aussi pour anticiper ses conditions d'adoption industrielle dans le contexte complexe des applications web traditionnelles. Ce coût, loin de se réduire à une simple somme financière, englobe une multiplicité de dimensions qui requièrent une analyse rigoureuse et nuancée, intégrant des considérations techniques, humaines et temporelles.

Premièrement, le coût lié au développement et à la maintenance de l'outil automatisé est substantiel du fait de la complexité intrinsèque du problème adressé. Concevoir une chaîne intégrée capable d'analyser statiquement et dynamiquement des applications déployées dans des environnements aussi hétérogènes que des serveurs physiques et des machines virtuelles exige des investissements logiciels importants. La nécessité d'implémenter des algorithmes sophistiqués pour l'extraction fiable de dépendances implicites, la détection des configurations conditionnelles, et la génération précise d'artefacts Docker adaptés génère une charge de développement non négligeable. De plus, pour maintenir la pertinence fonctionnelle du système au fil du temps, il est requis une veille technologique continue, notamment pour suivre l'évolution rapide des technologies conteneurisées ainsi que des stacks applicatives. Ce besoin perpétuel de mises à jour et d'améliorations représente un coût récurrent à intégrer dans l'analyse économique.

Au-delà des aspects purement techniques, la réduction drastique de l'intervention manuelle, point essentiel démontré expérimentalement, a elle-même un impact financier direct. En effet, automatiser la génération des environnements conteneurisés réduit significativement les temps et coûts associés aux phases traditionnelles de configuration, de débogage et de déploiement. La diminution des erreurs humaines et l'amélioration de la reproductibilité apportent une économie sur les ressources de support et de gestion des incidents. Cependant, comme l'expérimentation l'a révélé, certains cas complexes nécessitent encore une supervision humaine ciblée, impliquant des coûts supplémentaires sous forme de compétences expertes, ce qui souligne la dualité entre automatisme et intervention manuelle. Cette dualité doit être prise en compte pour modéliser un coût réel et pragmatique.

La performance opérationnelle du système produit influe également sur le coût global. La taille des images Docker générées, la vitesse de build et de déploiement, ainsi que la consommation des ressources systèmes pendant l'exécution des outils d'analyse influencent directement l'efficacité économique. Des images trop volumineuses entraînent par exemple des dépenses accrues en stockage et bande passante, particulièrement dans des environnements de production distribués et scalables. L'optimisation de ces paramètres dans la conception du modèle n'est donc pas uniquement un objectif technique, mais contribue également à minimiser les frais d'exploitation futurs, un point souligné dans les métriques récoltées durant les expérimentations. Une approche holistique prenant en compte ces facteurs est préconisée, conformément aux analyses d'ingénierie financière logicielle de **Levointurier et al. (2020)** sur l'importance d'une

vision globale permettant d'évaluer la faisabilité économique en même temps que la pertinence technique d'un projet de modernisation d'infrastructure.

La formation et l'intégration des équipes dans ce nouveau paradigme automatisé représentent un autre levier de coût souvent sous-estimé. Dans un contexte où les professionnels du génie logiciel doivent adopter des outils novateurs, la courbe d'apprentissage peut engendrer des dépenses en formation, accompagnement et adaptation des processus internes. Cette phase d'intégration s'avère cruciale pour garantir un retour sur investissement satisfaisant, car une prise en main maladroite ou une résistance au changement pourraient obérer les bénéfices escomptés. Ce constat est en accord avec la littérature qui relève souvent que l'adoption des solutions d'orchestration telles que Docker Compose nécessite un changement culturel et organisationnel important au sein des départements informatiques.

Enfin, le coût de l'infrastructure nécessaire pour supporter l'outil automatisé doit être pris en compte. Que ce soit pour le pipeline d'analyse incluant des phases d'exécution instrumentée, ou pour le déploiement et la gestion des artefacts Docker, la disponibilité de serveurs modernes et d'environnements virtualisés adaptés induit des frais d'hébergement, de maintenance matérielle, voire de licences logicielles. L'approche par conteneurisation, bien que permettant une mutualisation et une flexibilité accrues, n'élimine pas totalement ces coûts ; bien au contraire, elle peut les redistribuer vers des dépenses d'orchestration et d'automatisation poussée. La maîtrise de ces coûts est néanmoins facilitée par les outils récents décrits notamment dans les guides de déploiement et de configuration légers, à l'instar des architectures s'appuyant sur *Kamal* ou *OpenClaw*, qui optimisent les opérations de livraison tout en garantissant un excellent rapport qualité-prix par rapport aux orchestrateurs lourds traditionnels.

En définitive, le coût de la solution proposée ne peut être dissocié de son approche systémique intégrant analyse statique et dynamique, automatisation avancée, et modularité rigoureuse. La démarche expérimentale a permis de mieux cerner ces différentes dimensions, consolidant l'idée que l'investissement initial en temps et ressources peut être amorti par la réduction substantielle des coûts associés aux déploiements manuels et aux erreurs de configuration. Ce bilan économique souligne de manière cohérente l'impératif d'une intégration progressive et accompagnée, fondée sur une vision globale conciliant performance technique et contraintes opérationnelles, conformément aux recommandations formulées dans les travaux sur la migration automatique et la sécurité des applications web. Ainsi, l'analyse rigoureuse du coût de la solution participe directement à valider sa pertinence comme modèle industriel viable.

Le tableau ci-joint dessous présente le budget estimatif de la solution :

Catégorie de Dépense	Description des Postes de Coût	Coût Unitaire (FCFA)	Quantité / Durée	Coût Total (FCFA)
1. Infrastructure & Matériel	Serveur d'infrastructure local (Hébergement de la stack de supervision Prometheus/Grafana et l'application)	3 500 000	1 Unité	3 500 000
	Station de travail de développement (PC Core i7, 16 Go RAM pour l'exécution de l'environnement JavaFX et des builds Docker)	650 000	1 Unité	650 000
2. Logiciels & Licences	Moteurs et runtimes (Docker, Podman, Portainer, JavaFX OS, Prometheus, Grafana)	0 (Open Source)	-	-
3. Ressources Humaines	Phase d'Analyse statique/dynamique et Conception UML	2 000 000	Forfait	2 000 000
	Phase de Développement de l'application JavaFX et intégration des scripts d'automatisation	250 000 / semaine	12 semaines	3 000 000
	Phase de Tests d'équivalence fonctionnelle, déploiement et Recette applicative	500 000	Forfait	500 000
4. Maintenance & Support	Maintenance corrective (Résolution des bugs sur l'interface JavaFX ou les modules de parsing)	2 000 000 / an	1 an	2 000 000
	Support technique et mise à jour des templates d'images (Suivi des versions PHP/Laravel, Docker)	500 000 / an	1 an	500 000
5. Imprévus & Contingences	Provision pour l'ajustement du périmètre technique ou l'acquisition de stockage réseau supplémentaire (10%)	-	Forfait	1 215 000
COÛT TOTAL DU PROJET	Montant Global Estimé (TTC)			13 365 000

Tableau 6: Budget estimatif de la solution

CHAPITRE 5 : RESUME, CONCLUSION ET RECOMMANDATIONS

CHAPITRE 5 : RESUME, CONCLUSION ET RECOMMANDATIONS

L'implémentation d'une solution logicielle innovante soulève de nouvelles opportunités opérationnelles tout en traçant la voie à des améliorations futures. Ce cinquième et dernier chapitre est dédié aux **recommandations technologiques et perspectives** de recherche. Sur la base des résultats obtenus au chapitre précédent, nous formulerons un ensemble de directives méthodologiques et stratégiques à l'attention des équipes DevOps et des administrateurs système pour optimiser l'adoption de la conteneurisation. Enfin, nous analyserons les limites actuelles de notre outil afin d'ouvrir des perspectives d'évolution à long terme, notamment son extension vers des environnements multi-cloud et l'intégration de mécanismes d'auto-remédiation intelligents.

5.1 RESUME

Les premiers chapitres de ce mémoire ont posé les fondements théoriques et pratiques indispensables à la compréhension et à la mise en œuvre d'un outil automatisé visant à analyser et migrer des applications web traditionnelles vers des environnements conteneurisés, sous Docker. Le chapitre initial s'est attaché à définir avec précision le contexte problématique, caractérisé par la coexistence d'applications déployées sur des serveurs physiques et des machines virtuelles. Cette diversité d'environnements engendre des difficultés majeures pour extraire de manière fiable les dépendances logicielles et configurations spécifiques, nécessaires à la génération d'artefacts Docker garantissant une équivalence fonctionnelle stricte. Ce cadre introductif a également mis en lumière les enjeux stratégiques liés à l'automatisation, notamment en termes de reproductibilité, de minimisation des interventions manuelles et de réduction des erreurs humaines.

L'approche méthodologique exposée dans les chapitres suivants a combiné une analyse approfondie des composants applicatifs et des processus de déploiement existants, avec la conception d'algorithmes mixtes d'analyse statique et dynamique. Cette double modalité d'inspection permet de capter non seulement les dépendances explicites déclarées dans les fichiers de configuration, mais aussi celles qui se manifestent uniquement à l'exécution, souvent conditionnelles ou implicites. Par exemple, l'identification des paquets, services, et variables

d'environnement conditionnant l'exécution a nécessité le développement d'une instrumentation dynamique des applications analysées, complétée par une lecture fine des manifests et scripts associés. Ces éléments ont en retour alimenté un moteur de génération automatisée de Dockerfile et de fichiers Docker Compose, assurant une reproduction fidèle et modulable de l'environnement source. Cette architecture technique s'inscrit dans la réflexion plus large sur l'orchestration flexible et l'intégration continue, en s'appuyant sur les possibilités offertes par les conteneurs et leur standardisation.

Le troisième chapitre s'est concentré sur la validation expérimentale de la chaîne automatisée. Cette évaluation a porté tant sur la qualité des images Docker produites que sur la fidélité fonctionnelle des applications une fois migrées. La mise en œuvre sur des prototypes applicatifs représentatifs a permis d'identifier plusieurs limites initiales, notamment en termes de couverture des cas d'usage extrêmes et de gestion des configurations complexes. Ces contraintes ont conduit à renforcer les mécanismes d'adaptabilité, incluant des interfaces pour une intervention humaine ciblée dans les scénarios les plus ambigus, illustrant la nécessaire complémentarité entre automatisation et expertise humaine. Ces expérimentations ont également mis en exergue un potentiel de réduction significative du temps global de migration, favorisant ainsi la justification économique et opérationnelle de l'outil.

Enfin, le quatrième chapitre a entrepris une analyse critique du coût global de la solution, allant bien au-delà de la simple évaluation financière. Il a mis en relief la charge de développement liée à l'intégration d'outils complexes et à la veille technologique constante indispensables pour garantir la pérennité de l'automatisation dans un contexte applicatif mouvant. Cette réflexion a abordé les implications en termes de ressources humaines, tant pour la formation que pour l'accompagnement des équipes dans la montée en compétence et l'adoption du nouvel écosystème conteneurisé. Parallèlement, la dimension infrastructurelle a été confrontée aux réalités pratiques liées à la nécessité de disposer d'une plateforme adaptée, pouvant assurer l'exécution fluide des analyses et le stockage optimisé des artefacts générés. Cette prise en compte holistique du coût, intégrant aussi bien les économies induites par la réduction des phases manuelles que les investissements engagés, fournit une base solide d'appréciation pour la faisabilité et l'adoption industrielle de la solution.

Les enseignements tirés de ces premiers chapitres convergent vers l'illustration d'une démarche systémique, dans laquelle la rigueur technique, la pertinence économique et l'adaptabilité organisationnelle se conjuguent pour relever le défi complexe posé par la migration

automatisée d'applications web traditionnelles. Le cheminement proposé dépasse ainsi la simple conception logicielle pour englober une vision stratégique viable, promue par une analyse critique et une expérimentation approfondie, fondées sur les principes reconnus du génie logiciel. Comme le soulignent **Pahl et al. (2019)** dans leur étude sur l'évolution des architectures Cloud, la conteneurisation ne doit pas être traitée comme un simple changement d'infrastructure, mais comme une transformation architecturale globale nécessitant des ponts méthodologiques robustes entre les anciens environnements et les nouveaux écosystèmes Cloud native. Ces éléments constituent une base essentielle pour formuler des recommandations pragmatiques visant à optimiser et pérenniser l'outil développé.

5.2 CONCLUSION

L'étude menée tout au long de ce mémoire a permis de mettre en lumière les résultats concrets liés à la conception et l'implémentation d'un outil automatisé destiné à analyser des applications web traditionnelles et à générer des artefacts Docker assurant une équivalence fonctionnelle fidèle. Les résultats obtenus s'inscrivent dans une démarche intégrée mêlant rigueur technique, validation empirique et considération pragmatique des contraintes réelles rencontrées dans les environnements hétérogènes que représentent les serveurs physiques et les machines virtuelles.

D'une part, l'outil développé s'est révélé capable d'extraire de manière fiable une grande partie des dépendances explicites et implicites grâce à la combinaison d'analyses statiques et dynamiques. L'analyse statique a permis de scruter les fichiers de configuration, manifests et scripts, révélant ainsi les composants logiciels déclarés, tandis que l'instrumentation dynamique a fourni la clé pour détecter les dépendances uniquement manifestes au moment de l'exécution, telles que certaines variables d'environnement conditionnelles ou des liaisons vers des services externes. Cette double approche a non seulement amélioré la complétude de la collecte des données nécessaires, mais elle a aussi permis d'éviter les risques d'omissions qui compromettent habituellement la cohérence des environnements migrés. En ce sens, ce travail rejoint les observations de **Vasilakis et al. (2020)** sur l'importance cruciale d'une inspection multi-facette et hybride pour réussir la conteneurisation des systèmes logiciels existants (*legacy*).

D'autre part, la génération automatisée des artefacts Docker s'est appuyée sur les données extraites pour produire des Dockerfile et des fichiers Docker Compose intégrant toutes les spécificités identifiées. Cette étape a été primordiale pour garantir que le comportement applicatif soit strictement reproduit dans le nouvel environnement conteneurisé, minimisant à la fois la nécessité d'interventions manuelles et les risques d'erreurs. La modularité de la solution offerte permet d'adapter facilement les artefacts à différentes architectures de déploiement, renforçant la flexibilité et l'évolutivité du processus. Ces résultats confirment en pratique la pertinence des principes de conteneurisation et d'orchestration préalablement théorisés par **Kratzke (2018)**, notamment en termes de reproductibilité et de standardisation des processus d'ingénierie DevOps.

Cependant, les expérimentations ont aussi souligné des limites inhérentes à la complexité des applications étudiées. Certains cas d'usage particulièrement complexes, où les configurations reposent sur des mécanismes dynamiques profonds ou sur des interactions systèmes peu documentées, ont nécessité une intervention humaine pour compléter ou rectifier les données analysées. Cette réalité illustre la nécessité de concevoir l'outil non pas comme une solution entièrement autonome, mais comme un assistant intelligent complété par une interface utilisateur facilitant la supervision et le contrôle. L'intégration d'un tel paradigme s'aligne avec les thèses de **Nierstrasz et al. (2021)** sur la gestion des incertitudes dans les systèmes automatisés de modernisation logicielle, qui préconisent une complémentarité homme-machine harmonieuse plutôt qu'un automatisme absolu et aveugle.

L'évaluation économique de l'outil révèle également un équilibre à atteindre entre coûts et bénéfices. Alors que la réduction significative des temps de migration grâce à l'automatisation permet des gains substantiels en efficacité opérationnelle, l'investissement initial en R&D et en formation des équipes reste conséquent. De plus, la maintenance continue est requise pour assurer la compatibilité avec l'évolution des technologies sous-jacentes, notamment les mises à jour Docker ou des environnements ciblés. Cette dimension temporelle, largement analysée par **Levointurier et al. (2020)** dans leurs modèles de coûts appliqués aux infrastructures Cloud, souligne l'importance d'un engagement stratégique sur le long terme pour maximiser le retour sur investissement lors de la transformation numérique d'anciennes applications web.

Enfin, la mise en œuvre dans un contexte cloud ou hybride, favorisant la scalabilité et la disponibilité, s'avère judicieuse au regard des exigences croissantes en termes de performances et de gestion des ressources. La flexibilité offerte par les outils d'orchestration s'appuyant sur des

standards comme Docker Compose ou DockerSwarm s'intègre parfaitement à la vision proposée, garantissant un déploiement fluide et reproductible dans divers scénarios.

En somme, cette étude apporte une contribution significative à la problématique complexe de la migration automatisée des applications web traditionnelles vers des conteneurs Docker. La réussite repose sur l'élaboration d'une chaîne intégrée d'analyse et de génération, renforcée par une prise en compte réaliste des limites et contraintes techniques et organisationnelles. Cette démarche, étayée par une validation empirique rigoureuse et une réflexion économique fine, ouvre la voie à des développements futurs orientés vers une généralisation et une industrialisation accrue de l'outil. Les résultats obtenus confirment ainsi la pertinence d'une approche hybride mariant automatisation avancée et intervention humaine ciblée, tout en soulignant la nécessité d'un pilotage stratégique pour assurer la pérennité et l'adoption durable de telles solutions dans le domaine du génie logiciel.

5.3 RECOMMANDATIONS

Bien que cet outil d'aide à la migration automatisée apporte une réponse concrète, rapide et fiable aux défis de la conteneurisation en éliminant les erreurs humaines lors de la configuration de l'écosystème PHP (extensions PDO, serveurs Nginx/Apache, dépendances Composer), l'évolution logique de ce travail nous conduit à formuler trois (03) principales recommandations :

➤ **Le développement d'un module qui prendra en compte les autres langages web traditionnels**

Notre étude de cas s'est focalisée sur l'écosystème PHP en raison de sa prédominance historique dans les architectures web traditionnelles et de sa complexité de configuration. Cependant, pour accroître l'impact de cet outil assisté, il est fortement recommandé d'implémenter de nouveaux modules d'analyse capables de prendre en charge d'autres langages et environnements patrimoniaux tout aussi présents dans l'administration et les entreprises (tels que Java/JEE, Python, ou Node.js). L'objectif est de transformer cette solution en une plateforme universelle de modernisation d'applications capables de cartographier et de conteneuriser n'importe quel système legacy, indépendamment de sa pile technologique d'origine.

➤ **Le déploiement d'une infrastructure DevOps souveraine et d'un Registre National d'images**

Pour que cette dynamique de conteneurisation s'inscrive dans une logique de sécurité et de résilience globale, il est indispensable de lier cet outil à une **infrastructure DevOps locale et souveraine (au niveau national)**. Actuellement, le stockage et le téléchargement d'images de base (PHP, MariaDB) dépendent fortement de registres tiers internationaux (comme Docker Hub).

La mise en place d'un **Registre Privé National (Private Container Registry)** hébergé localement permettrait :

- ✓ D'assurer la **souveraineté numérique** en stockant localement les images Docker générées par notre outil pour les structures étatiques conformément à la circulaire de l'ANTIC qui voudrait que tous les données soit stockés localement.
 - ✓ De garantir la **sécurité des données et du code source** en évitant l'exposition des artefacts de déploiement sur des serveurs étrangers.
 - ✓ D'optimiser drastiquement la **bande passante** et la vitesse de déploiement (opérations de docker pull et push sur le réseau local), contournant ainsi les contraintes de connectivité internet extérieure.
- **L'automatisation de la scalabilité horizontale (Auto-scaling) pilotée par la télémétrie Prometheus**

Pour transformer cette plateforme d'aide à la migration en un écosystème de production résilient et auto-adaptatif, il est fortement recommandé d'intégrer un module de **scalabilité automatique (Auto-scaling)** basé sur les métriques en temps réel. Actuellement, notre stack de supervision (Prometheus et cAdvisor) collecte et affiche la consommation des ressources, tandis qu'Alertmanager notifie l'administrateur en cas de surcharge.

L'évolution logicielle consisterait à interconnecter les alertes de Prometheus avec l'orchestrateur de conteneurs pour déclencher des actions correctives immédiates sans intervention humaine. Concrètement :

- **Définition de seuils dynamiques** : Le système surveille en permanence une ressource clé **X** (telle que la charge CPU ou l'utilisation de la mémoire RAM du conteneur applicatif).
- **Déclenchement de la règle** : Dès que la ressource **X** atteint ou dépasse un seuil critique préétabli **Y** (par exemple, une saturation CPU de 80 % pendant plus de 3 minutes sur l'application *gedeq*), Prometheus lève une alerte spécifique.

- **Réponse automatisée (Scalabilité horizontale) :** Un script d'automatisation ou un webhook intercepte cette alerte et exécute instantanément une commande d'infrastructure pour ajouter **Z** nouvelles instances (répliques) du conteneur concerné (via un mécanisme de **docker compose up --scale applicatif=Z**).

En résumé, les recommandations formulées reposent sur une dynamique d'amélioration continue, associant enrichissement technologique et renforcement du facteur humain. Ce positionnement garantit non seulement la maîtrise des complexités liées à la migration automatisée, mais aussi une intégration harmonieuse dans les pratiques industrielles existantes. L'objectif doit rester la production d'artefacts Docker fidèles et reproductibles, avec un minimum d'intervention manuelle tout en maintenant une capacité d'adaptation face à la diversité des cas d'usage rencontrés, conformément aux enjeux soulevés par la problématique centrale de cette recherche.

RÉFÉRENCES BIBLIOGRAPHIQUES

- **Boettiger, C.** (2015). An introduction to Docker for reproducible research, with examples from the R ecosystem. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79. <https://doi.org/10.1145/2723872.2723882>
- **Cito, J., Gottschalk, P., Leitner, P., Inzinger, C., & Gall, H. C.** (2015). Runtime metric cloud tracking with Docker. Dans *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE)* (Vol. 2, pp. 891–892). IEEE. <https://doi.org/10.1109/ICSE.2015.291>
- **Cito, J., Schermann, G., Wittern, J. E., Leitner, P., Khan, S. I., & Gall, H. C.** (2017a). An empirical analysis of technical debt in Dockerfiles. Dans *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ACM ESEC/FSE)* (pp. 143–153). <https://doi.org/10.1145/3106237.3106277>
- **Fowler, M., & Lewis, J.** (2014). *Microservices: a definition of this new architectural term and its ideas*. MartinFowler.com. <https://martinfowler.com/articles/microservices.html>
- **Kratzke, N.** (2018). About microservices, containers and their orchestration: Current trends and low-hanging fruits. *Cloud Computing and Services Science*, 3–23. https://doi.org/10.1007/978-3-319-94959-8_1
- **Merkel, D.** (2014). Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2. <https://dl.acm.org/doi/10.5555/2600239.2600241>
- **Pahl, C., Brogi, A., Soldani, J., & Jamshidi, P.** (2019). Cloud Container Technologies: A State-of-the-Art Review. *IEEE Transactions on Cloud Computing*, 7(3), 677–692. <https://doi.org/10.1109/TCC.2017.2702586>
- **Pahl, C., & Lee, B.** (2015). Containers and clusters as Cloud software architecture providers in DevOps. Dans *2015 IEEE 3rd International Conference on Cloud Engineering (IC2E)* (pp. 397–400). IEEE. <https://doi.org/10.1109/IC2E.2015.82>
- **Wiggins, A.** (2017). *The Twelve-Factor App*. Manifeste méthodologique pour le développement d'applications cloud-native. <https://12factor.net/fr/>

Sitographie complétée (Web & Supports Vidéos)

- **Prodan, S.** (2025). *Dockprom: A web application monitoring stack for Docker with Prometheus, Grafana, cAdvisor, NodeExporter and AlertManager* [Dépôt GitHub]. GitHub. <https://github.com/stefanprodan/dockprom>
- **Xavki.** (2019, 29 novembre). *Superviser ses conteneurs Docker avec Prometheus / Grafana* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=UD181Kf4E0>

Table des matières

DEDICACES.....	ii
REMERCIEMENTS.....	iii
LISTE DES ABREVIATIONS	iv
LISTE DES FIGURES	v
LISTE DES TABLEAUX.....	vi
RESUME.....	vii
ABSTRACT	viii
CHAPITRE 1 : INTRODUCTION GENERALE.....	1
1.1 CONTEXTE ET JUSTIFICATION	1
1.2 PROBLEMATIQUE ACTUELLE	3
1.3 QUESTIONS DE RECHERCHE	7
1.4 OBJECTIFS DE L'ETUDE	8
1.5 LES PROPOSITIONS DE RECHERCHE.....	11
1.6 INTERET DE L'ETUDE.....	13
1.7 APPROCHE METHODOLOGIQUE.....	16
1.8 SCOPE ET LIMITES DE L'ETUDE	18
1.9 PLAN DU TRAVAIL.....	20
CHAPITRE 2 : REVUE DE LA LITTERATURE	24
2.1 ANALYSE DES CONCEPTS	24
2.1.1 Le Paradigme d'isolation et de virtualisation	24
2.1.2 Analyse comparative : Virtualisation vs Conteneurisation	25
2.1.3 Écosystème et outils de la conteneurisation.....	27
2.1.4 Orchestration de conteneurs	29
2.1.5 Administration des environnements conteneurisés	30
2.2 CADRE THEORIQUE DE LA MIGRATION APPLICATIVE	31
2.2.1 Principes théoriques de la migration automatique.....	31
2.2.2 Le concept d'Orchestration de conteneurs.....	32
2.3 REVUE DE LA LITTERATURE EMPIRIQUE : OUTILS DE MIGRATION ET D'ADMINISTRATION	32
2.3.1 Analyse comparative des outils d'aide à la migration	32
2.3.2 Le concept d'Administration, de Supervision et DevOps	33
CHAPITRE 3 : APPROCHE METHODOLOGIQUE	35
3.1 TYPE D'ETUDE.....	35
3.2 ETUDE CRITIQUE DE L'EXISTANT	37
3.2.1 Analyse individuelle des solutions existantes	38
3.2.2 Synthèse des limites de l'existant et justification de notre outil	39

3.3	MODELE DE L'ETUDE	40
3.4	CONCEPTION DE L'APPLICATION.....	42
3.4.1	Le langage de modélisation UML	42
3.4.2	Présentation des diagrammes UML	42
3.4.3	Architecture physique de l'application	54
3.4.4	Architecture logique de l'application	56
3.4.5	Technologies d'implémentation.....	58
3.4.6	Administration et Supervision des conteneurs	58
CHAPITRE 4 : RESULTAT ET EXPERIMENTATION		61
4.1	EXPERIMENTATION DU MODELE.....	61
4.2	ÉTUDE DE CAS PRATIQUE ET CONTEXTUALISÉE.....	63
4.2.1	Présentation de l'Outil d'Aide à la Migration Automatisée	64
4.2.2	Analyse comparative : Migration manuelle vs Migration automatisée par l'outil développé 69	
4.2.3	Analyse d'impact sur le cycle de vie du projet	70
4.2.4	Évaluation quantitative et analyse détaillée des métriques de performance	70
4.2.5	Quelques fichiers de configuration générés par l'outil	73
4.2.6	Architecture de Supervision de l'Environnement Migré	75
4.3	COUT DE LA SOLUTION.....	76
CHAPITRE 5 : RESUME, CONCLUSION ET RECOMMANDATIONS		81
5.1	RESUME	81
5.2	CONCLUSION	83
5.3	RECOMMANDATIONS.....	85
RÉFÉRENCES BIBLIOGRAPHIQUES		88
3.5	Problematiche.....	Erreur ! Signet non défini.
3.6	Problematiche.....	Erreur ! Signet non défini.
3.7	Objectifs.....	Erreur ! Signet non défini.
3.8	Hypotheses	Erreur ! Signet non défini.
3.9	Interet.....	Erreur ! Signet non défini.
3.10	Analyse des concepts chap 2.1.....	Erreur ! Signet non défini.
3.11	Approche methodologique	Erreur ! Signet non défini.
3.12	Scope et limite.....	Erreur ! Signet non défini.
3.13	Cadre theorique.....	Erreur ! Signet non défini.
3.14	REVUE DE LA LITTÉRATURE EMPIRIQUE.....	Erreur ! Signet non défini.
https://www.youtube.com/watch?v=_UD181Kf4E0 (Superviser ses conteneurs Docker avec Prometheus/Grafana).....		
		Erreur ! Signet non défini.

